The mini coding agent: course manual

The main manual. `PLAN.md` is folded in, not left beside it.

This manual explains *why* the code in each stage looks the way it does. It is the one document a student or a teacher should read from top to bottom. `PLAN.md` supplied the concrete build contract: exact files, APIs, defaults, and acceptance criteria. That contract now lives in **Part II**, so the theory in Parts 0–E and the build instructions sit in one book.

Read one stage of the manual, then read the matching **build companion** entry in Part II before you start typing. When you finish the stage, answer the checkpoint questions at the end of the theory chapter, then do the matching **student lab** in Part IV. Teachers should read **Part III** before the first session; it contains the run sheet, the live-demo script, the factory analogy, common misconceptions, and the assessment rubric.

> **What is where.** - **Parts 0–E** — the staged theory: the mental model, the design vocabulary, the failure modes. - **Part II** — the *build companion*: PLAN.md's files, API contracts, defaults and acceptance criteria, plus a teaching note and a lab for every stage. - **Part III** — the *teacher's handbook*: pacing, run sheets, demos, the factory analogy, misconceptions, rubric. - **Part IV** — the *student workbook*: per-stage labs, break-it labs, capstones, extension roadmap. - **Part V** — *deep dives*: context engineering, tool design, evaluation, threat modelling, the production path. - **Part VI** — *reference*: configuration, CLI, payloads, file map and the final acceptance checklist.
> The code excerpts are illustrative. Where this manual and the finished implementation disagree, the implementation and its tests win. Where the manual and `PLAN.md` disagree, this manual is now the merged source of truth.
> **Source material.** The long-form explanation of the finished harness is in `../mini-coding-agent-js/explainer-harness-js.md`. The build contract came from `PLAN.md` and is reproduced, expanded and cross-referenced in Part II. The finished implementation lives in `../mini-coding-agent-js/src/`; read it alongside the manual. The non-coding **factory analogy** in Part III is adapted from `../mini-coding-agent-js/explainers/factory_example.md`.

CONTENTS

What is in this manual

Table of contents 5
0.1 What an agent is 5
0.2 The six-line agent 9
0.3 Why we build conversation first 10
0.4 The architecture you are growing 11
0.5 The ten principles of the harness 13
Stage 0: Scaffold 14
Stage 1: Terminal language (ui.js) 16
Stage 2: Config and the model client 19
Stage 3: The simplest chat loop 25
Stage 4: Conversation history 28
Stage 5: The system prompt 31
Stage 6: Path policy 34
Stage 7: Read-only file tools 37
Stage 8: Sandbox and write tools 42
Stage 9: Command execution 46
Stage 10: Long-term memory 51
Stage 11: Lifecycle hooks 56
Stage 12: The ReAct loop 61
Stage 13: Offline demo 65
Stage 14: Sessions, trimming and CLI polish 68
Stage 15: The full test suite 72
Stage 16: Documentation and final acceptance 76
A. The message protocol 79
B. The safety model in one page 80
C. Every bound and cap 81
D. Debugging by symptom 82
E. Glossary 83
F. Reading the finished code 84
Closing note 85
Prerequisites 86
How to use the companion 87
Stage 0 build: Scaffold 88
Stage 1 build: Terminal language 89

Stage 2 build: Config and model client 90
Stage 3 build: Simplest chat loop 91
Stage 4 build: Conversation history 92
Stage 5 build: System prompt 93
Stage 6 build: Path policy 94
Stage 7 build: Read-only tools 95
Stage 8 build: Sandbox and write tools 96
Stage 9 build: Command execution 98
Stage 10 build: Long-term memory 100
Stage 11 build: Lifecycle hooks 102
Stage 12 build: The ReAct loop 104
Stage 13 build: Offline demo 105
Stage 14 build: Sessions and CLI polish 106
Stage 15 build: Full test suite 107
Stage 16 build: Documentation and acceptance 108
The course checklist 110
1. What this course is and is not 111
2. Learning outcomes by phase 112
3. Three ways to pace the course 113
4. The session run sheet 114
5. The live-demo script 115
6. The factory analogy 116
7. Common misconceptions and how to correct them 119
8. The debugging clinic 120
9. Assessment rubric 121
10. Teaching the hardest ideas 122
11. Classroom, recording and CI setup 123
1. How to use the labs 124
2. Per-stage labs 125
3. Break-it-on-purpose labs 127
4. Reflection prompts 128
5. Capstone projects 129
6. Extension roadmap 131
7. Self-assessment 132
1. What exactly is a harness? 133
2. The harness stack 134
3. Context engineering 135

4. Tool design 137
5. Evaluation 139
6. Security threat model 140
7. The production path 141
8. Observability and cost 143
9. Architecture decision records 144
10. How real harnesses compare 146
11. Reading list 147
A closing note for teachers 148
A. Configuration reference 149
B. CLI and exit-code reference 151
C. Tool response reference 153
D. Hook payload reference 155
E. File and API map 156
F. Final acceptance checklist 157

Page numbers refer to the folio at the foot of each page.

Table of contents

- Part 0: The idea before the code
- 0.1 What an agent is
- 0.2 The six-line agent
- 0.3 Why we build conversation first
- 0.4 The architecture you are growing
- 0.5 The ten principles of the harness
- Phase A: A chatbot that talks
- Stage 0: Scaffold
- Stage 1: Terminal language (`ui.js`)
- Stage 2: Config and the model client
- Stage 3: The simplest chat loop
- Stage 4: Conversation history
- Stage 5: The system prompt
- Phase B: Give the chatbot hands
- Stage 6: Path policy
- Stage 7: Read-only file tools
- Stage 8: Sandbox and write tools
- Stage 9: Command execution
- Phase C: Memory and observability
- Stage 10: Long-term memory
- Stage 11: Lifecycle hooks
- Phase D: The agent loop and polish
- Stage 12: The ReAct loop
- Stage 13: Offline demo
- Stage 14: Sessions, trimming and CLI polish
- Phase E: Hardening and documentation
- Stage 15: The full test suite
- Stage 16: Documentation and final acceptance
- Appendices
- A. The message protocol
- B. The safety model in one page
- C. Every bound and cap
- D. Debugging by symptom
- E. Glossary
- F. Reading the finished code
- Part II: The build companion — PLAN.md, stage by stage
- How to use the companion

- Stage 0 build: Scaffold
- Stage 1 build: Terminal language
- Stage 2 build: Config and model client
- Stage 3 build: Simplest chat loop
- Stage 4 build: Conversation history
- Stage 5 build: System prompt
- Stage 6 build: Path policy
- Stage 7 build: Read-only tools
- Stage 8 build: Sandbox and write tools
- Stage 9 build: Command execution
- Stage 10 build: Long-term memory
- Stage 11 build: Lifecycle hooks
- Stage 12 build: The ReAct loop
- Stage 13 build: Offline demo
- Stage 14 build: Sessions and CLI polish
- Stage 15 build: Full test suite
- Stage 16 build: Documentation and acceptance
- The course checklist
- Part III: The teacher's handbook
- 1. What this course is and is not
- 2. Learning outcomes by phase
- 3. Three ways to pace the course
- 4. The session run sheet
- 5. The live-demo script
- 6. The factory analogy
- 7. Common misconceptions and how to correct them
- 8. The debugging clinic
- 9. Assessment rubric
- 10. Teaching the hardest ideas
- 11. Classroom, recording and CI setup
- Part IV: The student workbook
- 1. How to use the labs
- 2. Per-stage labs
- 3. Break-it-on-purpose labs
- 4. Reflection prompts
- 5. Capstone projects
- 6. Extension roadmap
- 7. Self-assessment
- Part V: Deep dives on harness engineering

- 1. What exactly is a harness?
- 2. The harness stack
- 3. Context engineering
- 4. Tool design
- 5. Evaluation
- 6. Security threat model
- 7. The production path
- 8. Observability and cost
- 9. Architecture decision records
- 10. How real harnesses compare
- 11. Reading list
- Part VI: Reference
- A. Configuration reference
- B. CLI and exit-code reference
- C. Tool response reference
- D. Hook payload reference
- E. File and API map
- F. Final acceptance checklist

Part 0: The idea before the code

0.1 What an agent is

Strip the branding and the product categories away from any coding agent and three things remain:

1. **A model** that can only produce text. It cannot read a file, run a test, or remember yesterday. It maps a conversation to the next message.
2. **Tools** that can actually do something: list a folder, read a file, write a file, run a command.
3. **A loop** that takes the model's text, notices when it asks for a tool, runs the tool, and hands the result back to the model as another message.

Everything else you have seen a coding agent do — planning, sub-agents, retries, permissions, skills, project rules, memory — is a layer on top of those three. The layers matter, but they only make sense once the core is visible. This course builds the core in public, then adds two layers that earn their place immediately: **memory** and **lifecycle hooks**.

A chatbot is the same program with the loop deleted:

- one model call,
- one answer,
- no way to look at the world,
- no way to react to what it saw.

The difference between "answers questions about your code" and "changes your code" is not a bigger model. It is the loop. That sentence is worth memorising before Stage 0.

The three-column mental model

PIECE	LIVES IN	SPEAKS	CAN CHANGE THE WORLD?
Model	OpenRouter	Text and tool-call JSON	No
Tools	`src/tools.js`	Strings in, strings out	Yes, through the sandbox
Loop	`src/agent.js` (from Stage 12)	Messages	No, but it decides when tools run

The model never touches the filesystem. It *names* a tool and supplies arguments. The harness validates those arguments, enforces policy, executes the handler, and returns a string. That separation is the entire security architecture, and it is why a tool call can be refused without crashing anything.

0.2 The six-line agent

Here is the whole idea, written as pseudocode:

```
messages = [system prompt, user task]

repeat up to maxSteps:
    reply = model(messages, tools)          # text in, text out
    if reply has no tool calls: return reply.content
    for each tool call:
        messages.append(tool result)        # observation
```

This is not a simplification of an agent. It *is* the agent. Every stage in this course adds plumbing around one of those lines:

- Stages 0–5 make the first `model(messages)` call possible, and make the message array meaningful.
- Stages 6–9 make the `tool result` line real, with safety around it.
- Stage 10 makes memory cross process boundaries.
- Stage 11 makes the loop observable and extensible.
- Stage 12 extracts the loop into a library that can be tested.
- Stages 13–16 prove the behaviour, polish the program, and document it.

Think, act, observe, repeat

When you watch the finished harness run, every step reads like a sentence:

```
── step 1/12 ──────────────────────────────────────
think: Creating a script, then running it to check it works.
tool → create_file {"path":"demo/hello.js","content":"..."}
result ← created demo/hello.js (72 bytes)
```

- The model **thought**: it produced text explaining its intention.
- It **acted**: it emitted a tool call.
- The world **answered**: the harness returned a string.
- The loop went round again with that answer in hand.

That is the ReAct pattern: **Re**ason, **Act**, observe. You do not need a framework to implement it. You need an array, a `for` loop, and a `fetch`.

> **Key idea.** The loop is not an implementation detail. It is the agent. Delete the `for` and you have a chatbot with tools it will never call.

0.3 Why we build conversation first

The build order in `PLAN.md` is deliberate and slightly unusual. A traditional engineering plan would build the sandbox, then the tools, then the model client, and only then let the program talk. That plan is easier to diagram and much harder to learn from, because nothing visible happens until late.

This course builds in the opposite direction:

1. **Make it talk.** Config, HTTP, one model call. You have a chatbot in Stage 3.
2. **Give it memory of the conversation.** Stage 4.
3. **Give it a job description.** Stage 5.
4. **Give it hands.** Stages 6–9, one capability at a time.
5. **Give it persistence and observability.** Stages 10–11.
6. **Formalise the loop.** Stage 12.
7. **Prove it.** Stages 13–16.

The reason is motivational and pedagogical: every stage produces a runnable program that is immediately more capable than the last. You can feel the feature. You also learn the failure modes in the order a real user meets them:

- A bad key is a Stage 2 problem.
- A model that forgets the previous turn is a Stage 4 problem.
- A model that writes the wrong file is a Stage 8 problem.
- A command that hangs is a Stage 9 problem.
- A model that loops forever is a Stage 12 problem.

Building the loop first would hide all of those behind a harness you do not yet understand.

> **Rule of the course.** Do not skip ahead. Each stage adds one concrete capability and one conceptual boundary. If the program does not do something new and useful after a stage, the stage is not finished.

0.4 The architecture you are growing

By the end, the import graph points in one direction:

```
agent.js ── THE LOOP ──────────────────┐
   ├── prompt.js     the rules + memory
   ├── heartbeat.js  still waiting     │  what one step
   ├── llm.js        one POST          │  needs
   └── tools.js      schemas + run
         ├── workspace.js  containment
         └── sandbox.js    write fence
                                       │
cli.js ── THE PROGRAM ─────────────────┘  everything a
   ├── agent.js      runTask             person needs to
   ├── config.js     .env + defaults     actually run it
   ├── memory.js     src/memory.md
   ├── demo.js       the fake model
   ├── hooks.js      named moments
   └── ui.js         every chalk call
```

The important line is the asymmetry between the halves:

- `agent.js` **is a library.** It knows nothing about flags, colours, `.env`, `process.exit`, or the terminal. It takes dependencies as parameters and returns a plain object.
- `cli.js` **is the program.** It reads flags, loads config, prints banners, builds dependencies, calls `runTask`, and turns the result into an exit code.

Before Stage 12, `cli.js` owns the conversation loop directly. We extract it only once it is complex enough to deserve its own file. That order is intentional: extracting a loop you have not yet written by hand produces a library nobody understands.

The file-by-file questions

Each file answers one question a reader will ask. Keep this table nearby as you build; it is the map of the course.

FILE	THE QUESTION IT ANSWERS	INTRODUCED
`src/ui.js`	Where is the colour? (Nowhere else.)	Stage 1
`src/config.js`	Which key, model and limits — and where did each value come from?	Stage 2
`src/llm.js`	What exactly goes over the wire?	Stage 2
`src/cli.js`	What does a person type?	Stage 3
`src/prompt.js`	What is the model told before it starts?	Stage 5
`src/workspace.js`	How is a path stopped from leaving the sandbox?	Stage 6
`src/tools.js`	What can the model do, and what happens when it asks?	Stage 7
`src/sandbox.js`	May the agent change this file at all?	Stage 8
`src/memory.js`	What does it know that survives the process?	Stage 10
`src/hooks.js`	Where can outside code get a turn, and what may it change?	Stage 11
`src/agent.js`	Where is the loop?	Stage 12

FILE	THE QUESTION IT ANSWERS	INTRODUCED
`src/heartbeat.js`	Why doesn't a 60-second request look like a crash?	Stage 12
`src/demo.js`	What runs when there is no network, no key and no model?	Stage 13

0.5 The ten principles of the harness

These principles are the through-line of the whole course. Each stage implements one or more of them. They are also the vocabulary to use when you explain your code to somebody else.

#	PRINCIPLE	WHERE YOU WILL SEE IT
1	**Tool calling is the model's only ability to act.** The schemas *are* the prompt.	`tools.js`, Stage 7
2	**The loop is the agent.**	`agent.js`, Stage 12
3	**Everything is messages.** One array, four roles, no hidden state.	Stage 4 onward
4	**Errors are observations, not crashes.** A failed tool returns text.	Stage 7 onward
5	**Exit codes close the loop.** `exit code 0` versus `1` is how the model learns whether it worked.	Stage 9
6	**Human in the loop.** Commands ask by default; hard denies cannot be overridden.	Stage 9
7	**Least privilege.** Reads are open, writes are fenced, child processes get a minimal environment.	Stages 6–9
8	**Bounded autonomy.** `maxSteps`, timeouts, truncation guard, memory caps.	Stages 2, 9, 10, 12
9	**Extension points belong at named moments.** Hooks, not edits to the loop.	Stage 11
10	**Memory is a file, not a mystery.**	Stage 10

Phase A: A chatbot that talks

Goal of the phase: a working OpenRouter chat client. No tools, no sandbox, no memory — just you and the model, with configuration you can trust and output you can read.

What you can do at the end: hold a multi-turn conversation in the terminal, with a system prompt that shapes the model's behaviour.

Stage 0: Scaffold

What this stage is really about

Stage 0 looks like ceremony: a `package.json`, a `.gitignore`, two empty folders. It is actually where you decide two design constraints that the rest of the course depends on.

Constraint 1: one runtime dependency. The project uses `chalk` for colour and nothing else. No web framework, no agent SDK, no database, no ORM. The point is not asceticism for its own sake. Every dependency you add is a layer between you and the behaviour you are trying to understand. When the model call is a `fetch` you can read, a timeout is an `AbortSignal` you can trace. When it is an SDK, a timeout is a configuration option whose implementation you cannot see.

Constraint 2: a workspace folder. The agent operates from the repository root but may only *write* inside `workspace/`. This is the containment rule that Stage 6 formalises and Stage 8 enforces. Even at Stage 0, the folder exists so that the rule has somewhere to point.

The layout and why it is flat

```
mini-coding-agent-js/
  package.json          type: module, one dependency
  .gitignore            node_modules, .env, workspace/*
  src/                  all the code
  test/                 all the tests
  workspace/            the only folder the agent may write to
  .env                  your key (never committed)
```

The `src/` folder is flat on purpose. There is no `src/core/`, `src/adapters/`, `src/domain/`. Thirteen files that import in one direction do not need a directory hierarchy, and a flat folder makes the import graph visible in a single listing. Architecture is not the same thing as folder depth.

ES modules (`"type": "module"`) are used throughout because:

- `import.meta.dirname` gives a reliable project root without `__dirname` workarounds.
- Top-level `await` is available.
- `node --test` and modern `fetch` fit naturally.
- The browser and Node module semantics are closer, which matters if you later move code.

The workspace rule

All files the agent creates or edits go in `workspace/`. The agent starts commands there. The sandbox root is there. But the agent may *read* outside it — a library, a log, a `package.json` one folder up. That read/write asymmetry is introduced conceptually here and becomes code in Stage 6.

Two details to internalise now:

- The agent does not create subfolders of `workspace/` unless a tool call asks for a path with a subfolder, or a write handler creates parents.
- Relative paths in tool calls resolve from `workspace/`, not from wherever you launched the process.

Why `.gitignore` matters in a course

The agent will write files. Some of them will be experiments, some will be wrong, and some will contain output from commands. Ignoring `workspace/*` (except `.gitkeep`) keeps your repository about the harness, not about the demo debris. Ignoring `.env` is not optional: an API key in git history is a leaked key.

Checkpoint

1. Why does the course insist on a single runtime dependency?

A. Every dependency is a layer between you and the behaviour; one dependency keeps the model call, timeout and filesystem visible.

2. What is the difference between where the process starts and where the agent may write?

A. The process starts wherever you launch it; the agent may only write inside `workspace/`.

3. Why is `workspace/.gitkeep` committed while the rest of `workspace/` is ignored?

A. `.gitkeep` preserves the empty folder in git; everything else in `workspace/` is generated and should not be committed.

Stage 1: Terminal language (`ui.js`)

What this stage is really about

Stage 1 is not "make it pretty". It is **separation of presentation from logic**, and it teaches a contract that survives all the way to the test suite: *every visual helper returns a string; the caller decides whether to print it.*

That contract has three consequences:

1. `agent.js` can accept an injected `log` function and be tested without a terminal.
2. The entire palette can be changed in one file.
3. Model output containing backslashes, `<red>`, or ANSI-looking text prints verbatim, because `chalk` styles the exact string you give it rather than parsing markup.

A console library that interprets inline tags would quietly corrupt the very output the user is reading. A model that writes a file containing `<red>` should show `<red>`, not a colour change.

The return-a-string rule

```
// good: pure presentation
export const ui = {
  title: (text) => chalk.bold.cyan(text),
  step: (step, max, width = 40) =>
    chalk.blue(`── step ${step}/${max} ${'─'.repeat(width)}`),
  think: (text) => `${chalk.cyan('think:')} ${text}`,
};

// caller decides when and where
log(ui.step(1, 12));
log(ui.think('Creating a script, then running it.'));
```

Nothing inside `ui.js` calls `console.log`. If a helper printed directly, the loop would be untestable without capturing stdout, and the CLI could not redirect output.

Result classification: the colour is derived, not reported

A tool does not say "this failed". It returns a string, and the UI infers failure:

```
const isFailure = (text) =>
  /^(ERROR|DENIED)\b/.test(text) ||
  /\bexit code (?!0\b)\d+/.test(text);
```

That small regex is a visual contract for the whole course:

- **red** means the model has a problem to solve on the next step;
- **green** means the observation is clean.

It also explains why `ui.result` prints the first line prominently and puts extra detail underneath. A `run_command` result may be many lines; the first line is the verdict, the rest is evidence.

`explainExit` and the teaching value of exit codes

Exit code `1` is not a number the model can interpret by itself. `explainExit(1)` turns it into plain English: the command ran and then reported an error; the cause is in stderr; fix that, do not just re-run.

This is a small piece of prompt engineering in the UI layer, and it matters more than it looks. A model that sees only `exit code 1` may retry the same command. A model that sees *why* the code means failure is more likely to read stderr and change approach.

`explainExit` covers the common families:

- `0` success;
- `1` general failure;
- `2` shell misuse or bad argument;
- `3` a command-specific failure (not universal);
- `126` found but not executable;
- `127` command not found;
- `128–164` terminated by a signal, computed as `128 + signal number`;
- Windows-specific codes such as `9009` (command not found in `cmd.exe`).

You do not need every mapping memorised. You need the design idea: **turn machine signals into human explanations the model can act on.**

`forbidden` must be loud and greppable

```
⛔ FORBIDDEN SAVE ../escape.txt - path escapes the workspace: ../escape.txt
```

Bold red capitals, one line, the action and the requested path. Why so loud?

- It is the one line an audience should notice.
- It is greppable in logs.
- It is distinguishable from an ordinary `ERROR`.
- It is printed by exactly one place in the codebase (`sandbox.forbid`), so there is no ambiguity about where a refusal came from.

Model output verbatim

`chalk` does not parse tags. If a tool result contains `\n`, `\t`, `C:\Users\...`, or JSON with quotes, it prints as-is. Do not "helpfully" strip or interpret model output. The model needs to see the exact characters, and so do you.

Checkpoint

1. Why should `ui.js` never call `console.log`?

A. So the caller controls printing, the loop can be tested with an injected `log`, and output can be redirected.

2. How does `ui.result` decide between green and red?

A. `isFailure` checks the first line for `ERROR` / `DENIED` or a non-zero `exit code`.

3. Why is `FORBIDDEN` deliberately different from `ERROR`?

A. `FORBIDDEN` is a policy refusal, not a tool failure; it must be loud, greppable and emitted from one place.

Stage 2: Config and the model client

This stage has two halves with two different jobs: **configuration** decides what to send and where the values came from; **the client** sends it and turns transport failures into errors the program can handle.

2a. Configuration: precedence as a teaching tool

Three sources compete for the API key and the model name:

```
--model flag          ← highest
.env (nearest file)   ← beats the shell, on purpose
exported environment  ← lowest for anything .env defines
```

The unusual part is `.env` beating the shell. Most tools do the opposite. This course deliberately inverts it because a stale `OPENROUTER_API_KEY` in `~/.bashrc` would otherwise silently outrank the key you just wrote, and you would spend an afternoon debugging a `401`. Variables that `.env` does not mention still come from the environment.

Why report provenance? The banner says `model: ... (from .env)` or `key: sk-or...aaa5 (73 chars) from shell`. When a demo goes wrong, the first question is always "which value did it actually use?" A configuration system that cannot answer that question is a configuration system that wastes your time.

Finding `.env`: nearest file wins

```
export function findEnvFile(startDir) {
  let dir = path.resolve(startDir);
  for (;;) {
    const candidate = path.join(dir, '.env');
    if (fs.existsSync(candidate) && fs.statSync(candidate).isFile()) return candidate;
    const parent = path.dirname(dir);
    if (parent === dir) return null; // filesystem root
    dir = parent;
  }
}
```

Walking upward from the project root means a single `.env` at the repository root works for a package one level down. Missing `.env` is not an error. You may be running with a shell key, or with `--demo`, which needs no key at all.

Parsing `.env` supports the forms you actually meet in the wild:

- `KEY=VALUE`;
- optional `export` prefix;
- `#` comments;
- quoted values, including spaces;
- blank values, which are ignored.

Do not use a full dotenv library for this. The parser is part of the lesson: configuration is text, and text you can read is configuration you can debug.

Fingerprinting instead of printing

```
fingerprint('sk-or-v1-abcdefghijklmnopqrstuvwxyz1234')
// => 'sk-or...1234 (37 chars)'
```

Rules:

- unset → `not set`;
- length ≤ 12 → all `*`;
- otherwise → first 5 characters, `...`, last 4, plus the length.

A fingerprint is enough to tell two keys apart on stage and useless in a screenshot. Never print the whole key, even in "debug" mode. The habit is the point.

Model normalisation: making a helpful guess visible

OpenRouter requires `vendor/model`. A `.env` carried over from another project usually contains a bare name such as `gpt-5` or `claude-sonnet-4.5`. Rather than failing, the config maps prefixes:

PREFIX	VENDOR
`deepseek`	`deepseek`
`claude`	`anthropic`
`gpt`, `o1`, `o3`, `o4`, `chatgpt`	`openai`
`gemini`	`google`
`grok`	`x-ai`
`llama`	`meta-llama`
`qwen`	`qwen`
`mistral`, `mixtral`	`mistralai`

Two rules keep the magic honest:

1. If the name already contains `/`, it is left alone.
2. If normalisation changed the value, the banner says so in a dim note.

A helpful guess that is invisible is a future bug. A helpful guess that announces itself is a convenience.

`loadConfig` returns more than values

The config object should carry both the resolved values and their sources:

```
{
  apiKey, apiKeySource,        // 'env' | 'shell' | 'unset'
  model, modelSource,          // 'flag' | 'env' | 'default' | 'normalised'
  envFile,                     // path or null
  commandPolicy,               // 'ask' | 'allow' | 'deny'
  maxSteps,
  requestTimeoutMs,
  workspaceRoot,
  memoryFile,
  memoryMaxEntries,
  memoryPromptChars,
}
```

The CLI does not have to reconstruct any of this. The library returns facts, and the program prints them.

Defaults as a design statement

SETTING	DEFAULT	WHY
Model	`deepseek/deepseek-v4.1-flash-1017`	Cheap, capable, available; the course is not a model review
`CMD_POLICY`	`ask`	Safe by default; the human is the last line of defence
`MAX_STEPS`	`12`	Enough to look, edit, run, and recover; small enough to bound cost
`REQUEST_TIMEOUT_MS`	`90000`	Slow providers exist; a hung socket does not
`WORKSPACE_DIR`	`workspace`	Containment has a name
`MEMORY_FILE`	`src/memory.md`	Beside the code, outside the sandbox
`MEMORY_MAX_ENTRIES`	`60`	The file cannot grow without limit
`MEMORY_PROMPT_CHARS`	`1200`	The injected digest cannot eat the context window

Defaults are not arbitrary. Every one of them is a bound or a safety posture, and later chapters explain the mechanism behind each.

2b. The model client: one POST

There is no SDK in this project, on purpose. A tool-calling request to an OpenAI-compatible endpoint is a `POST` with three interesting fields: `model`, `messages`, `tools`. The whole client is roughly 66 lines.

```
export async function chat({ apiKey, model, messages, tools, signal, timeoutMs = 90_000 }) {
  if (!apiKey) throw new Error('OPENROUTER_API_KEY is not set');

  const timeout = AbortSignal.timeout(timeoutMs);
  const abort = signal ? AbortSignal.any([signal, timeout]) : timeout;

  const res = await fetch('https://openrouter.ai/api/v1/chat/completions', {
    method: 'POST',
    signal: abort,
    headers: {
      Authorization: `Bearer ${apiKey}`,
      'Content-Type': 'application/json',
      'X-Title': 'min-coding-js',
    },
    body: JSON.stringify({ model, messages, tools, max_tokens: 2048 }),
  });

  const text = await res.text();
  if (!res.ok) {
    const hint = res.status === 401 ? ' (bad or revoked key)'
      : res.status === 402 ? ' (out of credits)' : '';
    throw new Error(`OpenRouter ${res.status}${hint}: ${text.slice(0, 400)}`);
  }
  return JSON.parse(text);
}
```

The two timeout layers

`AbortSignal.timeout(timeoutMs)` is the entire liveness story at the transport layer. A socket that never answers becomes an error after 90 seconds instead of a hung agent.

`AbortSignal.any([signal, timeout])` composes two reasons to stop:

- the caller's cancellation signal (for Ctrl-C or tests), and
- the timeout.

`AbortSignal.any` is why Node 20 is a prerequisite. Without it you would have to wire up event listeners and remember to remove them.

The heartbeat in Stage 12 covers the same worry at the human layer: `… still waiting for the model (25s)`. One is about correctness; the other is about perception. You need both.

Failures here throw; failures in tools do not

This is the most important distinction in the stage:

- A bad key, a `402`, a dropped connection, a non-JSON body: the model cannot fix these. The run stops, the CLI exits `1`, and the human fixes the configuration.
- A failed edit, a missing file, a denied command: the model *can* fix these. They come back as text observations and the loop continues.

Ask yourself at every error site: **can the model do anything about this?** If yes, return a string. If no, throw.

Error handling details worth copying

- **Read the body as text first, then parse.** `res.json()` throws an unhelpful error when the body is HTML from a proxy. `res.text()` lets you include a truncated preview in the error.
- **Truncate the preview** (400 characters). Error bodies can be large and can contain the request.
- **Name the status and the likely cause.** `401 (bad or revoked key)` and `402 (out of credits)` save a support round-trip.
- **Handle a `.error` field in an otherwise-200 response.** Some providers return an error object with HTTP 200. Check for it.
- **Timeout message is explicit:** `request timed out after <seconds>s`. Do not surface `AbortError` unchanged.

`sanitizeAssistant`: do not echo the scratchpad

Providers return extra fields on the assistant message: `reasoning`, `reasoning_content`, refusal blocks, annotations. Echoing them back re-sends the model's private thinking on every subsequent step — tokens you pay for, and a context that fills with stale reasoning.

```
export function sanitizeAssistant(message) {
  const out = { role: 'assistant', content: message.content ?? '' };
  if (Array.isArray(message.tool_calls) && message.tool_calls.length > 0) {
    out.tool_calls = message.tool_calls;
    if (message.content == null) delete out.content;
  }
  return out;
}
```

Two subtleties:

1. **Keep only `role`, `content`, `tool_calls`.** Everything else is provider-specific.
2. **Delete `content` when it is null and there are tool calls.** An empty string is a valid content field; `null` alongside tool calls is not, for some providers. The conditional delete keeps the message valid without inventing a value.

Why `max_tokens` is small

`max_tokens: 2048` is deliberately small: it keeps each step cheap and keeps whole files out of a single reply. The cost is that long answers get cut off — which is exactly what the truncation guard in Stage 12 is for. A small limit plus a guard is better than a large limit plus a hung context window.

Checkpoint

1. Why does `.env` beat an exported shell variable in this project?

A. A stale shell key must not silently outrank the project's key; `.env` is the project's explicit choice.

2. What does `fingerprint` protect against, and why is a length included?

A. It lets you distinguish keys without exposing them; the length helps identify a truncated or padded value.

3. Why does `chat()` throw on a `401` but `runTool()` return an `ERROR` string for a bad edit?

A. A `401` is not fixable by the model; a bad edit is.

4. What breaks if you echo `reasoning_content` back to the provider?

A. It re-sends private reasoning, costs tokens, and fills context with stale thinking.

5. What happens when the caller passes a signal and the timeout fires first?

A. `AbortSignal.any` aborts on whichever fires first; the error should explain which.

Stage 3: The simplest chat loop

What this stage is really about

Stage 3 is the first time the program says something useful. It is also the first time you wire three layers together:

- **config** (`config.js`) supplies the key, model and timeout;
- **transport** (`llm.js`) sends one request;
- **presentation** (`ui.js`) prints the answer.

The goal is immediate gratification. Resist the urge to add tools, memory, hooks, or the full loop. This stage is one task, one model call, one answer.

The shape of the program

```
async function main() {
  const opts = parseArgs(process.argv.slice(2));
  if (opts.help) return usage();

  const config = loadConfig({ model: opts.model });
  const task = opts.task ?? await askTask();

  if (!task) {
    console.error('no task given');
    process.exitCode = 2;
    return;
  }

  const messages = [
    { role: 'system', content: 'You are a helpful assistant.' },
    { role: 'user', content: task },
  ];

  const reply = await chat({
    apiKey: config.apiKey,
    model: config.model,
    timeoutMs: config.requestTimeoutMs,
    messages,
  });

  console.log(ui.answer(reply.choices?.[0]?.message?.content ?? ''));
}
```

Everything here is deliberately simple. The interesting teaching points are in the edges.

Reading user input with `readline`

`askTask({ input, output })` is a small readline loop:

- Prompt: `task ›`, with the label green and the arrow white.
- Trim the raw input.
- Blank lines re-ask rather than submitting an empty task.
- Ctrl-C returns `null` (do not crash with an unhandled `SIGINT`).

- Use `for await (const raw of rl)` rather than a single `question()` call, so multi-line chunks and pasted input behave predictably.

The last point is a real-world detail. A model may not care, but a human pasting a multi-line task does. The prompt loop is part of the UX, not an afterthought.

Flag parsing without a library

```
function parseArgs(argv) {
  const opts = {};
  const rest = [];
  for (let i = 0; i < argv.length; i += 1) {
    const arg = argv[i];
    if (arg === '--help' || arg === '-h') opts.help = true;
    else if (arg === '--model') { i += 1; opts.model = argv[i] ?? ''; }
    else if (arg === '--max-steps') { i += 1; opts.maxSteps = Number(argv[i]); }
    else rest.push(arg);
  }
  return { opts, task: rest.join(' ') || undefined };
}
```

Why hand-rolled?

- The flag set is small and stable.
- Every parser library is another dependency and another failure mode.
- A course should show you the shape of argument parsing, not hide it.

Two rules to preserve:

1. `--help` must work with no API key. It is documentation, not a network operation.
2. Everything that is not a flag becomes the task string. That is why `node src/cli.js say hello` does the obvious thing.

Exit codes are part of the interface

Even at Stage 3, decide what failure means:

- **0**: a reply was printed.
- **1**: the request failed (network, key, credits, timeout).
- **2**: no task and stdin is not a terminal.

`2` deserves explanation. If you run the CLI in a CI job with no stdin, the prompt cannot be answered. Printing a hint and exiting `2` is better than hanging forever waiting for input. It also makes the program scriptable: a non-TTY caller learns immediately that it must pass a task argument.

The "one call" limitation

After Stage 3 the program has no memory. If you ask "what did I just say?", the model cannot know, because the `messages` array was created fresh for this request and thrown away afterwards. That is not a model failure. It is an architectural fact: **the conversation is the array, and the array is local to the call.** Stage 4 fixes it.

Checkpoint

1. Why does `--help` need to work without a key?

A. `--help` is documentation; requiring a key to read usage is hostile.

2. Why do we return `null` on Ctrl-C instead of letting the process throw?

A. Ctrl-C is a normal user action, not an exception; returning `null` lets the loop exit cleanly.

3. What does exit code `2` mean, and why is it different from `1`?

A. `2` means no task/no TTY; `1` means the request failed. They need different fixes.

4. Where does the assistant's reply live in the response object?

A. In `reply.choices[0].message.content` (and possibly `tool_calls`).

Stage 4: Conversation history

What this stage is really about

Stage 4 introduces **session state**. The assistant remembers what you just said because *you* keep the messages and send them again. The model has no memory between requests; the harness does.

This is the first appearance of the most important sentence in the course:

> **Messages are the memory.**

The `messages` array is the agent's entire working state. There is no database, no scratchpad file, no summary of what happened earlier. Delete the array and the agent has no idea it ever did anything.

The four roles

ROLE	WHO WRITES IT	WHAT IT CARRIES
`system`	you	The job description: behavioural rules, memory digest (later).
`user`	you	The task, or a corrective nudge (the truncation message is also a `user` turn).
`assistant`	the model	`content` (its thinking, printed as `think:`) and possibly `tool_calls` — its intentions.
`tool`	the harness	One string per call, tagged with the `tool_call_id` it answers. The only way the model learns what happened.

At Stage 4 there are only the first three. `tool` arrives in Stage 7, but the table is worth learning now because it explains why the array is structured the way it is.

The session loop

```
const messages = [{ role: 'system', content: SYSTEM_PROMPT }];

while (true) {
  const task = await askTask();
  if (task === null || task === '/quit' || task === '/exit') break;
  if (task === '/new') {
    messages.length = 0;
    messages.push({ role: 'system', content: SYSTEM_PROMPT });
    continue;
  }
  if (!task) continue;

  messages.push({ role: 'user', content: task });
  const reply = await chat({ ...config, messages });
  const answer = reply.choices?.[0]?.message?.content ?? '';
  messages.push({ role: 'assistant', content: answer });
  console.log(ui.answer(answer));
}
```

Three behaviours to notice:

1. **Every turn appends `user` → model call → append `assistant`.** The next request contains the whole conversation, so follow-up questions work.

2. `/new` **clears the conversation but not the process.** The system prompt is re-added; the workspace and (later) memory are untouched.
3. **The loop is still in** `cli.js`**.** That is correct for now. We extract it in Stage 12, after it has grown tools and hooks.

Why `/new` is not `/reset`

`/new` clears the conversation. It does not delete files, clear memory, or reset configuration. This distinction matters more and more as the course progresses:

- A conversation is *this sitting*.
- The workspace is *the project*.
- Memory is *what survives the process*.

Three different lifetimes, three different commands or files. Confusing them is a common source of "why did it forget?" and "why did it remember that?" bugs.

One-shot versus interactive

A task passed as an argument stays one-shot: run it, print the answer, exit. A task typed at the prompt starts a session. This is not just UX; it is what the demo and the tests want. A one-shot program is composable in a shell pipeline. A session is for a human exploring.

The session summary at exit (`session over - N turn(s)`) is a small observability feature: it tells you what happened without reading the transcript.

The context-window seed

Every message you send costs tokens. A long session grows the array until the provider rejects it or your bill becomes surprising. Stage 14 addresses this with `trimHistory`, but the seed is planted here: **history is a resource with a cost.** At Stage 4, just notice that a 50-turn conversation is 50 times the input of a one-shot request.

Checkpoint

1. Why does the model "remember" the previous turn if it has no memory?

A. The harness keeps the `messages` array and sends it again; the model itself has no memory.

2. What exactly does `/new` clear, and what does it leave alone?

A. It clears the conversation; the workspace and memory survive.

3. Why is the loop still in `cli.js` at this stage?

A. The loop is still simple enough to live in the CLI; extracting it now would hide the mechanics.

4. What is the difference in lifetime between `messages` and `workspace/`?

A. `messages` dies with the process; `workspace/` persists on disk.

Stage 5: The system prompt

What this stage is really about

Stage 5 separates **behavioural rules from control flow**. The loop should not contain the model's job description; that belongs in a prompt builder. The loop should not know whether memory exists; the prompt builder decides what to include.

Why a module instead of a string literal

A string literal in `cli.js` is fine for one rule. It becomes a problem when:

- the rules depend on which tools exist;
- the prompt contains a memory digest;
- tests need to assert the rules are present;
- a log line must report how many memories were injected.

`prompt.js` centralises "what the model is told" in one place. That is the same design instinct as `ui.js` centralising colour: one decision, one location, no drift.

The rules a coding agent needs

At Stage 5 the tools do not exist yet, so the prompt is only the behavioural contract. The plan's rules cover:

1. **Look before you write.** Read the file before editing it.
2. **Prefer `create_file` for new files**, `edit_file` for small changes, `write_file` for whole-file replacement.
3. **Copy `old_string` exactly**, including whitespace. The edit tool matches literally.
4. **Run what you wrote.** A file that has not executed is a hypothesis.
5. **Do not invent tool results.** The harness supplies them; you must wait for them.
6. **Be concise.** The user is watching a terminal.
7. **One tool call per step.** Interleaved edits are hard to audit.
8. **Remember useful facts**, not task progress. (Meaningful once Stage 10 exists.)

Why these rules and not a personality? Because each one prevents a concrete failure you will see in the demo:

- Without rule 1, the model edits a file it has not read and guesses the snippet.
- Without rule 3, `edit_file` returns "not found" and the model wastes a step.
- Without rule 4, the model claims success without proof.
- Without rule 5, the model hallucinates output and the loop loses its grounding.
- Without rule 7, a single reply can contain three interleaved edits that are impossible to follow live.

This is prompt engineering as ordinary code: rules in a string, composed by a function, tested like any other behaviour.

`buildSystemPrompt({ memory, log })`

```
export function buildSystemPrompt({ memory = null, log = () => {} } = {}) {
  const rules = systemRules(TOOL_NAMES);
  const digest = memory?.digest?.() ?? '';
  if (!digest) return rules;
  log(ui.dim(`memory ← ${memory.label} - ...`));
  return `${rules}\n\n${digest}`;
}
```

At Stage 5 there is no memory, so the digest is always empty. The signature is deliberately future-proof: the loop can call `buildSystemPrompt({ memory })` from the first day and never change the call site.

The log line matters. When memory is added, the banner will say `memory ← src/memory.md - 5 memories added to the system prompt`. Observability is built in at the point where the decision is made, not reconstructed later from the transcript.

Tools are passed but empty

`systemRules(tools = TOOL_NAMES)` accepts a tool list so the rules can mention only what exists. At Stage 5, `cli.js` passes `[]` because no tools are built. At Stage 7, the list grows to the read-only tools. At Stage 8, the write tools. At Stage 9, `run_command`.

This is why a generic "you are a coding agent with these tools" prompt is inferior: it names tools the model cannot call, which teaches it to hallucinate calls. **The prompt should describe the program the model is actually attached to.**

The system prompt is rebuilt, not carried

This is a subtle but crucial habit. In Stage 12, the system prompt will be rebuilt for every task:

```
const messages = [
  { role: 'system', content: buildSystemPrompt({ memory, log }) },
  ...history.filter((m) => m.role !== 'system'),
  { role: 'user', content: task },
];
```

Why rebuild?

- Memory may have been written by the previous task; a carried system message would be stale.
- The tool list may have changed between runs.
- `/new` should restore the prompt without duplicating it.

At Stage 5, simply observe that the system prompt is the *first* message on every turn and that `/new` keeps it. The rebuild pattern becomes explicit in Stage 14.

Checkpoint

1. Why keep the model's behavioural rules out of `cli.js`?

A. Behaviour is a separate decision from control flow; tests and memory need one place to build the prompt.

2. Why pass an empty tool list at Stage 5 instead of listing all eight tools optimistically?

A. Naming tools that do not exist teaches the model to hallucinate calls.

3. What are the two failure modes rule 3 (copy `old_string` exactly) prevents?

A. It prevents a failed exact match and wasted steps; it also prevents editing the wrong occurrence.

4. Why is the system prompt rebuilt for each task rather than appended to history?

A. Rebuilding lets memory saved by the previous task appear in the next task's prompt and avoids stale/duplicate system messages.

Phase B: Give the chatbot hands

Goal of the phase: the model can list, read, create, edit, delete and run things — with a fence around writes and a human in the loop for commands.

What you can do at the end: ask the agent to create a script, run it, edit it, and verify the edit, while a write outside `workspace/` is refused.

This is the phase where the program stops being a chat client and becomes an agent. It is also the phase where safety stops being theoretical.

Stage 6: Path policy

What this stage is really about

Stage 6 is the foundation of agent safety, and it begins with a sentence worth putting on a wall:

> **Reads are open; writes are fenced.**

Why asymmetric?

- An agent that cannot read a library, a log, a `package.json` one folder up, or a doc outside its own folder is useless. Reading is how it learns context.
- A write that leaves the workspace is an incident. It can overwrite a configuration, drop a file on the Desktop, or modify something the user did not offer.

So `workspace.resolve()` is strict and used for every write. `workspace.resolveRead()` is permissive and used for every read.

Why `path.resolve` plus `startsWith` is not enough

The naive containment check is:

```
const abs = path.resolve(root, requested);
if (!abs.startsWith(root)) throw new Error('escape');
```

This fails on Windows in several ordinary-looking ways:

REQUEST	WHY THE NAIVE CHECK FAILS
`C:\Windows\x.txt`	`path.resolve(root, 'C:\\Windows\\x.txt')` returns the absolute path; `startsWith(root)` is false, so it may be caught — but drive-relative forms are subtler.
`\\server\share\x`	UNC paths bypass drive-rooted containment entirely.
`~/Desktop/x.txt`	`~` is not expanded by `path.resolve`; a later expansion could point anywhere.
`%USERPROFILE%\x`	Environment expansion is not traversal, so `..` checks miss it.
`Desktop\notes.txt`	If the cwd is inside a user profile, this resolves happily to a real Desktop.
`../../secrets.txt`	Caught late by containment, but after other checks have been bypassed.
`link-to-home\x`	A junction or symlink passes a string check; the real target may be outside.

The lesson: **containment is a policy, not a string comparison.** You enumerate the escape shapes you know, expand what the shell would expand, and resolve symlinks before comparing.

The policy, in order

`workspace.resolve(requested)` rejects:

1. empty or non-string values;
2. absolute paths (`/foo`, `C:\foo`);
3. drive-relative paths (`C:foo`);

4. UNC paths (`\\server\share`, `//server/share`);
5. `~`, `$`, `%` prefixes;
6. a first segment that looks like a home or system folder (`users`, `windows`, `program files`, `desktop`, `documents`, `downloads`, `appdata`, `onedrive`, `library`, `system32`, and similar).

Then it expands variables **before** containment checks:

- `~` → home directory;
- `%VAR%` → Windows environment variable;
- `${VAR}` and `$VAR` → POSIX-style environment variable.

Then it resolves the path against the root and checks containment. The order matters: expanding after the check would let `%USERPROFILE%\x` slip through.

Symlink hardening: `realResolve`

A junction inside the workspace can point outside it. A string check sees `workspace/link-to-home/x`, which looks contained. The real target is `C:\Users\me\x`.

`realResolve` handles this:

1. Take the absolute path.
2. Walk up to the deepest ancestor that actually exists.
3. Resolve that ancestor with `fs.realpathSync`.
4. Re-append the not-yet-created tail.
5. Re-check containment against the real path.

Why "deepest existing ancestor"? Because the file you are about to create does not exist yet, so you cannot `realpath` it. The closest existing parent is the best available truth.

This is defense in depth: even if a link is created after the check, the next check resolves the link's target. It is not perfect against a race, but it closes the obvious hole.

Why `PathError`

A dedicated error class lets the sandbox distinguish "the path policy refused this" from "an unexpected I/O error happened". The sandbox converts `PathError` into `ForbiddenError` with an action and a message; everything else is a bug and should surface.

This is a recurring pattern: **classify errors by who can fix them.** A `PathError` is a policy decision; an `EACCES` is an environment problem; a `TypeError` is a programming bug.

`rel` is the user-facing path

`resolve` returns both `abs` and `rel`:

- `abs` is what the filesystem handler uses;
- `rel` is what the model and the user see.

`rel` is always `/`-separated, even on Windows, and the workspace root is `"."`. Why normalise? Because model output, logs and tests should not vary by platform. A result that says `demo/hello.js` is readable everywhere; `demo\hello.js` is not.

Checkpoint

1. Why are reads open but writes fenced?

A. Reads are how the agent learns context; writes are how it causes incidents.

2. Why must variable expansion happen before containment checks?

A. Expansion changes the effective path; checking before expansion lets `%VAR%`/`~` slip through.

3. What does `realResolve` protect against that a string check does not?

A. A symlink or junction can point outside the workspace while its name looks contained.

4. Why is `rel` normalised to forward slashes?

A. Cross-platform consistency in logs, results and tests.

5. What error class should a refused path raise, and why not a generic `Error`?

A. `PathError` separates a policy refusal from an unexpected I/O error so the sandbox can convert it.

Stage 7: Read-only file tools

What this stage is really about

Stage 7 introduces the first tools the model can invoke, and with them the core protocol of tool calling:

- a **schema** the model reads;
- a **dispatcher** that validates and runs;
- a **string observation** the model reads next.

This is the stage where "everything is messages" becomes concrete. The model emits JSON; the harness returns a string; the string becomes a `tool` message; the loop continues.

Tool calling is the model's only ability to act

The model never touches the filesystem. It emits something like:

```
{
  "id": "call_1",
  "type": "function",
  "function": {
    "name": "read_file",
    "arguments": "{\"path\":\"demo/hello.js\",\"limit\":50}"
  }
}
```

The harness:

1. reads `name`;
2. parses `arguments` (a JSON *string*, not an object);
3. validates types and required fields;
4. dispatches to a handler;
5. returns a string;
6. appends `{ role: 'tool', tool_call_id, content }`.

The model sees the result on the next request. That is the whole mechanism.

The schema is the prompt

When the model chooses a tool it reads nothing but these descriptions. That makes them the highest-leverage text in the codebase — and the first place to fix a model that keeps picking the wrong tool.

```
{
  type: 'function',
  function: {
    name: 'create_file',
    description: 'Create a NEW file with the given content. Fails if the file already exists - use edit_file or
write_file for existing files.',
    parameters: {
      type: 'object',
      properties: {
        path: { type: 'string', description: 'File path relative to the workspace root' },
        content: { type: 'string', description: 'Full file content' },
      },
      required: ['path', 'content'],
      additionalProperties: false,
    },
  },
}
```

Note what the description does:

- states the **precondition** ("fails if the file already exists");
- names the **alternative** ("use edit_file or write_file");
- keeps the parameter descriptions short and concrete.

`additionalProperties: false` discourages invented arguments. It is not a security boundary (the dispatcher validates anyway), but it is a prompt signal.

Validation before dispatch

```
function validate(name, rawArgs) {
  const schema = SCHEMAS_BY_NAME[name];
  let args;
  try {
    args = typeof rawArgs === 'string' ? JSON.parse(rawArgs) : rawArgs;
  } catch {
    return { error: `ERROR: ${name} arguments are not valid JSON` };
  }
  // type checks, integer checks, required fields
  return { args };
}
```

Why validate?

- Models send `"limit": "20"` instead of `20`. A type error two frames down is confusing; an immediate `ERROR: "limit" must be an integer, got string` is actionable.
- Models omit required fields. Say which one.
- Models invent arguments. Reject unknown keys when `additionalProperties` is false.

The error string is itself a prompt. It should say what went wrong and what to do, not just "invalid".

A handler returns a string, always

This is the fourth principle: **errors are observations, not crashes.**

Three rules hold for every tool:

1. **Return a string.** Success, denial, unexpected exception — everything is text the model reads on the next step.
2. **Parse and type-check arguments before dispatch.** Do not let a bad type become a crash.
3. **Unknown names are survivable.** A hallucinated tool name gets a helpful list and the loop continues.

A thrown error would escape the loop and end the run. A returned `ERROR: …` is an observation the model can act on. The distinction is not cosmetic; it is the difference between a one-step repair and a dead session.

`list_files`: bounded and sorted

A recursive listing can explode. The handler bounds it:

- `path` defaults to `'.'`;
- `depth` defaults to `3`, max `6`;
- skip `.git`, `node_modules`, `.venv`, `__pycache__`;
- directories print `<rel>/`, files print `<rel> (<size>b)`;
- stop at `depth` and `200` lines;
- end with `<dirs> folder(s), <files> file(s) under <start.rel>`.

Why this shape?

- **Sorted** output is deterministic, which makes tests and human reading easier.
- **Skipping dependency folders** keeps the answer about the user's code, not `node_modules`.
- **A summary line** tells the model it saw a complete or truncated view.
- **A line cap** prevents a giant listing from consuming the context window.

Missing folder → `ERROR: no such folder: <rel>`. That is a valid observation, not a crash.

`read_file`: numbered, paged, honest about truncation

Parameters:

- `path` (required);
- `offset` (default `1`);
- `limit` (default `200`, max `2000`).

Behaviour:

- missing file → `ERROR: no such file: <rel>`;
- directory → `ERROR: <rel> is a folder - use list_files`;
- header: `"<rel> (<total lines>)"`;
- numbered lines right-aligned to width 4 followed by `|`;
- append `"\n... [<n> more lines]"` when truncated;
- clamp `offset` to at least `1`, `limit` to `1..2000`.

Why line numbers? Because the model will quote fragments back to `edit_file`, and line numbers make it possible to talk about location. Why a total line count? Because "you are seeing 50 of 800 lines" changes the model's strategy. Why a truncation marker? Because silently showing the first 200 lines teaches the model that the file is 200 lines long.

The manual tool loop in `cli.js`

At Stage 7 the loop is still in the CLI, and it is deliberately manual:

```
let reply = await chat({ ...config, messages, tools: TOOL_SCHEMAS });

while (reply.choices[0].message.tool_calls?.length) {
  const message = reply.choices[0].message;
  messages.push(sanitizeAssistant(message));

  for (const call of message.tool_calls) {
    console.log(ui.tool(call.function.name, call.function.arguments));
    const result = await runTool(call.function.name, call.function.arguments, ctx);
    console.log(ui.result(result));
    messages.push({
      role: 'tool',
      tool_call_id: call.id,
      content: result,
    });
  }

  reply = await chat({ ...config, messages, tools: TOOL_SCHEMAS });
}
```

This is the ReAct pattern in about twenty lines. It teaches the shape before Stage 12 formalises it. The important details:

- `tool_calls` **is an array.** The plan asks for one call per step in the prompt, but the protocol allows several; dispatch them in order.
- **Every** `tool_call_id` **must get a matching** `tool` **message.** A provider rejects a conversation with an unanswered call.
- **The result is a string, even when it is an error.** Append it anyway.
- **Append the assistant message before the tool results.** The order is: assistant intention, then observations.

Why reads outside the workspace are allowed

`read_file` uses `resolveRead`, so `../package.json` works. That is a feature, not an oversight: the agent often needs to understand the package it lives in. The fence is about writes.

The asymmetry has a teaching benefit: in the demo, the model reads `../package.json` immediately after a write escape is refused. The audience sees the policy boundary, not a blanket "no".

Checkpoint

1. Why must a tool handler return a string rather than throw?

A. A throw ends the run; a string lets the model recover on the next step.

2. Why is `arguments` a JSON string, and what happens if you pass an object?

A. The protocol defines `arguments` as a JSON string; passing an object means the dispatcher must normalise it.

3. Why does `read_file` include a total line count and a truncation marker?

A. They tell the model it saw a truncated view and that the file is larger.

4. What happens if the model calls an unknown tool?

A. It gets a helpful list of available tools and the loop continues.

5. Why is the tool loop still in `cli.js` at this stage?

A. It is still teaching the ReAct shape; formal extraction comes in Stage 12.

Stage 8: Sandbox and write tools

What this stage is really about

Stage 8 adds **defense in depth** around writes, and it establishes the one place in the codebase that prints `FORBIDDEN`. It also teaches the honest limits of a sandbox built out of path checks.

There are two layers here, and they answer different questions:

- `workspace.js`: *does this path stay inside the root?*
- `sandbox.js`: *may the agent change this file at all?*

The sandbox delegates path policy to the workspace, converts refusals into a loud, typed error, and centralises the count of refusals.

Why a separate sandbox layer?

You could call `workspace.resolve()` directly in every write handler. Why add a wrapper?

1. **One place to count refusals.** `sandbox.refusals` is a useful statistic and a test hook.
2. **One place to print `FORBIDDEN`.** If three handlers each format their own refusal, the output drifts and the tests get brittle.
3. **One place to encode action semantics.** `save`, `edit`, `delete` are different actions with different rules (for example, deleting the root is always refused).
4. **One place to convert `PathError` into `ForbiddenError`.** The tool layer can then return `FORBIDDEN: ...` consistently.

`ForbiddenError` carries context

```
export class ForbiddenError extends Error {
  constructor(action, requested, reason) {
    super(`${action.toUpperCase()} ${requested} - ${reason}`);
    this.name = 'ForbiddenError';
    this.code = 'FORBIDDEN';
    this.action = action;
    this.requested = requested;
    this.reason = reason;
  }
}
```

Why fields as well as a message?

- The tool layer can return `FORBIDDEN: ${err.message}` without re-parsing a string.
- Tests can assert `err.code === 'FORBIDDEN'` rather than matching prose.
- The UI can print a structured line.

`check`: the single gate

```
check(action, requested) {
  if (!ALLOWED.has(action)) throw new TypeError(`unknown sandbox action "${action}"`);

  let target;
  try {
    target = this.workspace.resolve(requested);
  } catch (err) {
    if (err instanceof PathError) this.forbid(action, requested, err.message);
    throw err;
  }

  if (action === ACTIONS.DELETE && target.rel === '.') {
    this.forbid(action, requested, 'the sandbox root itself cannot be deleted');
  }
  return target;
}
```

Notice the two different failure modes:

- An **unknown action** is a programming bug → `TypeError`. It should not be caught as a refusal.
- A **path policy violation** is an expected refusal → `ForbiddenError`, printed and returned to the model.

This distinction recurs: expected failures are observations; unexpected failures are bugs.

`save`, `remove`, and the root guard

- `save(requested, content)`: `check(SAVE)`, create parents, write, return `{ existed, bytes }`.
- `remove(requested)`: `check(DELETE)`, `rmdirSync` for empty directories, `unlinkSync` for files; missing returns `{ missing: true }`.
- Deleting the sandbox root itself is refused.
- `summary()` returns `"save, edit, delete - only inside <root>"`.

Why refuse the root? Because `delete_file(".")` would remove the entire workspace, which is never what a user wants from a single tool call. It is a cheap guard against a catastrophic mistake.

Write tools: three different jobs

The three file-writing tools are not redundant. Each encodes a different intent and a different safety property:

TOOL	INTENT	SAFETY PROPERTY
`create_file`	Make a new file	**Fails if it already exists** — forces the model to edit rather than clobber.
`write_file`	Replace a whole file	Explicitly allowed to overwrite; creates parents.
`edit_file`	Change a fragment	Exact match; refuses ambiguity.

Why does `create_file` refuse to overwrite? Because "create" should not silently destroy. A model that wants to replace a file should say so with `write_file`, and the transcript then shows the intent. A model that re-runs a script and calls `create_file` gets an error it can recover from, rather than wiping a user's work.

`edit_file`: the interesting one

Seven of the eight tools are obvious. `edit_file` is where an agent's output is most likely to be subtly wrong, and where good harness design turns that into a recoverable moment.

```
edit_file(ws, args) {
  const target = ws.resolve(args.path);
  if (!fs.existsSync(target.abs)) return `ERROR: no such file: ${target.rel}`;

  const oldStr = String(args.old_string);
  const newStr = String(args.new_string);
  if (oldStr === '') return 'ERROR: old_string must not be empty';

  const source = fs.readFileSync(target.abs, 'utf8');
  const count = source.split(oldStr).length - 1;

  if (count === 0) {
    return `ERROR: old_string not found in ${target.rel}. Read the file and copy the snippet exactly, including
whitespace.`;
  }
  if (count > 1 && !args.replace_all) {
    return `ERROR: old_string appears ${count} times in ${target.rel}. Add surrounding context to make it unique, or
pass replace_all: true.`;
  }

  const updated = args.replace_all
    ? source.split(oldStr).join(newStr)
    : source.replace(oldStr, () => newStr); // function form: no $1 surprises

  fs.writeFileSync(target.abs, updated, 'utf8');
  return `edited ${target.rel}: ${args.replace_all ? count : 1} replacement(s), first at line ${lineNo}`;
}
```

Every failure mode, and why it is a message

SITUATION	WHAT COMES BACK	WHY THAT IS RIGHT
Snippet is not in the file	`ERROR: old_string not found … copy the snippet exactly, including whitespace.`	The model hallucinated whitespace. The message says what to do next, so the following step usually fixes it.
Snippet appears 4 times	`ERROR: old_string appears 4 times … pass replace_all: true`	Silent first-match replacement would edit the wrong function. Ambiguity must be an error, not a guess.
Snippet is `""`	`ERROR: old_string must not be empty`	An empty search string would otherwise "match" everywhere.
Replacement contains `$1`	replaced literally	`String.replace` interprets `$1` when the second argument is a string; the callback form removes the footgun.
File does not exist	`ERROR: no such file: …`	Points at the missing step without being asked.

The `$1` detail is a classic JavaScript trap worth understanding:

```
'hello world'.replace('world', '$&!');   // 'hello world!'
'hello world'.replace('world', () => '$&!'); // 'hello $&!'
```

When the replacement is a string, `$&`, `$1`, `$'` and friends are special. When it is a function, the return value is used literally. For a code-editing tool, literal is the only safe choice: a replacement containing `$1` is almost certainly code, not a backreference.

The `FORBIDDEN` contract

A refused write:

1. prints exactly one bold red line: `⛔ FORBIDDEN SAVE ../escape.txt - path escapes the workspace: ../escape.txt`;
2. throws `ForbiddenError`;
3. is converted by the tool dispatcher into `FORBIDDEN: ...` text;
4. is returned to the model as an observation;
5. is appended to the message array like any other result.

The model can then choose another path. The loop does not crash. The user sees one loud line. That is the full contract.

Honest limitations

This sandbox is a **path policy**, not an operating-system sandbox. It does not isolate processes, restrict syscalls, or prevent a command from writing outside the workspace. Stage 9 adds a command-line scan, but that is still a seatbelt, not a container. Say this out loud in your documentation and your demo. Implying the demo is safe by construction is worse than admitting its limits.

Checkpoint

1. What are the two questions `workspace.js` and `sandbox.js` answer?

A. `workspace.js`: is this path contained? `sandbox.js`: may the agent change this file at all?

2. Why is an unknown sandbox action a `TypeError` rather than a `ForbiddenError`?

A. Unknown action is a programming bug; a path refusal is an expected policy outcome.

3. Why does `create_file` refuse to overwrite, while `write_file` does not?

A. "Create" should not silently destroy; replacement should be explicit via `write_file`.

4. Why does `edit_file` use the callback form of `String.replace`?

A. The callback form treats `$1`, `$&` literally, which is what code replacement needs.

5. Why is deleting the sandbox root refused separately from the path check?

A. Deleting the root is a catastrophic case that deserves its own explicit guard and message.

Stage 9: Command execution

What this stage is really about

Stage 9 gives the model a shell, which is both the most useful and the most dangerous tool in the harness. The theory here is about **layered policy**:

1. a hard deny-list that no policy can override;
2. a command-line scan for visible write targets outside the workspace;
3. a configurable policy (`deny` / `ask` / `allow`);
4. a human approval prompt;
5. a restricted child environment;
6. output clipping and timeouts.

No single layer is sufficient. Together they make the common cases safe and the remaining risk explicit.

Why a shell at all?

Because "run what you wrote" is the difference between a plausible edit and a verified one. Exit codes are how the loop closes: a file that has not executed is a hypothesis. The model needs to run tests, linters, and scripts, and a shell is the universal interface to them.

But a shell is also a general-purpose escape hatch. It can copy, delete, download, and execute arbitrary code. That is why it gets the most policy.

The hard deny-list

Eight patterns are checked **before** the policy is consulted, so `allow` cannot unblock them:

PATTERN	REFUSED BECAUSE
`rm -rf /`, `rm -rf ~`	recursive delete of a root path
`del /s`, `del /q`	recursive or quiet Windows delete
`format`, `diskpart`	disk formatting
`shutdown`, `reboot`	machine power control
`git push --force`, `git push -f`	force push
`curl … | sh`, `wget … | bash`	piping the internet into a shell
`:(){ :|:& };:`	fork bomb
`> /dev/sda`	raw disk write

These are not a security boundary. They are a seatbelt against a model that has read too many bad shell examples. The point is to make the catastrophic cases impossible, not to make every case safe.

Why check before policy? Because a user who types `--yes` is approving ordinary commands, not `format C:`. The hard deny-list is a statement that some commands are never part of the task.

Scanning the command line for writes

The file tools were never the whole story. `copy hello.js %USERPROFILE%\Desktop\hello.js` leaves the sandbox without touching a file tool. The command line says where it is writing, so `sandbox.checkCommand()` reads it first.

The scanner tokenises the command, then extracts write targets by verb family:

COMMAND FAMILY	WRITE TARGET
`copy`, `cp`, `move`, `mv`, `xcopy`, `ren`, `rename`, `ln`, `install`, `scp`, `rsync`	last positional (require at least two)
`robocopy`	positional argument 2
`del`, `erase`, `rd`, `rmdir`, `rm`, `md`, `mkdir`, `touch`, `tee`, `unlink`, `truncate`, `shred`	every positional
`curl`	after `-o`, `--output`, `-O`, `--remote-name`
`wget`	after `-O`, `--output-document`
PowerShell `out-file`, `set-content`, `add-content`, `new-item`, `ni`, `remove-item`, `copy-item`, `move-item`, `rename-item`	after `-destination`, `-dest`, `-path`, `-filepath`, `-literalpath`
`>` / `>>` redirection	the following token (`2> file` handles a stream id)

Then each target is passed through `workspace.resolve()`, the same policy a `write_file` uses. A refusal prints the same `FORBIDDEN` line and throws before the command runs.

Tokenisation is not parsing

`tokenizeCommand(text)` splits on whitespace while honouring single and double quotes, and emits shell operators as separate tokens: `>`, `>>`, `<`, `|`, `||`, `&`, `&&`, `;`. Quotes are stripped.

This is deliberately not a full shell parser. It handles the common cases and fails closed on ambiguity. Two details matter:

- **Quoted arguments with spaces stay one token.** Otherwise a filename with a space looks like two positional arguments.
- **A switch is `--flag`, `-f`, or Windows `/x`.** A POSIX path such as `/home` is not a switch. Distinguishing them prevents a path from being skipped as an option.

Policy layers

`CMD_POLICY`	BEHAVIOUR	WHEN YOU WOULD USE IT
`ask` (default)	Every command stops and waits for `y/N`. `n` returns `DENIED: the user did not approve this command.`	Live demos and any run you are watching. The pause is the point.
`allow`	Commands run without asking. Set by `--yes`, `--allow-commands`, or `CMD_POLICY=allow`.	The scripted demo, CI, or a task you already trust.
`deny`	No command runs at all; the model gets a denial and can still edit files.	Read-mostly tasks, or a run with no terminal.

Order of checks in `run_command`:

```
trim → empty? → hard deny-list → sandbox.checkCommand → policy
  → deny: return DENIED
  → allow: proceed
  → ask: confirm via readline
  → execute
```

Notice that the sandbox scan happens **before** approval. A `copy` to the Desktop never reaches the prompt. This is better UX and better security: do not ask a human to approve something you already know is forbidden.

The non-TTY rule

With no terminal attached (piped input, CI, a scheduled run), the `ask` policy degrades to **deny**. An unanswerable prompt must never become an approval.

This is why `confirm` defaults to `async () => 'deny'`. The default is the safe answer, and a caller must opt into asking. It is a small design decision with a large consequence: the harness is safe by default in automation.

The child environment allow-list

The child process must not inherit the harness environment. If it did, a command could print `OPENROUTER_API_KEY`, and the model would see it in the observation. The demo checks exactly this.

`safeEnv(source)` keeps only:

`PATH`, `PATHEXT`, `HOME`, `USERPROFILE`, `TMPDIR`, `TEMP`, `TMP`, `SYSTEMROOT`, `COMSPEC`, `SHELL`, `LANG`, `LC_ALL`, `TERM`.

Why these?

- `PATH` and `PATHEXT` are needed to find commands.
- `HOME` / `USERPROFILE` are needed by many tools.
- Temp variables are needed by compilers and installers.
- `SYSTEMROOT` / `COMSPEC` are needed by `cmd.exe`.
- `SHELL`, `LANG`, `LC_ALL`, `TERM` are needed for predictable Unix behaviour.

Everything else is dropped, including API keys, tokens, and unrelated configuration. This is least privilege at the process boundary.

Execution details

```
exec(command, {
  cwd: workspace.root,
  timeout: 30_000,
  maxBuffer: 1024 * 1024,
  windowsHide: true,
  encoding: 'utf8',
  env: safeEnv(process.env),
});
```

- `cwd: workspace.root` makes relative paths predictable.

- `timeout: 30_000` stops a hanging command from hanging the run.
- `maxBuffer: 1 MB` stops a chatty command from exhausting memory.
- `windowsHide: true` avoids a console window flash on Windows.
- `encoding: 'utf8'` makes the result a string.

Result format

```
exit code <n>
stdout:
<first lines>
stderr:
<first lines>
```

Rules:

- include `stdout:` only when non-empty;
- include `stderr:` only when non-empty;
- add `(no output)` when both are empty;
- clip each stream to 6,000 characters, appending `... [<n> chars truncated]`;
- on timeout, append `[tool] command timed out after 30s` to stderr and return a non-zero code.

Why include the exit code first? Because it is the verdict. The model should be able to decide "did this work?" before reading the output. Why clip? Because a test suite can print megabytes, and the model does not need all of it to act.

Why `run_command` is the most important teaching tool

It demonstrates four principles at once:

- **Exit codes close the loop.** `exit code 0` versus `1` is how the model knows whether its code works.
- **Human in the loop.** The default is to ask.
- **Least privilege.** The environment is allow-listed.
- **Bounded autonomy.** There is a timeout and an output cap.

It is also the tool that makes the sandbox's limitations honest. The command-line scan is not isolation. A one-liner can still write anywhere. Say so.

Checkpoint

1. Why is the hard deny-list checked before the policy?

A. Some commands are never part of a legitimate task; policy cannot override that.

2. Why is the command-line write scan done before the human approval prompt?

A. To avoid asking a human to approve something already known to be forbidden.

3. Why does `ask` degrade to `deny` when stdin is not a terminal?

A. An unanswerable prompt must never become an approval.

4. What does the child environment allow-list protect against?

A. It prevents API keys and unrelated secrets from leaking into tool output.

5. Why is the exit code the first line of the result?

A. It is the verdict: did the command succeed? The output is evidence.

6. What can still escape the sandbox even with all these layers?

A. A script or one-liner can still write anywhere; the scan reads the command line, not the code.

Phase C: Memory and observability

Goal of the phase: the agent can remember facts across runs, and you can see exactly what it is doing at every lifecycle moment.

What you can do at the end: save a durable note, find it again in a later process, and print a trace of all nine lifecycle hooks.

Stage 10: Long-term memory

What this stage is really about

Everything so far lives and dies inside one process. `messages` is working memory; when the process exits it is gone — including lessons that were expensive to learn ("the tests are offline", "this repo spells it Brighton", "force-push is refused").

Stage 10 adds the smallest thing that actually persists: **a markdown file** and two tools that read and write it.

> **Memory is a file, not a mystery.**

No embeddings, no vector store, no summarising, no write on every turn. One file a human can read, diff and delete a line from. "Why did it remember that?" is a `cat`, not an investigation.

Why a file, not a database?

- **No cache, no drift.** Every method reads the file. A hand edit, a deletion or a `git checkout` is picked up on the next call. A memory module with a cache is a memory module that lies.
- **It diffs like code.** `src/memory.md` appears in `git status`, diffs cleanly, and can be reviewed in a pull request.
- **You can audit it.** `node src/cli.js --memory` prints the file and the exact block the prompt received.
- **Nothing to migrate.** One bullet per memory; the bullet *is* the record.

This is not the final word in memory architecture. It is the first thing that works, and every real system grows past it. Starting here teaches the boundary: what persists, what costs context, and what a human can trust.

The format

```
# memory

Long-term notes this agent keeps between runs. One bullet per memory:
`- [tag] text _(YYYY-MM-DD)_` where tag is fact | pref | fix | gotcha.

This file IS the store - edit it by hand, delete a wrong line, or commit it.

- [fact] The sandbox demo script lives at demo/hello.js and is safe to regenerate with create_file. _(2026-10-28)_
- [pref] The user wants console output, not files, for anything under three lines. _(2026-10-28)_
```

Tags:

TAG	MEANING
`fact`	A stable fact about the project or environment.
`pref`	A user preference.
`fix`	A known repair or workaround.
`gotcha`	A trap that costs a step if forgotten.

Why tags? They give `recall` a second dimension to search and they encourage the model to classify what it is saving. They also make the file scannable for a human.

Parsing is forgiving on purpose

```
export function parseMemory(source) {
  const entries = [];
  for (const line of String(source ?? '').split(/\r?\n/)) {
    const m = line.match(
      /^\s*[-*]\s*\[([a-z]+)\]\s*(.+?)\s*(?:_\((\d{4}-\d{2}-\d{2})\)_)?\s*$/i,
    );
    if (!m) continue;
    const text = normalise(m[2]);
    if (!text) continue;
    entries.push({ tag: m[1].toLowerCase(), text, date: m[3] ?? 'undated' });
  }
  return entries;
}
```

Design choices:

- accept `-` or `*` bullets;
- accept an optional date (hand-written notes may not have one);
- normalise whitespace in the text;
- ignore prose, headings, and comments.

A parser that rejects the whole file on one malformed line would be hostile to the human editor. A parser that skips what it does not understand is a parser you can live with.

The `Memory` class

```
new Memory({
  root,
  file,
  maxEntries = 60,
  maxEntryChars = 300,
  rememberChars = 2000,
  promptChars = 1200,
  promptEntries = 12,
  now = () => new Date(),
});
```

Every number is a bound, and every bound has a reason. The next sections explain them.

Writing: `remember`

```
remember(rawText, { tag = 'fact' } = {}) {
  const text = normalise(rawText).slice(0, this.options.rememberChars);
  const entry = {
    tag,
    text: text.slice(0, this.options.maxEntryChars),
    date: today(this.now()),
  };

  const entries = this.entries();
  const existing = entries.findIndex(
    (e) => e.text.toLowerCase() === text.toLowerCase(),
  );
  if (existing !== -1) {
    entries[existing] = { ...entries[existing], date: entry.date, tag: entry.tag };
    this.save(entries);
    return { ok: true, duplicate: true, entry: entries[existing], total: entries.length };
  }

  entries.push(entry);
  const dropped = Math.max(0, entries.length - this.options.maxEntries);
  if (dropped) entries.splice(0, dropped);
  this.save(entries);
  return { ok: true, entry, dropped, total: entries.length };
}
```

Three behaviours worth teaching:

1. **Duplicates refresh the date instead of appending.** Saying the same thing again is not new information.
2. **The oldest entries are dropped at `maxEntries`.** Like `maxSteps`, the file is bounded on purpose.
3. **The text is truncated to `maxEntryChars`.** No single note can dominate the digest.

Reading: eager and lazy

The two access paths cost different things, so the harness has both.

PATH	WHAT IT IS	WHAT IT COSTS
Eager — `digest()`	The newest 12 entries, capped at 1200 characters, appended to the system prompt.	A fixed number of characters. It cannot grow with the file, so it cannot quietly eat the context window.
Lazy — `recall`	A substring search over the whole file, when the model asks for it.	One tool call, and only when the task needs it.

The digest always says what it left out:

```
## What you remember from earlier runs
These notes live in memory.md and were learned before this task started.
Treat them as strong hints, not facts to trust blindly: the code may have moved on since.
- [gotcha] create_file refuses to overwrite, so re-running the demo fails at step 2. _(2026-10-24)_
- [fix] The demo script lives at demo/hello.js; delete it before a clean demo run. _(2026-10-28)_
- [pref] The user wants console output, not files, for anything under three lines. _(2026-10-28)_
(9 older memories not shown - search memory.md with the recall tool)
```

That last line is the difference between "this is all I know" and "this is what fits — ask for the rest". It is one line of code and a large improvement in honesty.

Integration: four lines in the loop

```
const messages = [
  { role: 'system', content: buildSystemPrompt({ memory, log }) },
  { role: 'user', content: task },
];
const ctx = { workspace, commandPolicy, memory, confirm };
```

No new message role, no special channel, no vector store. Memory is text in the one message the model always receives.

The actual decision lives in `prompt.js`:

```
const digest = memory.digest();
if (!digest) return rules;
return `${rules}\n\n${digest}`;
```

"What the model is told" is one decision made in one place: the rules, then the digest, then a log line saying how many memories were carried.

The two tools

`remember`

- params: `text` (required), `tag` (optional);
- no memory store → `ERROR: no memory store in this run (started with --no-memory)`;
- empty → `ERROR: nothing to remember - the note was empty`;
- duplicate → `already remembered in <label> (<total> total), date refreshed: <formatted entry>`;
- new → `remembered in <label> (<total> total[, dropped the N oldest]): <formatted entry>`.

`recall`

- params: `query` (optional), `limit` (default 20);
- no memory → same error;
- no hits → `no memories in <label> match "<query>" (<size> stored)`;
- hits → formatted entries.

Both are ordinary tools: they appear in the transcript, they return strings, and they are subject to the same validation and error handling as file tools.

Why `remember` is not automatic

Nothing writes memory at the end of a run. A model that saved a memory every step would be filling your context with its own diary. The system prompt says: durable facts only, never task progress. The write is a deliberate tool call, visible in the transcript and auditable.

The caps

KNOB	DEFAULT	WHAT IT BOUNDS
`MEMORY_MAX_ENTRIES`	60	the file — the oldest entries are dropped when it is reached

KNOB	DEFAULT	WHAT IT BOUNDS
`MEMORY_PROMPT_CHARS`	1200	the injected block; `0` turns injection off and leaves `recall` working
`maxEntryChars`	300	one memory, so no single note can dominate
`rememberChars`	2000	one `remember` call
`promptEntries`	12	how many of the newest appear in the digest

The most interesting combination is `MEMORY_PROMPT_CHARS=0`: the harness changes from "stuff what fits" to "retrieve when needed", with no code change. That is a design parameter, not a feature flag bolted on later.

CLI side-doors

- `--memory`: prints the store, the digest, and the raw file, then exits `0`.
- `--no-memory`: disables memory for that run.
- `MEMORY_FILE`: points at a different store (an absolute path shares one store across checkouts).

The side-door is important for trust. "What does it remember about me?" should be a command, not an interrogation of the model.

Checkpoint

1. Why does every `Memory` method read the file instead of using a cache?

A. A cache can disagree with the file; reading every time makes hand edits authoritative.

2. Why does a duplicate refresh the date instead of appending?

A. Repeating a note is not new information; refreshing the date keeps recency without duplicating.

3. What is the difference between the eager digest and lazy `recall`, and why have both?

A. Eager is bounded and always present; lazy is unbounded search on demand. Both are needed to control cost while keeping access.

4. Why does the digest state how many memories were hidden?

A. Honesty: the model should know it is seeing a bounded slice and can ask for the rest.

5. What does `MEMORY_PROMPT_CHARS=0` change, and what does it leave working?

A. It disables injection but leaves `recall` working — retrieve-on-demand instead of stuff-what-fits.

6. Why is a memory write a tool call rather than an automatic end-of-run action?

A. Automatic writes would fill context with a diary; a visible tool call is auditable and intentional.

Stage 11: Lifecycle hooks

What this stage is really about

The loop is fixed on purpose — that is what makes it readable — but a fixed loop is not *extensible*. "Log every command", "never let it touch `.env`", "tell me when it gives up": every one of those used to mean editing `agent.js`, and every one of those edits is a chance to break the loop for everybody.

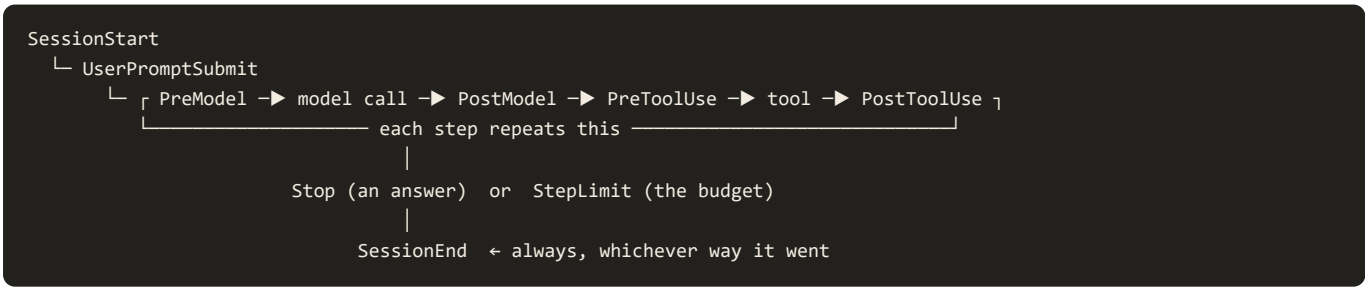
A **hook** is a function attached to a named moment in the loop. The loop calls all nine moments, and your code never edits the loop at all.

This is the chapter that answers the question an audience always asks: *"So how would you make this real?"* Permissions, audit logs, redaction, compaction, notifications — they are all the same mechanism: a named pause plus a decision.

The nine moments

#	HOOK	ICON	COLOUR	FIRES	WHAT A HOOK MAY DO
1	`SessionStart`	🚀	cyan	once, before the first model call	**refuse to start** — or post a note
2	`UserPromptSubmit`	✍️	blue	once, as the task enters the loop	**rewrite the task**, or block the run
3	`PreModel`	🧠	magenta	before every model call	observe: the request is assembled, not sent
4	`PostModel`	💬	cyan	after every reply, before dispatch	observe: answer or tool call, visible first
5	`PreToolUse`	🛠️	bright yellow	before every tool call	**block** — the tool never runs
6	`PostToolUse`	📦	green	after every tool call	**replace** the observation the model reads
7	`Stop`	🏁	bright green	the model answered and the loop is ending	observe: summary, cost, notification
8	`StepLimit`	⏳	yellow	the loop hit `maxSteps`	observe: escalate, dump the transcript
9	`SessionEnd`	🔒	red	always — finished or not	observe: the last code to run in the process

One run, in order



The hooks sit exactly where the loop pauses. Nothing is interleaved by magic. If you add a tenth hook, add it where the code already pauses; a hook for something the loop does not do is documentation, not an extension point.

Why these nine?

Because the nine are the places the loop *already* pauses:

- before it starts,
- before the first request,
- before and after each reply,
- before and after each tool call,
- on success,
- on exhaustion,
- and at the end.

A hook system should mirror the control flow you have, not invent a control flow you wish you had.

The contract

A hook is called with a payload object and its **return value** decides what happens:

```
return undefined;                      // observe only - the run is untouched
return { note: 'checked' };            // say something; still untouched
return { task: '…and run the tests' }; // UserPromptSubmit: rewrite the task
return { replace: 'sk-or-[redacted]' }; // PostToolUse: change what the model reads
return { block: true, reason: 'no' };   // SessionStart/UserPromptSubmit/PreToolUse: refuse
```

That is the whole API, deliberately three verbs wide: **observe**, **rewrite**, **block**.

Four hooks can change the run

- `SessionStart`: refuse to start.
- `UserPromptSubmit`: rewrite the task or block the run.
- `PreToolUse`: block a tool call.
- `PostToolUse`: replace the observation.

The other five only watch. That asymmetry is what makes hooks a safe first extension point: most of them cannot break anything.

Blocking is an observation, not an exception

When `PreToolUse` blocks a call, the tool is never dispatched *and* the model is told why:

```
DENIED: a PreToolUse hook blocked this call (no pushes from the sandbox). Try another approach.
```

The loop cannot crash on a "no", and the model gets its next turn to choose differently. This is principle 4 applied to permissions: **a refusal is an observation.**

A throwing hook is reported and ignored

One broken hook must not take the loop with it. The handler is caught, printed as `hook PostToolUse (…) threw: … - ignored`, and the next handler runs.

Why is this essential? Because every `tool_call_id` must still get a matching tool message, or the next request is invalid. A crashing hook that skipped message appending would corrupt the conversation.

This is a general design rule for extension points: **isolate the extension.** The core loop's invariants are more important than any individual plugin.

The registry

```
export class Hooks {
  on(name, fn, { id, once } = {}) { /* validate name, store handler */ }
  off(name, id) { /* detach */ }
  count(name) { /* how many */ }
  has(name) { /* any attached? */ }
  attached() { /* all events with handlers */ }
  async run(name, payload = {}) { /* call in order, collect decisions */ }
}
```

`run` returns one object:

```
{
  event, ran, blocked, reason, task, replace, notes, errors
}
```

The loop reads `blocked`, `task`, `replace`, and otherwise ignores the rest.

Why collect into one object rather than a boolean? Because a single hook system must express four different decisions (note, rewrite, replace, block) without the loop learning four APIs.

First block wins

```
if (decision.block) {
  out.blocked = true;
  out.reason = decision.reason ? String(decision.reason) : `${name} blocked this step`;
  break;
}
```

Why stop at the first block? Because continuing would let a later hook "unblock" a refusal, which is confusing and unsafe. A block is final for that moment.

`traceHooks`

`traceHooks({ log, detail, explain })` returns a `Hooks` instance with one observer per event. It prints:

```
  🛠  [HOOK:PreToolUse] before every tool call - run_command {"command":"node demo/hello.js"}
```

The trace is on by default: every run prints each lifecycle moment as one line, tagged `[HOOK:Name]`, followed by what that moment is *for* and what this firing carried. `--no-hooks` silences it; `--hooks` prints the catalogue.

The catalogue is generated from the same `HOOK_EVENTS` array the loop fires, so the documentation and the console cannot drift apart. That is a small but powerful idea: **generate the teaching material from the code.**

A permission layer and a redactor, in twelve lines

This is a real, runnable use of the API:

```
import { runTask } from './agent.js';
import { Hooks, printHookCatalogue } from './hooks.js';

const hooks = new Hooks({ log: console.log });

// 1. A permission layer. The model asks; the hook decides.
hooks.on('PreToolUse', ({ name, args }) => (
  name === 'run_command' && /\bgit push\b/.test(args)
    ? { block: true, reason: 'no pushes from the sandbox' }
    : undefined
));

// 2. Redaction. Whatever a command printed, a key never reaches the model.
hooks.on('PostToolUse', ({ result }) => ({
  replace: String(result).replace(/sk-or-\S+/g, 'sk-or-[redacted]'),
}));

// 3. Failures get a lifecycle too.
hooks.on('SessionEnd', ({ ok, reason, durationMs }) => ({
  note: `run ${ok ? 'ok' : `failed (${reason})`} in ${durationMs}ms`,
}));

printHookCatalogue({ hooks });
await runTask({ task, workspace, commandPolicy, maxSteps, chatFn, hooks });
```

With that attached, a `git push origin main` never runs, and the model is told so as a tool result:

```
tool → run_command {"command":"git push origin main"}
result ← DENIED: a PreToolUse hook blocked this call (no pushes from the sandbox). Try another approach.
```

Tools versus hooks

> **A tool is how the model acts on the world; a hook is how you act on the loop.**

Tools are the model's hands, described in the prompt; the model chooses them. Hooks are yours: not described to the model, no vote from it, and they run whether it likes it or not. That asymmetry is the safety story of a real harness.

Checkpoint

1. Why are there exactly nine hooks, and how were they chosen?

A. They are the places the loop already pauses; a hook for something the loop does not do is documentation, not an extension point.

2. Which four hooks can change the run, and what can each change?

A. `SessionStart` (refuse start), `UserPromptSubmit` (rewrite/block task), `PreToolUse` (block call), `PostToolUse` (replace observation).

3. Why does a hook that throws not stop the loop?

A. One broken extension must not break the loop's invariants, especially the tool-call/result pairing.

4. Why is a `PreToolUse` block returned as an observation rather than thrown?

A. A refusal is a normal outcome the model can recover from; a throw would end the run.

5. Why does the first block win?

A. A later hook must not be able to unblock a refusal; that would be confusing and unsafe.

6. Why is the hook catalogue generated from the same data the loop uses?

A. Generated from the same data the loop fires, so docs and behaviour cannot drift.

Phase D: The agent loop and polish

Goal of the phase: extract the conversation loop into a testable library, add an offline demo that proves the safety guards, and polish the CLI with sessions, history trimming, a banner, and exit codes.

What you can do at the end: run a multi-task session, run a complete scripted demo with no network, and explain the entire lifecycle from a single banner.

Stage 12: The ReAct loop

What this stage is really about

Stage 12 formalises the loop as a library: `runTask`. It is the chapter where "the loop is the agent" becomes literal code you can read in one sitting.

The extraction is deliberately late. By now you have built the loop by hand in `cli.js`, added tools to it, and wired hooks into it. You know what belongs there. Extracting earlier would produce a library nobody understands.

The signature

```
export async function runTask({
  task,
  workspace,
  sandbox = null,
  commandPolicy,
  maxSteps,
  chatFn,
  confirm,
  hooks = null,
  memory = null,
  history = [],
  log = console.log,
  heartbeatMs = 5000,
}) { /* ... */ }
```

Return shapes:

```
{ ok: true,  answer, messages, steps }
{ ok: false, reason, messages }
```

Why an object rather than throwing? Because a task outcome is data the CLI can inspect and turn into an exit code. `request_failed` and `max_steps` are expected outcomes, not exceptional ones.

The golden rule

`agent.js` is a library. It knows nothing about flags, colours, `.env`, or `process.exit`. The CLI owns all process concerns. Tests can run the loop with a fake model and no network.

This is not purity for its own sake. It is what makes the loop testable:

- pass a fake `chatFn`;
- pass a temp `workspace`;
- pass a fake `confirm`;
- pass `log: () => {}`;
- assert on the returned `messages` and `ok`.

No network, no API key, no terminal. The entire Stage 15 test suite depends on this boundary.

The seven beats per step

For `step = 1..maxSteps`:

1. **Print a step heading.** `ui.step(step, maxSteps)`.
2. **Fire** `PreModel` with `{ step, maxSteps, messages, tools: TOOL_SCHEMAS }`.
3. **Start the heartbeat and call the model.** - On throw: stop the heartbeat, print `ui.error`, fire `SessionEnd`, return `{ ok: false, reason: 'request_failed', messages }`. - On success: stop the heartbeat.
4. **Extract the choice and message.** Fire `PostModel` with `{ step, message, finishReason }`.
5. **Truncation guard.** If `finish_reason === 'length'` and there are no tool calls: - append the assistant message; - append a `user` message: `"You were cut off by the length limit. Continue with smaller tool calls."`; - print a warning; - `continue` to the next step. Never dispatch truncated tool calls.
6. **Append** `sanitizeAssistant(message)`**.** If `content` is non-empty, print `ui.think(...)`.
7. **Dispatch tool calls or finish.** - **No calls:** print the answer, fire `Stop`, fire `SessionEnd`, return `{ ok: true, answer, messages, steps: step }`. - **Calls:** for each in order:
   1. read `name` and `rawArgs`;
   2. print `ui.tool(name, rawArgs)`;
   3. fire `PreToolUse`; if blocked, observation = `DENIED: a PreToolUse hook blocked this call (<reason>). Try another approach.`; otherwise `await runTool(name, rawArgs, ctx)`;
   4. print `ui.result(result)`;
   5. fire `PostToolUse`; if `replace !== null`, use the replacement;
   6. append exactly one `{ role: 'tool', tool_call_id: call.id, content: String(post.replace ?? result) }`.

After the loop exhausts `maxSteps`:

- print `ui.error('stopped: hit the <maxSteps>-step limit')`;
- fire `StepLimit`, then `SessionEnd`;
- return `{ ok: false, reason: 'max_steps', messages }`.

The truncation guard is the interesting one

A cut-off reply often ends mid-JSON:

```
{"path": "hello.js", "cont
```

Dispatching that would produce `ERROR: could not parse arguments` for a tool call the model never finished, and the model would spend a step apologising for a bug it did not have.

Refusing to dispatch it is the difference between a harness that respects the model and one that argues with it. The corrective message is a `user` turn, not a system change: the model is told what happened and asked to continue smaller.

Why the system prompt is rebuilt

```
const messages = [
  { role: 'system', content: buildSystemPrompt({ memory, log }) },
  ...history.filter((m) => m.role !== 'system'),
  { role: 'user', content: task },
];
```

Two details are the whole trick:

1. **The system message is rebuilt**, so a memory saved by the previous task is already in the prompt the next task sends.
2. **The history is filtered** to remove any old system message, so there is exactly one.

If you carried the old system message forward, memory would be stale and the prompt would slowly accumulate duplicates.

The hook default

```
const hook = hooks ?? new Hooks();
```

Why not `if (hooks) await hooks.run(...)` at every call site? Because that would litter the loop with conditionals. A null-object default lets every call site call `hook.run(...)` unconditionally. This is a small pattern worth learning: **make the absent case a working object, not a special case.**

Entry guard: `agent.js` is not the program

Running `node src/agent.js` should print:

```
agent.js is the loop, not the program.
Run the CLI instead:  node src/cli.js --help
```

and exit. `agent.js` must not import `cli.js`. The dependency arrow points one way.

The heartbeat

```
export function heartbeat(log, everyMs = 5000) {
  const started = Date.now();
  const timer = setInterval(() => {
    const seconds = Math.round((Date.now() - started) / 1000);
    log(ui.waiting(`  … still waiting for the model (${seconds}s)`));
  }, everyMs);
  timer.unref?.();
  return { stop: () => clearInterval(timer) };
}
```

Why `unref()`? A timer that holds the event loop open would turn a clean exit into a program that hangs for five more seconds. `unref` says "this timer must not keep the process alive on its own".

Why arm it just before the call and stop it just after, including on the failure path? Because a leaked timer prints "still waiting" after the run is over, which is worse than no heartbeat.

Checkpoint

1. Why is `runTask` a library rather than part of `cli.js`?

A. So it can be tested and read without the CLI; the dependency arrow points one way.

2. Why are `request_failed` and `max_steps` returned rather than thrown?

A. They are expected task outcomes the CLI turns into exit codes; throwing would lose the message array.

3. Walk through the seven beats from memory. Which beat fires which hook?

A. Step heading → `PreModel`; model call; `PostModel`; truncation guard; append/sanitise; `Stop` / `PreToolUse`; `PostToolUse`; `StepLimit` / `SessionEnd`.

4. Why is a truncated tool call not dispatched?

A. A truncated call is half a JSON object; dispatching it produces a confusing error for a call the model never finished.

5. Why is the system prompt rebuilt for every task?

A. Memory may have changed, and a carried system message would be stale or duplicated.

6. What does `hooks ?? new Hooks()` buy the call sites?

A. It makes the absent case a working object, so call sites need no conditionals.

7. Why does the heartbeat call `unref()`?

A. So the timer never keeps the process alive after the run is finished.

Stage 13: Offline demo

What this stage is really about

Stage 13 proves the harness without a network, a key, or a model. A **fake model** replays a fixed script, and every other seam is the real one: the real loop, the real dispatcher, the real sandbox, the real memory tools. The only thing missing is one POST.

This is the stage where you learn that a demo can be a test.

The fake model

```
export function demoChat() {
  let i = 0;
  return async () => {
    const message = DEMO_SCRIPT[Math.min(i, DEMO_SCRIPT.length - 1)];
    i += 1;
    return {
      choices: [{
        finish_reason: message.tool_calls ? 'tool_calls' : 'stop',
        message,
      }],
    };
  };
}
```

It has the same return shape as OpenRouter. That is the whole of `--demo`: one ternary in the CLI chooses `demoChat()` instead of the HTTP client.

Why this works as a test

Because the reply is data, the script can do something a live model will not do on cue: deliberately try to escape the sandbox. Steps 6–10 do exactly that, and `sandboxScorecard()` reads the results back and prints which guards actually held.

That is a claim the program checks, not a slide.

The script

The 14 tool steps plus a final answer:

1. `list_files` `{ path: '.' }`
2. `create_file` `{ path: 'demo/hello.js', content: ... }`
3. `run_command` `{ command: 'node demo/hello.js' }`
4. `edit_file` `{ path: 'demo/hello.js', old_string: ..., new_string: ... }`
5. `run_command` `{ command: 'node demo/hello.js' }`
6. `write_file` `{ path: '../escape.txt', content: 'must never be written' }` → expect `FORBIDDEN`
7. `read_file` `{ path: '../package.json', limit: 5 }` → allowed
8. `write_file` `{ path: '~/Desktop/pwned.txt', content: ... }` → expect `FORBIDDEN`

9. `run_command` `{ command: 'copy demo/hello.js %USERPROFILE%\\Desktop\\hello-copy.js' }` → expect `FORBIDDEN`
10. `run_command` `{ command: 'git push --force origin main' }` → expect `DENIED`
11. `run_command` that echoes `OPENROUTER_API_KEY` → expect `absent`
12. `run_command` that exits `1` → expect `exit code 1`
13. `remember` `{ text: ..., tag: 'fact' }`
14. `recall` `{ query: 'demo' }`

Then a final answer summarising everything.

Why these steps?

- 1–5 are the loop: look, create, run, edit, run.
- 6–10 are the fence: path escape, open read, home write, shell copy, force push.
- 11–12 are process safety: environment isolation, exit codes.
- 13–14 are persistence: write and read memory.

Every capability in the course appears at least once.

The scorecard

`sandboxScorecard(messages)` maps `tool_call_id` to the tool result, runs regex checks, and returns rows:

```
[
  { ok: true,  label: 'a write that escapes the workspace is refused', seen: 'FORBIDDEN: SAVE ../escape.txt' },
  { ok: true,  label: 'reads are open - one folder up works',          seen: '../package.json (22 lines)' },
  { ok: true,  label: 'a home-folder write is refused',                seen: 'FORBIDDEN: SAVE ~/Desktop/pwned.txt' },
  { ok: true,  label: 'a shell copy out of the sandbox is refused',    seen: 'FORBIDDEN: COMMAND COPY ' },
  { ok: true,  label: 'dangerous commands are denied, whatever the policy', seen: 'DENIED: blocked by policy' },
  { ok: true,  label: 'the child process cannot see the API key',      seen: 'OPENROUTER_API_KEY=absent' },
  { ok: true,  label: 'run_command reports the real exit code',        seen: 'exit code 1' },
]
```

Why a scorecard rather than a few `console.log` lines?

- It is **checkable**: the run exits non-zero if any guard regressed.
- It is **readable**: each row says what was tested and what was seen.
- It is **teachable**: the audience sees the claim and the evidence side by side.

The demo exits `0` on 7/7 checks; otherwise `process.exitCode = 1`.

Why `--demo` sets `maxSteps = DEMO_SCRIPT.length + 1`

The script has 14 tool steps, then a final answer. The loop needs 15 steps to reach the answer. Setting `maxSteps` to `15` makes the demo deterministic and prevents a bug in the script from turning into a step-limit failure. The `+ 1` is for the answer.

The demo is also a re-run lesson

Run the demo a second time and step 2 fails: `demo/hello.js` already exists, so `create_file` refuses to clobber it. `edit_file` may also fail because the line has already been edited. Both arrive as strings, the loop absorbs them, and the run still ends with an answer and a clean scorecard.

That is not a bug in the demo. It is principle 4 in action, and exactly what a careful human would do.

Checkpoint

1. What does `demoChat()` replace, and what does it leave real?

A. It replaces the HTTP client only; the loop, tools, sandbox, hooks and memory are real.

2. Why can the demo test guards that a live model might not exercise?

A. Because the script can deliberately attempt escapes that a live model might not try on cue.

3. Why does `--demo` need a `maxSteps` of 15?

A. 14 tool steps plus the final answer.

4. Why is the scorecard better than a transcript claim?

A. It is checkable: the process exits non-zero if a guard regresses, and each row shows the evidence.

5. What happens on a second demo run, and why is that instructive?

A. `create_file` refuses to overwrite, so the run demonstrates recoverable errors and still completes.

Stage 14: Sessions, trimming and CLI polish

What this stage is really about

Stage 14 is about the difference between a **task** and a **session**, and about making the program honest at the boundaries: history is trimmed, the banner is reproducible, and exit codes mean something.

The session loop

```
let history = [];
while (true) {
  const line = await askTask();
  if (line === null || line === '/quit' || line === '/exit') break;
  if (line === '/new') { history = []; continue; }
  if (!line) continue;

  const result = await runTask({ ...base, task: line, history });
  history = trimHistory(result.messages);
  printSummary(result);
}
```

Three behaviours come straight out of the design:

1. **The step budget restarts.** `runTask` is one task, and its `for` starts at 1, so the second task gets the full 12 steps — not "step 5 of 12" continued from the first.
2. **The conversation is carried, the system prompt is not.** The CLI passes the last 24 messages as `history`, so "now add tests" knows what was just built; the system message is rebuilt, so a memory saved by task one is already in front of the model in task two.
3. `/new` **forgets the conversation, not the project.** Files stay, `src/memory.md` stays.

`trimHistory`: why the cut must land on a `user` message

```
export function trimHistory(messages, max = MAX_HISTORY) {
  const kept = (messages ?? []).filter((m) => m.role !== 'system');
  let start = Math.max(0, kept.length - Math.max(0, max));
  while (start < kept.length && kept[start].role !== 'user') start += 1;
  return start >= kept.length ? [] : kept.slice(start);
}
```

Why not simply take the last 24 messages?

Because providers reject conversations that:

- start with a `tool` result (there is no assistant call that it answers), or
- end with an unanswered assistant `tool_calls` (the tool results were cut off).

Moving `start` forward until it lands on a `user` message guarantees a valid conversation boundary. If no safe boundary exists, return `[]` — a fresh conversation is better than an invalid one.

This is a subtle but important lesson: **truncation is not just about size; it is about protocol validity.**

The banner

Before step 1, print a reproducible header:

```
min-coding-js
workspace : <root>
sandbox   : save, edit, delete - only inside <root> (reads: anywhere)
model     : <model> (from <source>) | scripted (no network)
key       : <fingerprint> from <source> | - in demo
env file  : <path> | <path> (missing)
commands  : allow | ask | deny
timeout   : <seconds>s per model call
tools     : <comma-separated TOOL_NAMES>
memory    : <summary> | off (--no-memory)
hooks     : on: each of 9 prints as [HOOK:name] ... | 9 available, not printed (--no-hooks)
```

Why this is worth the code:

- **It is one header, not scattered logs.** When a demo goes wrong, you read one block.
- **It records provenance.** `(from .env)` answers the first debugging question.
- **It records posture.** `commands: ask` tells you whether a pause is expected.
- **It records capability.** The tool list tells you what the model can do in this run.

The banner is a contract: what you see is what the run uses.

Exit codes

CODE	MEANING
0	The model stopped calling tools and gave an answer, or the session ended normally.
1	The request failed — bad key, no credits, timeout, non-JSON, fatal error.
2	No task: nothing typed at the prompt, the field closed, or stdin is not a terminal.
3	Stopped at the step limit.

`taskExitCode(result)`:

```
if (!result.ok && result.reason === 'request_failed') return 1;
if (!result.ok && result.reason === 'max_steps') return 3;
return 0;
```

Why distinguish `1` and `3`? Because they mean different things to a caller:

- `1` is a configuration or transport problem; retrying without a change is pointless.
- `3` is an autonomy problem; the task may need a tighter prompt or a larger budget.

A script can branch on the difference.

The entry guard

```
if (import.meta.url === pathToFileURL(process.argv[1]).href) {
  main().catch((err) => {
    console.error(`fatal: ${err.stack}`);
    process.exitCode = 1;
  });
}
```

Why check that `cli.js` is the entry file? So that importing `cli.js` from a test does not run the program. This is the same lesson as `agent.js` being a library: **module top-level code should not have side effects.**

Warning on a non-OpenRouter model

If live mode is on and the model lacks a `/`, the banner warns:

```
warning: model is not an OpenRouter slug...
```

If normalisation changed a bare name, a dim note shows the final slug. A helpful guess that announces itself is a convenience; a silent one is a future bug.

The session summary

After each task:

```
── task finished ──────────────────────────────────────
  ✓ answer in 4 step(s) - 9 message(s) carried into the next task
  type another task, /new to clear the conversation, /quit to exit
```

Why print the message count? Because it makes the cost of history visible. The next task sends that many messages plus the system prompt. When the count grows, the bill grows.

Checkpoint

1. What is the difference between a task and a session?

A. A task is one `runTask` call; a session is multiple tasks in one process carrying history.

2. Why does the step budget restart for each task?

A. The step budget lives in `runTask`'s `for`, which starts at 1 per task.

3. Why must `trimHistory` start on a `user` message?

A. Providers reject histories starting with a tool result or ending with an unanswered tool call.

4. What does the banner record, and why is provenance part of it?

A. Workspace, sandbox posture, model and provenance, key fingerprint, env file, policy, timeout, tools, memory, hooks.

5. What do exit codes `1`, `2` and `3` each mean?

A. `1` request/fatal failure; `2` no task/no TTY; `3` step limit.

6. Why is `main()` called only when `cli.js` is the entry file?

A. So importing `cli.js` from a test does not run the program.

Phase E: Hardening and documentation

Goal of the phase: prove the harness offline, and write documentation that teaches the architecture rather than just the commands.

Stage 15: The full test suite

What this stage is really about

Stage 15 is where the library boundary pays off. Every seam is injectable, so the whole harness can be tested with:

- a fake `chatFn`;
- a temporary directory;
- an injected `log`;
- a fake `confirm`;
- no network, no key, no terminal.

The shape of a good test suite

The suites map to the architecture:

SUITE	WHAT IT PROVES
`workspace.test.js`	Path policy, containment, symlink safety.
`sandbox.test.js`	Refusals, root deletion, tool-layer conversion.
`tools.test.js`	All eight tools, validation, `safeEnv`, policy.
`shell-sandbox.test.js`	Tokenisation, write-target scanning.
`agent.test.js`	Loop behaviour, truncation, heartbeat, demo guards.
`hooks.test.js`	Registry, blocking, replacing, tracing, catalogue.
`memory.test.js`	Round-trip, duplicates, bounds, digest, forget, search.
`prompt.test.js`	Rules, injection, logging.
`llm.test.js`	Timeout, errors, sanitisation.
`config.test.js`	`.env` precedence, fingerprint, model normalisation.
`session.test.js`	Commands, trimming, follow-up tasks.
`ui.test.js`	Styling, verbatim output, result classification.
`input.test.js`	`askTask`, blank lines, Ctrl-C.

Why this mapping matters: a test suite that mirrors the module graph is a test suite you can navigate. When `sandbox.js` changes, you know exactly which tests should fail.

Testing the model client without a network

Mock `globalThis.fetch`:

```
const original = globalThis.fetch;
globalThis.fetch = async (url, init) => {
  assert.match(init.body, /"model":"deepseek/);
  return new Response(JSON.stringify({
    choices: [{ message: { role: 'assistant', content: 'hi' } }],
  }), { status: 200, headers: { 'Content-Type': 'application/json' } });
};
try {
  const reply = await chat({ apiKey: 'test', model: 'deepseek/deepseek-v4.1-flash-1017', messages: [] });
  assert.equal(reply.choices[0].message.content, 'hi');
} finally {
  globalThis.fetch = original;
}
```

Always restore the original in a `finally`. A leaked mock makes the next test lie.

Test the failure paths too:

- `401` → error mentions bad key;
- `402` → error mentions credits;
- non-JSON body → error includes a truncated preview;
- timeout → error says `timed out`;
- missing key → error says `OPENROUTER_API_KEY is not set`.

Testing the loop without a model

```
const chatFn = async ({ messages }) => ({
  choices: [{
    finish_reason: 'stop',
    message: { role: 'assistant', content: 'done' },
  }],
});

const result = await runTask({
  task: 'do nothing',
  workspace,
  commandPolicy: 'deny',
  maxSteps: 3,
  chatFn,
  confirm: async () => 'deny',
  log: () => {},
});

assert.equal(result.ok, true);
assert.equal(result.answer, 'done');
assert.equal(result.steps, 1);
```

For a tool call, return a `tool_calls` message, then a final answer on the next call. Use a counter in the fake model.

Testing the truncation guard

Return `finish_reason: 'length'` with no tool calls, then a normal answer. Assert:

- the truncated message is appended;
- a `user` corrective message is appended;

- the tool call is not dispatched;
- the loop continues.

Testing hooks

- Attach a hook and assert the payload shape.
- Attach two and assert order.
- Return `{ block: true }` and assert the next hook does not run.
- Throw from a hook and assert the loop continues and the error is collected.
- Attach `{ once: true }` and assert it runs once.
- `off` by id and assert it is gone.
- Unknown event name → throws.

Testing memory with a temp file

Use `fs.mkdtempSync(path.join(os.tmpdir(), 'memory-test-'))`, point `Memory` at a file inside it, and clean up in `after`. Assert:

- write and read back survive a new `Memory` instance;
- duplicate refreshes the date, no duplicate bullet;
- `maxEntries` drops the oldest;
- long text is truncated;
- empty text is refused;
- digest is bounded and states the hidden count;
- `forget` removes matches;
- hand edits are picked up immediately.

Why `npm test` must be offline

Because a test suite that needs a key is a test suite nobody runs. The offline suite is the contract: **the whole harness can be exercised without a network.** That is only possible because of the library boundary from Stage 12 and the fake model from Stage 13.

Checkpoint

1. Why does the test suite mirror the module graph?

A. It makes failures easy to locate: a module change maps to a suite.

2. How do you mock `fetch` safely?

A. Replace `globalThis.fetch`, assert on the request, restore in `finally`.

3. Why must the loop be testable without a model?

A. Because the loop is the agent; testing it without a network is the proof of the library boundary.

4. What are the three most important loop tests?

A. A plain answer stops; tool calls are dispatched and fed back; a failure result is fed back and recovered from. Add truncation and step-limit tests.

5. Why must memory tests use a temporary directory?

A. Memory writes to disk; a real `src/memory.md` would make tests order-dependent and destructive.

Stage 16: Documentation and final acceptance

What this stage is really about

Documentation is part of the architecture, not an afterthought. A README that only lists commands teaches nothing; a README that explains the loop, the tools, the safety model and the extension points teaches the reader to maintain the harness after you leave.

What the README must contain

- A one-paragraph summary.
- Quick start: `npm install`, copy `.env.example` to `.env`, set the key, `npm start`, `npm run demo`.
- The loop diagram and the seven beats.
- A file-by-file tour, with `agent.js` first.
- The eight tools and what each is for.
- The read/write asymmetry and the shell limitation.
- The nine lifecycle hooks, with a table.
- Long-term memory: file format, eager digest vs lazy recall, `--memory`.
- Sessions and history trimming.
- A configuration table with defaults and precedence.
- Exit codes.
- The safety model: workspace fence, hard denies, environment allow-list, command policy, and an explicit "this is not a container" statement.
- Examples: a Desktop write that ends in `FORBIDDEN`, a prompt that ends with `remember`.

Why "this is not a container" belongs in the docs

Because overclaiming safety is worse than admitting limits. A reader who believes the sandbox is isolation may run the agent on a machine where a mistake matters. A reader who understands it is a path policy and a command-line seatbelt will make an informed choice.

The same honesty applies to the other limits:

- no streaming;
- no context compaction, just a cut;
- no sub-agents, retries or skills;
- memory is one markdown file with substring search;
- one tool at a time in spirit.

Each limit is where a production agent grows a layer. Naming them tells the reader where to extend.

`demo_prompt.md`

Provide copy-pasteable prompts that exercise the whole system:

- `create hello.js that prints today's date and time, run it, and save its output to hello-output.txt`;
- `save a file test.txt that contains 'HELLO WORLD' and save to the desktop` (must end in `FORBIDDEN`);
- `npm run demo`, `node src/cli.js --memory`, `node src/cli.js --demo`;
- `work out how this package is tested, run the tests, and remember the command...`.

Why a prompt file? Because a demo is a performance, and a performance needs a script. It also doubles as an acceptance test: if a prompt stops producing the advertised result, the documentation is stale.

Final acceptance

The twenty checks in `PLAN.md` are the contract. Group them mentally:

- **Install and run:** `npm install`, `npm test`, `--help`, `--hooks`, `--memory`.
- **Demo:** `--demo` completes 14 steps, 7/7 checks, exit 0.
- **Live mode:** with a valid key, a task runs; without a key, exit 1.
- **Input edge cases:** no task + no TTY exits 2; step limit exits 3.
- **Safety:** write outside workspace refused; read outside allowed; `CMD_POLICY=deny` refuses commands; hard denies survive `--yes`; child cannot read the key.
- **Persistence and observability:** memory survives restarts; every run prints hook traces; `--no-hooks` silences them.
- **Sessions:** `/new`, `/quit`, `/exit`; follow-up sees history but a fresh prompt.
- **Architecture:** `agent.js` direct execution prints a hint; no framework, no agent SDK, no database, no hidden global state.

If all twenty hold, you have not just built a demo. You have built a harness whose behaviour you can explain, test and extend.

Checkpoint

1. Why is the README part of the architecture?

A. It teaches the architecture and the limits; without it, the harness is a black box that only you can maintain.

2. Why must the docs state the sandbox's limits?

A. Overclaiming safety leads to unsafe use; stating limits lets the reader make an informed choice.

3. What makes a good demo prompt?

A. One that exercises a real capability, has an observable result, and optionally demonstrates a guard failing safely.

4. Pick three final acceptance checks and explain which design decision makes each one pass.

A. Example: "write outside workspace refused" passes because `Workspace.resolve` is strict and the sandbox converts `PathError` into `FORBIDDEN`; "child cannot see key" passes because `safeEnv` allow-lists variables; "step limit exits 3" passes because `runTask` returns `reason: 'max_steps'` and `taskExitCode` maps it.

Appendices

A. The message protocol

The agent's entire state is an array of messages. Four roles:

ROLE	WRITTEN BY	REQUIRED FIELDS	NOTES
`system`	harness	`content`	Rebuilt per task; exactly one.
`user`	harness	`content`	The task or a corrective nudge.
`assistant`	model	`content`, optional `tool_calls`	Sanitised before appending.
`tool`	harness	`tool_call_id`, `content`	One per tool call; content is always a string.

Rules:

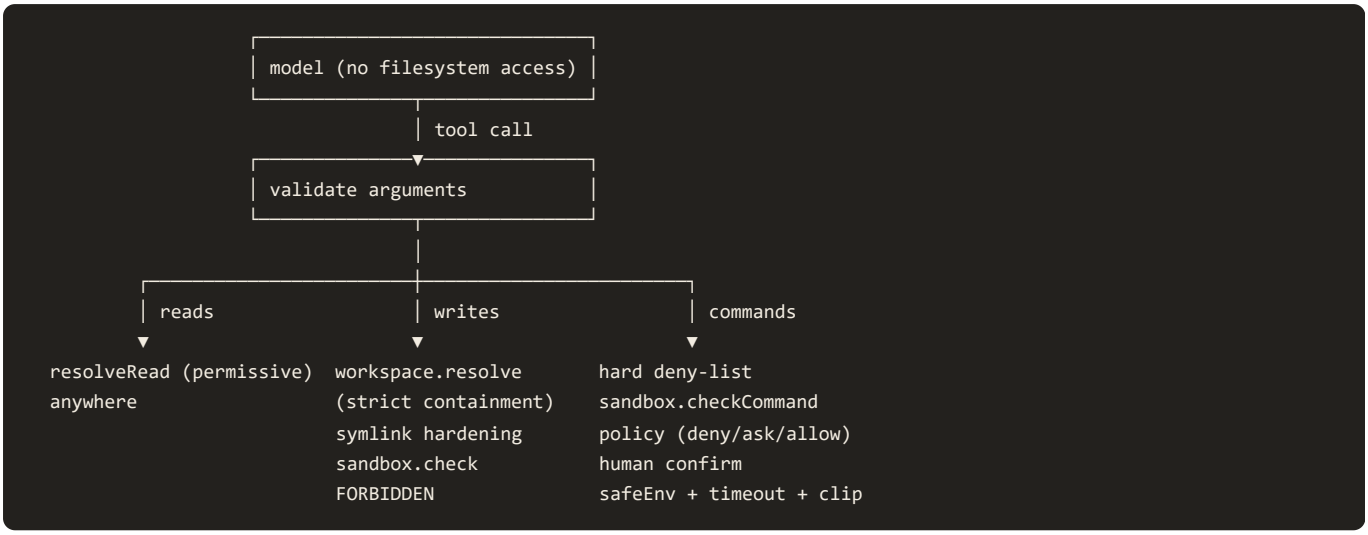
1. **Every `tool_call_id` gets exactly one matching `tool` message.** Otherwise the next request is malformed.
2. **Tool content is a string, even for errors.** `ERROR: ...`, `DENIED: ...`, `FORBIDDEN: ...`.
3. **The assistant message is appended before its tool results.** Order matters.
4. **The system message is not carried in history.** Filter it out and rebuild.
5. **A truncated assistant reply with no tool calls is appended, then followed by a `user` corrective message.** Never dispatch half a call.

A tool call in the wire format

```
{
  "id": "call_abc",
  "type": "function",
  "function": {
    "name": "read_file",
    "arguments": "{\"path\":\"src/cli.js\",\"limit\":40}"
  }
}
```

`arguments` is a JSON **string**. You must parse it. A model that emits invalid JSON gets an `ERROR: ... arguments are not valid JSON` observation, not a crash.

B. The safety model in one page



The five layers:

1. **Workspace containment** — writes must resolve inside `workspace/`, including after variable expansion and symlink resolution.
2. **Sandbox gate** — one place prints `FORBIDDEN` and converts policy errors into observations.
3. **Command scanning** — visible shell write targets are checked with the same policy before the command runs.
4. **Command policy** — `deny`, `ask`, `allow`; `ask` degrades to deny without a TTY.
5. **Process hygiene** — minimal child environment, timeout, output cap.

What it is not:

- not a container;
- not a syscall filter;
- not protection against a script that writes files;
- not a substitute for running the agent somewhere disposable.

C. Every bound and cap

BOUND	DEFAULT	WHERE	WHAT IT PREVENTS
`maxSteps`	12	`config.js`, `runTask`	A model that never concludes.
Request timeout	90 s	`llm.js`	A socket that never answers.
`max_tokens`	2048	`llm.js`	Whole files in one reply; runaway cost.
Command timeout	30 s	`tools.js`	A command that hangs the run.
Command output cap	6,000 chars/stream	`tools.js`	A chatty command eating the context window.
`maxBuffer`	1 MB	`child_process`	Memory exhaustion from output.
`list_files` depth	3 (max 6)	`tools.js`	Recursive listings that explode.
`list_files` lines	200	`tools.js`	A giant listing in context.
`read_file` limit	200 (max 2,000)	`tools.js`	Reading a huge file in one step.
Memory entries	60	`memory.js`	The file growing without limit.
Memory prompt chars	1,200	`memory.js`	The digest eating the context window.
Memory entry chars	300	`memory.js`	One note dominating.
`remember` chars	2,000	`memory.js`	One call dominating.
Digest entries	12	`memory.js`	Too many memories in the prompt.
`MAX_HISTORY`	24	`cli.js`	A session growing without bound.
`maxEntries` / tool caps	—	tests	Regression of any of the above.

Every bound has a visible consequence rather than a silent stall. That is the design rule.

D. Debugging by symptom

SYMPTOM	FIRST PLACE TO LOOK
`401` / bad key	`.env` precedence, `fingerprint`, `--model` flag.
`402`	OpenRouter credits, not the harness.
Model never calls tools	Tool schemas not passed, prompt does not mention tools, model lacks function calling.
Tool call arguments fail to parse	Truncation (`finish_reason: length`), `max_tokens`, model emits prose.
`old_string not found`	Model hallucinated whitespace; read the file first; copy exactly.
`old_string appears N times`	Add context or `replace_all: true`.
`FORBIDDEN` on a legitimate path	Path policy; check the first segment, `~`, `%VAR%`, symlinks.
Command never runs under `ask`	No TTY → deny by design.
Command runs without asking	`--yes`, `CMD_POLICY=allow`.
Child cannot find a command	`safeEnv` dropped a needed variable; check the allow-list.
Model loops forever	`maxSteps`; look for a tool result it cannot interpret.
Memory not injected	`--no-memory`, `MEMORY_PROMPT_CHARS=0`, empty digest.
Memory grows without bound	`MEMORY_MAX_ENTRIES`, duplicate handling.
Hooks do not fire	`--no-hooks`, event name typo, handler attached to the wrong event.
Hook blocks everything	Payload inspection; first block wins; check the reason.
Session forgets the previous task	`trimHistory`, `/new`, history cut on a tool message.
Provider rejects history	Cut lands on a `tool` result or an unanswered `tool_calls`.
Program hangs after a run	Heartbeat timer not `unref`'d or not stopped.

E. Glossary

Agent — a model plus tools plus a loop. The loop is the distinguishing part.

Assistant message — the model's reply: `content` and possibly `tool_calls`.

Bounded autonomy — every loop and every expensive operation has a cap.

Digest — the bounded, newest-first memory block injected into the system prompt.

Dispatcher — `runTool`: validates arguments, finds the handler, catches errors, returns a string.

Eager memory — the digest read at the start of a run.

`edit_file` — exact-snippet replacement with ambiguity detection.

`ForbiddenError` — typed refusal from the sandbox, carrying action, requested path and reason.

Hard deny-list — command patterns refused before policy is consulted.

Heartbeat — a timer that prints "still waiting" during a slow model call.

Hook — a function attached to a named lifecycle moment; may observe, rewrite or block.

Lazy memory — `recall`, a search performed on demand.

`maxSteps` — the number of model calls a single task may make.

Message array — the agent's entire working state.

Path policy — the rules in `workspace.js` that decide whether a path is contained.

Provenance — where a configuration value came from (flag, `.env`, shell, default).

ReAct — Reason, Act, Observe: the loop pattern.

`runTask` — the library function that runs one task.

`safeEnv` — the child-process environment allow-list.

Sandbox — the write fence around save/edit/delete and shell writes.

Sanitise — rebuild an assistant message with only the fields the harness needs.

Schema — the JSON description of a tool that the model reads.

Session — multiple tasks in one process, carrying conversation history.

System prompt — the model's job description, rebuilt per task.

Tool — a named capability with a schema and a handler.

`tool_call_id` — the identifier that ties a tool result to the assistant's request.

Truncation guard — refusing to dispatch a reply cut off by the length limit.

Workspace — the folder the agent may write to.

F. Reading the finished code

If you want to read the finished harness in the order that teaches the most:

1. `agent.js` — the whole loop, 215 lines including comments. Read it twice.
2. `prompt.js` — what the model is told, including memory.
3. `tools.js` — the schemas *are* the prompt; read the descriptions carefully.
4. `workspace.js` and `sandbox.js` — the safety story, in that order.
5. `hooks.js` — the extension points.
6. `memory.js` — the persistence story.
7. `llm.js` — one POST, plus sanitisation.
8. `config.js` — precedence and provenance.
9. `cli.js` — the program, if you need it.
10. `demo.js` — what runs when the wifi does not.

Read `agent.js` first and in full. It is the spine. Everything else is a dependency of it or a caller of it.

Closing note

You now have the full conceptual ladder:

- a model call is just a POST;
- a conversation is just an array;
- a tool is a schema plus a function that returns a string;
- a sandbox is a policy, not a container;
- a hook is a named pause;
- memory is a file;
- the loop is the agent.

Every one of those sentences is small enough to hold in your head, and together they explain the behaviour of every coding agent you will meet. The rest is layers.

Part II: The build companion — PLAN.md, stage by stage

This part is the marriage of the two documents. For every stage you get four things:

1. **Build** — the files to create or modify and the concrete contract, distilled from `PLAN.md`.
2. **Prove it** — the acceptance criteria, written as checkboxes you can actually run.
3. **Teach it** — what to emphasise, the live-demo cue, the usual mistakes, and roughly how long the stage should take.
4. **Try it** — one student exercise that is more than "run the command".

The theory chapters (Parts 0–E) answer *why*. This companion answers *what exactly* and *how do I know it works*. Read them as a pair: theory first, then the build entry, then type.

Prerequisites

- **Node.js 20 or newer** — ES modules, `AbortSignal.any`, and `node:test`.
- **One runtime dependency:** `chalk` **v5+ (ESM).** No framework, SDK, database or ORM.
- **An OpenRouter API key** for live mode only. The demo, the tests, `--help`, `--hooks` and `--memory` need no key and no network.
- **A terminal at least 100×30** for the banner and scorecard. Windows `cmd.exe`, PowerShell, Git Bash and a Unix shell are all usable; `run_command` inherits whichever shell launches the CLI.

How to use the companion

- **Students:** before a stage, read the theory chapter. During the stage, keep the build entry open. Before moving on, tick every box in *Prove it* and do *Try it*.
- **Teachers:** the *Teach it* block is your lesson plan in miniature. It names the one idea the stage exists to teach, the demo that lands it, the misconceptions to pre-empt, and a time budget.
- **Everyone:** the checkboxes are not decoration. If a box is unticked, the next stage will not make sense, because every stage depends on the previous contract.

A stage is done when the program does something new **and** every acceptance box is ticked. "It compiled" is not a stage.

Stage 0 build: Scaffold

Files to create - `package.json` - `.gitignore` - `workspace/.gitkeep` - `src/.gitkeep`

Build - `package.json`: `"type": "module"`, `"engines": { "node": ">=20" }`, exactly one runtime dependency (`chalk`), and scripts for `start`, `demo`, `test` (add `hooks` and `memory` once those flags exist). - `.gitignore`: `node_modules`, `.env`, and everything in `workspace/` except `.gitkeep`. - Project layout: `src/`, `test/`, `workspace/`, flat and boring.

Prove it - [ ] `cd mini-coding-agent-js && npm install` installs exactly `chalk`. - [ ] `npm ls --depth=0` shows `chalk` and nothing else in `dependencies`. - [ ] `node --test` reports `0 tests` without failing. - [ ] `workspace/` exists and is git-ignored except `.gitkeep`. - [ ] `type: "module"` is set. - [ ] `node --version` is 20 or newer.

Teach it

The one idea: constraints chosen on day zero shape every later stage. Two constraints are doing the work — one dependency, one writable folder.

Demo cue: run `npm ls --depth=0` live, then ask "what would this look like if we had installed an agent SDK?" Nothing to show, and that is the point: the model call would be behind a wall.

Usual mistakes: adding a second dependency "just for flags"; nesting `src/core/`; forgetting `"type": "module"` and then fighting `import`; committing `.env`.

Time: 20 minutes.

Try it: write a one-paragraph defence of the single-dependency rule that a colleague who wants `commander` would accept. Then write the one sentence that would change your mind.

Stage 1 build: Terminal language

Files to create - `src/ui.js` - `test/ui.test.js`

Build - Every helper returns a string. Nothing in `ui.js` prints. - `ui.result(text)` derives colour from the text: red when the first line starts with `ERROR`, `DENIED` or `FORBIDDEN`, or contains a non-zero `exit code N`; green otherwise. - `explainExit(code)` maps `0`, `1`, `2`, `3`, `126`, `127`, `128–164` and Windows `9009` to plain English. - `ui.forbidden(action, requested, reason)` prints one loud, greppable line: ⛔ `FORBIDDEN SAVE ../escape.txt - path escapes the workspace: ../escape.txt`. - Required helpers (exact surface from PLAN): `title`, `field`, `kv`, `step`, `think`, `tool`, `result`, `forbidden`, `pass`, `fail`, `rule`, `hookTitle`, `hookLine`, `hookField`, `hookLive`, `hookNote`, `hookTrace`, `hookFlow`, `memory`, `exitNote`, `answerRule`, `answer`, `warn`, `waiting`, `error`, `note`, `prompt`, `dim`.

Prove it - [ ] Every helper returns a string. - [ ] Styles become ANSI on a TTY and read back as the original text when colour is off. - [ ] A failed result is red; a clean result is green. - [ ] Exit code 1 gets a description via `explainExit`. - [ ] A successful command shows its first stdout line in a dim annotation. - [ ] `forbidden` produces bold red capitals. - [ ] Model text containing `<red>`, backslashes or JSON prints verbatim.

Teach it

The one idea: presentation is a separate decision from logic, and the contract is "return a string, let the caller print".

Demo cue: pipe a search for `console.log` through `src/ui.js` and show there are none. Then change the palette in one place and show the whole app change.

Usual mistakes: helpers that call `console.log`; a home-made tag parser that eats `<red>` in model output; deriving failure from a boolean passed in rather than from the observation itself.

Time: 60–90 minutes.

Try it: add `ui.kv` to the palette and write a test that asserts the label is padded and the value is not interpreted. Then explain why `result` should not take an `ok` argument.

Stage 2 build: Config and model client

Files to create - `src/config.js` - `src/llm.js` - `test/config.test.js` - `test/llm.test.js`

Build (config) - `PROJECT_ROOT = path.resolve(import.meta.dirname, '..')`. - `findEnvFile(startDir)`: nearest `.env` wins, walking to the filesystem root. - `loadDotEnv(file)`: `KEY=VALUE`, optional `export`, `#` comments, quoted values, blank values ignored. - **Precedence:** explicit overrides (`--model`) > nearest `.env` > shell environment > defaults. `.env` beats the shell on purpose. - `fingerprint(secret)`: `not set`; all `*` when ≤ 12 chars; otherwise `first5...last4 (N chars)`. - `normalizeModel`: bare names map to a vendor; a name containing `/` is left alone; a change is reported. - `loadConfig(overrides, { envFile, startDir })` returns values **and provenance**: `apiKeySource`, `modelSource`, `envFile`, `apiKeyOverrodeShell`, `modelNormalised`, and the bounds below. - Defaults: model `deepseek/deepseek-v4.1-flash-1017`, `CMD_POLICY=ask`, `MAX_STEPS=12`, `REQUEST_TIMEOUT_MS=90000`, `WORKSPACE_DIR=workspace`, `MEMORY_FILE=src/memory.md`, `MEMORY_MAX_ENTRIES=60`, `MEMORY_PROMPT_CHARS=1200`.

Build (llm) - `chat({ apiKey, model, messages, tools, signal, timeoutMs = 90000 })`. - Missing key throws `OPENROUTER_API_KEY is not set...`. - Compose caller signal and timeout with `AbortSignal.any([signal, AbortSignal.timeout(timeoutMs)])`. - POST `https://openrouter.ai/api/v1/chat/completions` with `Authorization`, `Content-Type`, `X-Title: min-coding-js`. - Body `{ model, messages, tools, max_tokens: 2048 }`. - Read the body as text first; parse after the status check. - `401` hints bad/revoked key; `402` hints out of credits; timeout says `request timed out after <n>s`; a `.error` field on a 200 is still an error. - `sanitizeAssistant(message)` rebuilds with only `role`, `content`, `tool_calls`; drops `reasoning` / `reasoning_content`; deletes `content` only when it is null and tool calls exist.

Prove it - [ ] `loadConfig` resolves `.env`, shell, overrides in the right order. - [ ] `.env` beats a stale shell key; shell still supplies names `.env` does not define. - [ ] A missing `.env` is not an error. - [ ] The fingerprint never exposes the whole key. - [ ] Bare model names normalise to `vendor/model`, and the change is reported. - [ ] Override is reported only when the value genuinely differed. - [ ] `sanitizeAssistant` drops reasoning and keeps only the three fields. - [ ] `llm` tests mock `fetch`; timeout, `401`, `402`, non-JSON and `.error` are all covered offline.

Teach it

The one idea: configuration is a set of facts with **provenance**, and transport failures throw while tool failures return strings.

Demo cue: put a deliberately stale key in the shell and a good one in `.env`; show the banner naming `.env` as the source. Then show `key: sk-or...abcd (73 chars)`, never the key.

Usual mistakes: printing the key; using `res.json()` before checking status; forgetting to restore a mocked `fetch`; treating `reasoning_content` as harmless.

Time: 20 minutes (config) + 2–3 hours (client and tests).

Try it: add one config value (say `TEMPERATURE`) end to end: parse, default, provenance, banner line, test. Then write down which layer should own the decision if two sources disagree.

Stage 3 build: Simplest chat loop

Files to create - `src/cli.js` (minimal) - `test/input.test.js`

Build - `askTask({ input, output })`: readline prompt `task ›`; trim; blank re-asks; Ctrl-C returns `null`; `for await (const raw of rl)`. - Hand-rolled flag parsing for `--help`, `--model`, `--max-steps`; every non-flag becomes the task. - Load config, build `[system, user]`, call `chat`, print `ui.answer`. - No task and stdin is not a TTY → hint and exit `2`. - Request failure → exit `1`. - `--help` works with no key.

Prove it - [ ] `node src/cli.js --help` prints usage with no key. - [ ] `node src/cli.js "Say hello"` reaches OpenRouter and prints the reply. - [ ] No task + non-TTY exits `2` with a helpful message. - [ ] Missing/invalid key exits `1` with a clear message. - [ ] `--model` overrides `.env`.

Teach it

The one idea: this is the first runnable payoff — config + transport + presentation in one short program.

Demo cue: live chat once, then ask "what did I just say?" and show it cannot know. That sets up Stage 4.

Usual mistakes: requiring a key for `--help`; hanging when stdin is not a terminal; letting Ctrl-C throw.

Time: 60–90 minutes.

Try it: add a `--dry-run` flag that prints the exact request body that *would* be sent, without a key. What does that teach you about testing?

Stage 4 build: Conversation history

Files to modify - `src/cli.js`

Build - Keep one `messages` array for the process. - Interactive loop with `interactive session - type a task below (/quit exits)`. - Commands `/quit`, `/exit`, `/new`. - Every turn appends `user` → model call → `assistant`. - `/new` clears the array and re-adds the system prompt, nothing else. - One-shot mode still works. - Exit prints `session over - N turn(s)`.

Prove it - [ ] A follow-up references the previous turn. - [ ] `/new` clears the conversation. - [ ] `/quit` and `/exit` end the session. - [ ] The session summary prints the turn count. - [ ] One-shot still works.

Teach it

The one idea: messages are the memory; the model has none.

Demo cue: ask for a name, then ask "what is my name?" — works. Then `/new` and ask again — fails. That contrast is the whole stage.

Usual mistakes: rebuilding `messages` each turn; making `/new` delete files; carrying the system message twice after `/new`.

Time: 45–60 minutes.

Try it: implement `/history` that prints the roles in order without contents. Use it to explain why the next request costs more than the first.

Stage 5 build: System prompt

Files to create - `src/prompt.js` - `test/prompt.test.js`

Files to modify - `src/cli.js`

Build - `systemRules(tools = [])` returns the prose contract: look before writing; prefer `create_file` for new files, `edit_file` for small changes, `write_file` for whole-file replacement; copy `old_string` exactly; run what you wrote; never invent tool results; be concise; one tool call per step; remember durable facts only. - PLAN's signature shows `systemRules(tools = TOOL_NAMES)`. At Stage 5 `tools.js` does not exist yet, so either default to `[]` or keep a local constant and let Stage 7 wire `TOOL_NAMES` in. `cli.js` passes `[]` explicitly for now. - `buildSystemPrompt({ memory, log })` returns rules plus an optional memory digest. - At this stage memory is absent and `cli.js` passes `tools = []`. - The system message is first on every turn; `/new` keeps it.

Prove it - [ ] Rules are the first `system` message on every turn. - [ ] `/new` keeps the system prompt. - [ ] No tool names are listed while no tools exist. - [ ] `buildSystemPrompt` accepts `memory` and `log` without using them yet. - [ ] `prompt.test.js` asserts the rule text and the injection point.

Teach it

The one idea: the prompt is program text — composed, tested, and rebuilt — not a magic string.

Demo cue: ask the model to edit a file it has not read, with and without rule 1. Show the difference in behaviour.

Usual mistakes: naming tools that do not exist; putting the prompt in `cli.js`; carrying a stale system message across tasks.

Time: 45–60 minutes.

Try it: delete one rule and predict the specific failure it causes. Then reproduce it. Restore the rule.

Stage 6 build: Path policy

Files to create - `src/workspace.js` - `test/workspace.test.js`

Build - Export `PathError`. - `new Workspace(root)`: resolve, `mkdir -p`, store `fs.realpathSync(root)`. - `workspace.resolve(requested)` — the strict write policy. Reject: empty/non-string; absolute paths; drive-relative (`C:foo`); UNC (`\\server\share`, `//server/share`); `~`, `$`, `%` prefixes; and a forbidden first segment (`users`, `windows`, `program files`, `desktop`, `documents`, `downloads`, `appdata`, `onedrive`, `library`, `system32`, and similar). - Expand `~`, `%VAR%`, `${VAR}`, `$VAR` **before** containment checks. - Symlink hardening: resolve the deepest existing ancestor with `fs.realpathSync`, re-append the not-yet-created tail, re-check containment. - `workspace.resolveRead(requested)` — permissive: absolute paths, outside paths, `~` all allowed. - `rel` is always `/`-separated; root is `.`.

Prove it - [ ] Legitimate relative paths resolve and `rel` is correct. - [ ] Every escape shape in the rejection list is refused. - [ ] Reads outside the workspace are allowed. - [ ] A symlinked folder inside the workspace cannot smuggle a write outside. - [ ] A missing path is still checked against its deepest existing ancestor. - [ ] `PathError` is distinguishable from an I/O error.

Teach it

The one idea: reads are open, writes are fenced — and containment is a policy, not `startsWith`.

Demo cue: put a junction inside the workspace pointing at the Desktop, then ask for a write through it. Show the refusal.

Usual mistakes: checking `startsWith(root)` alone; expanding variables after the check; treating `/home` on Unix as a Windows switch; forgetting that the file does not exist yet so `realpath` must stop at the deepest existing ancestor.

Time: 2–3 hours.

Try it: write a table of ten escape attempts, predict each refusal reason, then run the tests. Which one surprised you?

Stage 7 build: Read-only tools

Files to create - `src/tools.js` (partial: `list_files`, `read_file`) - `test/tools.test.js` (read-only tests)

Files to modify - `src/prompt.js` (mention the two tools)

Build - OpenAI function-calling schema shape: `{ type: 'function', function: { name, description, parameters: { type: 'object', properties, additionalProperties: false } } }`. - `validate(name, rawArgs)`: raw may be a JSON string or an object; parse, type-check, integer-check, required fields, unknown keys. - `runTool(name, rawArgs, ctx)`: dispatch; unknown tool → helpful error string; handler exceptions become text observations. - `list_files`: `path` default `'.'`; `depth` default 3, max 6; `resolveRead`; missing → `ERROR: no such folder: <rel>`; walk recursively, sorted; skip `.git`, `node_modules`, `.venv`, `__pycache__`; directories print `<rel>/`, files print `<rel> (<size>b)`; stop at depth and 200 lines; end with `<dirs> folder(s), <files> file(s) under <start.rel>`. - `read_file`: `path` required; `offset` default 1; `limit` default 200, max 2000; `resolveRead`; missing → `ERROR: no such file: <rel>`; directory → `ERROR: <rel> is a folder - use list_files`; header `"<rel> (<total lines>)"`; numbered lines right-aligned width 4 then `|`; truncation marker `... [<n> more lines]`; clamp offset ≥ 1 and limit 1..2000. - CLI: build `TOOL_SCHEMAS`, pass to `chat`; after a reply with `tool_calls`, dispatch, append `role: 'tool'` results, call the model again. Print `ui.tool` and `ui.result`.

Prove it - [ ] The model can call `list_files` and see a listing. - [ ] The model can call `read_file` and see numbered contents. - [ ] Bad arguments return text errors, not crashes. - [ ] Reading outside the workspace succeeds (`../package.json`). - [ ] Missing file → `ERROR: no such file`. - [ ] Directory read → `ERROR: ... is a folder`. - [ ] Unknown tool name gets a helpful list. - [ ] Every `tool_call_id` receives exactly one `tool` message.

Teach it

The one idea: tool calling is the model's only ability to act, and the schema *is* the prompt.

Demo cue: change a tool description live and show the model choose differently. That is more convincing than any slide about prompts.

Usual mistakes: forgetting `arguments` is a JSON string; throwing from a handler; allowing a listing to walk `node_modules`; showing a truncated file without the marker.

Time: 2–3 hours.

Try it: add a deliberately bad argument to a test (`"limit": "20"`) and improve the error until a reader knows exactly what to change.

Stage 8 build: Sandbox and write tools

Files to create - `src/sandbox.js` - `test/sandbox.test.js`

Files to modify - `src/tools.js` (add `create_file`, `write_file`, `edit_file`; PLAN also lists a `delete_file` tool — see the note below) - `src/cli.js` (wire the sandbox)

> **Note on `delete_file`.** PLAN lists `delete_file` at this stage and the sandbox must support `remove`. The finished eight-tool harness keeps `sandbox.remove` and its root guard but does not expose a model-facing `delete_file` tool. Implement it if you want the ninth tool, but keep the root guard and the same `FORBIDDEN` contract; do not let the tool count drift in the README without saying so.

Build - `SANDBOX_DIR = path.resolve(import.meta.dirname, '..', 'workspace')`. - `ACTIONS = { SAVE: 'save', EDIT: 'edit', DELETE: 'delete' }`. - `ForbiddenError` carries `action`, `requested`, `reason`; `name = 'ForbiddenError'`, `code = 'FORBIDDEN'`. - `new Sandbox({ workspace, root, log })`. - `sandbox.check(action, requested)`: unknown action → `TypeError`; delegate to `workspace.resolve`; convert `PathError` to `ForbiddenError`; increment `refusals`; print one `ui.forbidden`; throw. - `sandbox.save(requested, content)`: check, create parents, write, return `{ existed, bytes }`. - `sandbox.remove(requested)`: check, `rmdirSync` for empty dirs, `unlinkSync` for files; missing → `{ missing: true }`; refuse deleting the root. - `sandbox.summary()`: `"save, edit, delete - only inside <root>"`. - `create_file`: check; exists → `ERROR: already exists - use edit_file or write_file`; else write and return `created <rel> (<bytes> bytes)`. - `write_file`: check; create parents; overwrite; return `created` or `overwrote`. - `edit_file`: `path`, `old_string`, `new_string`, `replace_all` (default false); check; missing file → `ERROR`; empty `old_string` → `ERROR`; count exact occurrences; 0 → `ERROR: old_string not found... Read the file and copy exactly, including whitespace.`; >1 without `replace_all` → `ERROR: old_string appears N times...`; replace literally (callback form or `split/join`); return `edited <rel>: <n> replacement(s), first at line <line>`.

Prove it - [ ] `create_file` writes inside the workspace. - [ ] `create_file` refuses to clobber. - [ ] `write_file` overwrites. - [ ] `edit_file` works with exact matching. - [ ] `edit_file` treats `$1` literally. - [ ] `edit_file` fails on missing file, empty `old_string`, and ambiguity. - [ ] Every escape shape is refused for save/edit/delete. - [ ] Deleting the sandbox root is refused. - [ ] Unknown action to `check()` throws `TypeError`. - [ ] A refused write prints exactly one `FORBIDDEN` line.

Teach it

The one idea: defense in depth — the workspace answers "is it contained?", the sandbox answers "may this action happen at all?"

Demo cue: run a write to `../escape.txt` and show the single loud line, then show the model receiving `FORBIDDEN:` and choosing another path.

Usual mistakes: formatting refusals in three places; using `String.replace` with a replacement string (the `$&` trap); treating an unknown action as a refusal; letting `create_file` overwrite.

Time: 3–4 hours.

Try it: write a test that proves `edit_file` does not interpret `$&`. Then explain why a silent first-match replacement would be worse than an error.

Stage 9 build: Command execution

Files to create - `test/shell-sandbox.test.js`

Files to modify - `src/sandbox.js` (`checkCommand`, `tokenizeCommand`, `shellWriteTargets`) - `src/tools.js` (`run_command`)

Build - `tokenizeCommand(text)`: split on whitespace honouring single/double quotes; emit operators as tokens (`>`, `>>`, `<`, `|`, `||`, `&`, `&&`, `;`); strip quotes. - `shellWriteTargets(command)`: - redirections `>`/`>>` → following token (handle `2> file`); - `DEST_LAST` commands (`copy`, `cp`, `move`, `mv`, `xcopy`, `ren`, `rename`, `ln`, `install`, `scp`, `rsync`) → last positional, at least two; - `robocopy` → positional argument 2; - `ALL_POSITIONAL` commands (`del`, `erase`, `rd`, `rmdir`, `rm`, `md`, `mkdir`, `touch`, `tee`, `unlink`, `truncate`, `shred`) → every positional; - downloaders: `curl` `-o`/`--output`/`-O`/`--remote-name`, `wget` `-O`/`--output-document`; - PowerShell: `out-file`, `set-content`, `add-content`, `new-item`, `ni`, `remove-item`, `copy-item`, `move-item`, `rename-item`, with `-destination`, `-dest`, `-path`, `-filepath`, `-literalpath`; - strip `.exe`, `.cmd`, `.bat`, `.com`, `.ps1`; compare case-insensitively; - a switch is `--flag`, `-f`, or Windows `/x`; a POSIX path like `/home` is not a switch. - `run_command` order: trim → empty error → hard deny-list → `sandbox.checkCommand` → policy (`deny`/`allow`/`ask`) → human confirm → execute. - CLI: add `--yes`/`-y` and the `--allow-commands` alias; implement `makeConfirm()` in `cli.js` so the approval prompt is one injectable function. - Hard deny-list before policy: recursive delete of a root path; `del` with `/s` or `/q`; `format`; `diskpart`; `shutdown`; `reboot`; `git push --force`/`-f`; `curl`/`wget` piped into `sh`/`bash`; fork bomb; `> /dev/sd...`. - Execute with `node:child_process.exec` in `workspace.root`, `timeout: 30000`, `maxBuffer: 1 MB`, `windowsHide: true`, `encoding: 'utf8'`. - `safeEnv(source)` keeps only `PATH`, `PATHEXT`, `HOME`, `USERPROFILE`, `TMPDIR`, `TEMP`, `TMP`, `SYSTEMROOT`, `COMSPEC`, `SHELL`, `LANG`, `LC_ALL`, `TERM`. - Result format: `exit code <n>`; optional `stdout:`; optional `stderr:`; `(no output)` if both empty; clip each stream to 6,000 chars with `... [<n> chars truncated]`; on timeout append `[tool] command timed out after 30s` and return non-zero.

Prove it - [ ] `run_command` returns exit code, stdout, stderr. - [ ] Hard-denied commands are refused even with `--yes`. - [ ] `CMD_POLICY=deny` refuses all commands. - [ ] A shell copy to the Desktop is refused before running. - [ ] A redirect outside is refused before running. - [ ] Quoted arguments with spaces stay one token. - [ ] The child env does not contain `OPENROUTER_API_KEY`. - [ ] A timeout is handled gracefully. - [ ] `safeEnv` keeps only allow-listed names.

Teach it

The one idea: layered policy. No layer is sufficient; together they make the common case safe and the residual risk explicit.

Demo cue: run the same `copy` command under `ask`, then show it is refused by the scan *before* the prompt. The absence of a prompt is the lesson.

Usual mistakes: checking policy before the hard deny-list; letting `ask` become allow when there is no TTY; inheriting the parent environment; trying to parse a full shell with a tokenizer and pretending it is complete.

Time: 4–5 hours.

Try it: write five commands your scanner should catch and five it deliberately will not (obfuscated writes). Explain why "not caught" is not the same as "safe".

Stage 10 build: Long-term memory

Files to create - `src/memory.js` - `src/memory.md` (initial store) - `test/memory.test.js`

Files to modify - `src/prompt.js` (inject the digest) - `src/tools.js` (add `remember`, `recall`) - `src/cli.js` (wire memory)

Build - Format: `- [tag] text _(YYYY-MM-DD)_`; tag ∈ `fact`, `pref`, `fix`, `gotcha`. - `parseMemory`: accept `-` or `*`, optional date, normalise whitespace, ignore prose/headings. - `Memory({ root, file, maxEntries = 60, maxEntryChars = 300, rememberChars = 2000, promptChars = 1200, promptEntries = 12, now })`. - `remember(text, { tag })`: normalise; truncate; empty → error; duplicate refreshes date; new appends; drop oldest over `maxEntries`. - `forget(match)`: delete entries whose text contains the match, case-insensitive. - `search(query, { limit = 20 })`: AND over words; empty query = all; newest first. - `digest({ entries, chars })`: newest first; add while it fits; state hidden count. - `summary()`: `<label> (<n> memory/memories[, newest YYYY-MM-DD])`. - `remember` tool: params `text` (required), `tag` (optional); no store → `ERROR: no memory store in this run (started with --no-memory)`; empty → `ERROR: nothing to remember - the note was empty`; duplicate → `already remembered... date refreshed`; new → `remembered...`. - `recall` tool: params `query` (optional), `limit` (default 20); no hits → `no memories in <label> match "<query>" (<size> stored)`. - CLI: `--memory` side-door prints store, digest and raw file, exits 0; `--no-memory`; `MEMORY_FILE`.

Initial `src/memory.md` — the store is a normal file a student can open:

```
# memory

Long-term notes this agent keeps between runs. One bullet per memory:
`- [tag] text _(YYYY-MM-DD)_` where tag is fact | pref | fix | gotcha.

This file IS the store - edit it by hand, delete a wrong line, or commit it.
The agent reads a bounded slice at the start of a run and can search the rest
with the `recall` tool.

- [pref] The user's name is Craig. _(2026-10-28)_
- [pref] The user lives in Brighton, UK. _(2026-10-28)_
- [gotcha] The run_command shell in this sandbox is Windows cmd.exe, so Unix tools like `cat`, `ls` may fail; use
`type`, `dir` etc. instead. _(2026-10-28)_
- [pref] The user is a Python developer. _(2026-10-28)_
- [pref] The user has two dogs. _(2026-10-28)_
```

Prove it - [ ] Write and read back survive a new `Memory` instance. - [ ] Duplicate refreshes the date; no duplicate bullet. - [ ] `maxEntries` drops the oldest. - [ ] Long memory is truncated to the per-entry cap. - [ ] Empty/whitespace note is refused. - [ ] The digest is bounded, newest first, and states the hidden count. - [ ] Empty/disabled store adds and logs nothing. - [ ] `forget` removes matches. - [ ] Hand edits are picked up immediately. - [ ] `remember` / `recall` work through tools. - [ ] No memory store returns an error string, not a throw. - [ ] `--memory` prints the digest and exits 0. - [ ] `--no-memory` disables memory for that run.

Teach it

The one idea: memory is a file, not a mystery; eager digest and lazy recall are two cost profiles for the same store.

Demo cue: `cat src/memory.md` on screen, edit it by hand, run again, and show the change is already known. Then `node src/cli.js --memory` and read the exact injected block.

Usual mistakes: caching entries and letting the cache disagree with the file; writing task progress instead of durable facts; making the digest unbounded; forgetting to state what was hidden.

Time: 3–4 hours.

Try it: set `MEMORY_PROMPT_CHARS=0`, run a task that must recall, and explain the trade-off. Which tasks get worse? Which get cheaper?

Stage 11 build: Lifecycle hooks

Files to create - `src/hooks.js` - `test/hooks.test.js`

Files to modify - `src/cli.js` (wire hooks into the manual tool loop)

Build - Nine events with exact names, icons, colours, when and powers:

#	NAME	ICON	COLOUR	WHEN	CAN
1	`SessionStart`	🚀	cyan	once, before the first model call	refuse, note
2	`UserPromptSubmit`	✍️	blue	once, as the task enters the loop	rewrite task, block
3	`PreModel`	🧠	magenta	before every model call	observe
4	`PostModel`	💬	cyan	after every reply	observe
5	`PreToolUse`	🛠️	yellowBright	before every tool call	block
6	`PostToolUse`	📦	green	after every tool call	replace observation
7	`Stop`	🏁	greenBright	the model answered, loop ending	observe
8	`StepLimit`	⏳	yellow	maxSteps reached	observe
9	`SessionEnd`	🔒	red	always, finished or not	observe

- Four deciding hooks: `SessionStart`, `UserPromptSubmit`, `PreToolUse`, `PostToolUse`.
- Return shapes: `{ note }`, `{ task }` (submit only), `{ replace }` (post-tool only), `{ block: true, reason }`; first block wins.
- Thrown hooks are reported and ignored; never fatal.
- `Hooks`: `on(name, fn, { id, once })`, `off(name, id)`, `count(name)`, `has(name)`, `attached()`, `run(name, payload)`.
- `traceHooks({ log, detail, explain })`: one tracer per event; prints `<icon> [HOOK:<Name>] <when> - <detail>`.
- `printHookCatalogue({ log, hooks })`; `HOOK_SUMMARY = { total: 9, deciding: 4, names }`.
- CLI: `--hooks` prints the catalogue and exits 0; `--trace-hooks` (default on); `--no-hooks`; fire all nine at the right moments.

Prove it - [ ] Every hook has icon, colour, when, gets, may, why. - [ ] The catalogue prints all 9 and marks attached hooks. - [ ] An unknown hook name throws. - [ ] Handlers run in order; the first block wins. - [ ] A throwing hook is reported, not fatal. - [ ] Notes print; `once` handlers detach. - [ ] `off` by id works. - [ ] Trace prints `[HOOK:Name]` by default. - [ ] `--no-hooks` silences trace. - [ ] `--hooks` prints the catalogue with no key.

Teach it

The one idea: a tool is how the model acts on the world; a hook is how *you* act on the loop.

Demo cue: attach a twelve-line `PreToolUse` hook that blocks `git push`; show the model receive `DENIED:` and try another route.

Usual mistakes: building hooks as notifications only (no decisions); letting a thrown hook kill the loop; letting a later hook unblock; hard-coding the catalogue separately from the event list.

Time: 3–4 hours.

Try it: write a `PostToolUse` redactor for `sk-or-...` and a `SessionEnd` note that prints the duration. Then prove a key cannot reach the model by running the demo.

Stage 12 build: The ReAct loop

Files to create - `src/agent.js` - `src/heartbeat.js` - `test/agent.test.js`

Files to modify - `src/cli.js` (replace the manual loop with `runTask`)

Build - `runTask({ task, workspace, sandbox = null, commandPolicy, maxSteps, chatFn, confirm, hooks = null, memory = null, history = [], log = console.log, heartbeatMs = 5000 })`. - Return `{ ok, answer, messages, steps }` or `{ ok: false, reason, messages }`. - Rebuild the system prompt every task; filter old system messages out of `history`. - `ctx = { workspace, sandbox, commandPolicy, memory, confirm }`; `confirm` defaults to deny. - `hooks ?? new Hooks()` so every call site can call `hook.run` unconditionally. - Direct execution prints the "agent.js is the loop, not the program" hint; never import `cli.js`. - Seven beats per step: 1. blank line + `ui.step`. 2. fire `PreModel`. 3. start heartbeat; call `chatFn`; on throw stop heartbeat, print error, fire `SessionEnd`, return `request_failed`; otherwise stop heartbeat. 4. extract choice/message; fire `PostModel`. 5. truncation guard: `finish_reason === 'length'` with no tool calls → append assistant, append user "You were cut off by the length limit. Continue with smaller tool calls.", warn, `continue`. Never dispatch truncated calls. 6. append `sanitizeAssistant`; print `ui.think` if content. 7. no calls → print answer, fire `Stop`, `SessionEnd`, return `ok`; calls → for each: print tool, fire `PreToolUse`, block or run, print result, fire `PostToolUse`, append exactly one tool message. - After `maxSteps`: print error, fire `StepLimit` then `SessionEnd`, return `max_steps`. - Entry guards: `SessionStart` / `UserPromptSubmit` blocked → return `blocked_by_hook`; a rewrite replaces `messages[taskIndex]`. - `heartbeat(log, everyMs)` → `{ stop }`; prints `… still waiting for the model (<s>s)`; `timer.unref?.()`.

Prove it - [ ] Tools run, then the loop stops on a plain answer. - [ ] A failing tool result is fed back so the model can correct. - [ ] The loop is capped at `maxSteps`. - [ ] A truncated reply is not dispatched; the model is asked to continue smaller. - [ ] The heartbeat prints waiting lines during slow calls. - [ ] Blocked hooks produce the right exit path. - [ ] The step limit fires `StepLimit` then `SessionEnd`. - [ ] A follow-up task sees previous conversation but a fresh system prompt. - [ ] Direct execution prints the hint and does not import the CLI.

Teach it

The one idea: the loop is the agent; extracting it late makes it understandable, and making it a library makes it testable.

Demo cue: run a task with a fake `chatFn` in a test file, live, and show the whole agent run with no network.

Usual mistakes: carrying the system message in history; dispatching truncated calls; leaking a heartbeat timer; throwing task outcomes instead of returning them; letting `agent.js` import `cli.js`.

Time: 4–6 hours.

Try it: write a fake model that always returns a tool call and assert the step limit. Then write one that returns a bad tool name and assert the loop recovers.

Stage 13 build: Offline demo

Files to create - `src/demo.js`

Files to modify - `src/cli.js` (`--demo`) - `test/agent.test.js` (demo assertions)

Build - `demoChat()` returns an async `chatFn` walking `DEMO_SCRIPT`. - Tool call shape: `{ id, type: 'function', function: { name, arguments: JSON.stringify(args) } }`. - 14 tool steps + final answer: 1. `list_files` `{ path: '.' }` 2. `create_file` `{ path: 'demo/hello.js', content: ... }` 3. `run_command` `{ command: 'node demo/hello.js' }` 4. `edit_file` on the greeting 5. `run_command` again 6. `write_file` `../escape.txt` → expect `FORBIDDEN` 7. `read_file` `../package.json` → allowed 8. `write_file` `~/Desktop/pwned.txt` → expect `FORBIDDEN` 9. `run_command` shell copy to Desktop → expect `FORBIDDEN` 10. `run_command` `git push --force` → expect `DENIED` 11. `run_command` that echoes `OPENROUTER_API_KEY` → expect `absent` 12. `run_command` that exits 1 → expect `exit code 1` 13. `remember` 14. `recall` - Each assistant step has a short natural-language `content` before its tool call. - `maxSteps = DEMO_SCRIPT.length + 1 = 15`. - `sandboxScorecard(messages)`: map `tool_call_id` to result; regex checks; rows `{ ok, label, seen }`. - Checks: `s6 /^FORBIDDEN: SAVE \.\.\/escape\.txt/`; `s7 /^\.\.\/package\.json \(\d+ lines\)/`; `s8 /^FORBIDDEN: SAVE ~\/Desktop\/pwned\.txt/`; `s9 /^FORBIDDEN: COMMAND COPY /`; `s10 /^DENIED: blocked by policy/`; `s11 /OPENROUTER_API_KEY=absent/`; `s12 /^exit code 1\b/`. - CLI: `--demo` uses the scripted chat, `maxSteps = 15`, auto-approves commands, prints the scorecard; exits 0 on 7/7, else `process.exitCode = 1`.

Prove it - [ ] `node src/cli.js --demo` completes 14 steps + final answer. - [ ] It creates/edits/runs `workspace/demo/hello.js`. - [ ] It refuses `../escape.txt`, `~/Desktop/pwned.txt`, and the shell copy. - [ ] It allows the read of `../package.json`. - [ ] It denies `git push --force`. - [ ] The child prints `OPENROUTER_API_KEY=absent`. - [ ] It reports exit code 1 with the explanation. - [ ] It remembers and recalls. - [ ] The scorecard prints **7/7 checks held**. - [ ] The process exits 0.

Teach it

The one idea: a demo can be a test; the fake model makes the safety guards reproducible.

Demo cue: run `--demo` twice. The second run hits `create_file` refusing to clobber and still finishes cleanly. That is recoverability, live.

Usual mistakes: setting `maxSteps` to 14; letting the demo skip the real sandbox; treating a red row as a presentation bug rather than a regression.

Time: 2–3 hours.

Try it: add an eighth scorecard check for a guard the course does not yet cover (for example, a `delete_file` of the root). Make it pass, then make it fail on purpose.

Stage 14 build: Sessions and CLI polish

Files to modify - `src/cli.js` (major polish) - `test/session.test.js`

Build - `MAX_HISTORY = 24`. - `trimHistory(messages, max)`: drop system; keep at most `max`; move `start` forward until it lands on a `user` message; if no safe boundary, return `[]`. - `runSession`: `askTask` → command or task → `runTask({ ...base, task, history })` → `history = trimHistory(result.messages)`. - Every task gets a fresh step budget. - After each task: divider, pass/fail, message-count hint, retry hint. - On exit: `session over - N task(s) run.` - Banner before step 1: workspace, sandbox, model + source, key fingerprint + source, env file, command policy, timeout, tools, memory summary, hooks state. - Warnings: live mode + model lacks `/`; normalised bare name note. - Exit codes: `0` normal; `1` request/fatal; `2` no task/no TTY; `3` step limit. - `taskExitCode(result)` mapping as above. - `main()` catches fatal errors, prints `fatal: <stack>`, sets `process.exitCode`. - Only call `main()` when `import.meta.url === pathToFileURL(process.argv[1]).href`.

Prove it - [ ] `/new` clears conversation history but leaves workspace and memory untouched. - [ ] `/quit` and `/exit` end the session. - [ ] A follow-up sees previous conversation but a fresh system prompt with the latest memory. - [ ] Every task starts its own step budget. - [ ] A task-rewriting hook rewrites the new task, not history. - [ ] No task + non-TTY exits 2. - [ ] Step limit exits 3. - [ ] Live mode without a key exits 1. - [ ] The banner prints on every run. - [ ] The exit note explains non-zero codes.

Teach it

The one idea: task versus session, and honesty at the boundaries — trimmed history, reproducible banner, meaningful exit codes.

Demo cue: run two tasks in one session, show the message count grow, then `/new` and show it reset while the workspace file survives.

Usual mistakes: cutting history at an arbitrary index; carrying the system prompt; forgetting that providers reject an unanswered `tool_calls` tail; letting tests import `cli.js` and run `main()`.

Time: 3–4 hours.

Try it: force a history that would start with a `tool` message and prove `trimHistory` returns a valid boundary (or `[]`).

Stage 15 build: Full test suite

Files to create - Any remaining test files; make every prior suite comprehensive.

Files to modify - Bring each `test/*.test.js` up to the original specification.

Build / required suites - `workspace.test.js` — path policy, containment, symlink safety. - `sandbox.test.js` — refusals, root deletion, tool-layer conversion. - `tools.test.js` — all eight tools, validation, `safeEnv`, policy. - `shell-sandbox.test.js` — tokenisation, write-target scanning. - `agent.test.js` — loop behaviour, truncation, heartbeat, demo guards. - `hooks.test.js` — registry, blocking, replacing, tracing, catalogue. - `memory.test.js` — round-trip, duplicates, bounds, digest, forget, search. - `prompt.test.js` — rules, injection, logging. - `llm.test.js` — timeout, errors, sanitisation. - `config.test.js` — `.env` precedence, fingerprint, model normalisation. - `session.test.js` — commands, trimming, follow-up tasks. - `ui.test.js` — styling, verbatim output, result classification. - `input.test.js` — `askTask`, blank lines, Ctrl-C.

Prove it - [ ] `npm test` runs fully offline. - [ ] All suites pass. - [ ] No real network calls. - [ ] No API key required.

Teach it

The one idea: the test suite is the architecture made executable; if a module cannot be tested alone, the boundary is wrong.

Demo cue: unplug the network (or set an invalid key) and run `npm test`. It should not care.

Usual mistakes: leaking a mocked `fetch`; writing memory into the real `src/memory.md`; asserting ANSI codes without accounting for TTY; testing the model instead of the loop.

Time: 6–10 hours.

Try it: pick the weakest suite and add one test that would have caught a real bug you hit earlier in the course.

Stage 16 build: Documentation and acceptance

Files to create - `README.md` - `demo_prompt.md` - `.env.example` (if absent)

Build — README - One-paragraph summary. - Quick start: `npm install`, copy `.env.example` to `.env`, set the key, `npm start`, `npm run demo`. - The loop diagram and the seven beats. - File-by-file tour, `agent.js` first. - The eight tools and what each is for. - The read/write asymmetry and the shell limitation. - The nine lifecycle hooks, with a table. - Long-term memory: file format, eager digest vs lazy recall, `--memory`. - Sessions and history trimming. - Configuration table with defaults and precedence. - Exit codes. - Safety model: workspace fence, hard denies, environment allow-list, command policy, and an explicit "this is not a container" statement. - Examples: a Desktop write that ends in `FORBIDDEN`; a prompt that ends with `remember`.

Build — `demo_prompt.md` - `create hello.js that prints today's date and time, run it, and save its output to hello-output.txt` - `save a file test.txt that contains 'HELLO WORLD' and save to the desktop` (must end in `FORBIDDEN`) - `npm run demo`, `node src/cli.js --memory`, `node src/cli.js --demo` - `work out how this package is tested, run the tests, and remember the command...`

Prove it — the twenty final checks - [ ] 1. `npm install` succeeds with only `chalk` as a runtime dependency. - [ ] 2. `npm test` runs fully offline and all suites pass. - [ ] 3. `node src/cli.js --help` prints usage without a key. - [ ] 4. `node src/cli.js --hooks` prints the full nine-hook catalogue without a key. - [ ] 5. `node src/cli.js --memory` prints the store and the exact digest. - [ ] 6. `node src/cli.js --demo` completes the script with **7/7 checks held**. - [ ] 7. Live mode with a valid key runs a task through OpenRouter. - [ ] 8. Live mode without a key prints the missing-key error and exits 1. - [ ] 9. No task + no TTY exits 2. - [ ] 10. A task that never yields an answer stops at the step limit and exits 3. - [ ] 11. A write outside `workspace/` prints one ⛔ `FORBIDDEN` line and returns a `FORBIDDEN:` observation without touching disk. - [ ] 12. A read outside `workspace/` succeeds. - [ ] 13. `CMD_POLICY=deny` refuses all commands; hard-denied commands are refused even with `--yes`. - [ ] 14. The child shell cannot read `OPENROUTER_API_KEY`. - [ ] 15. Memory survives process restarts; duplicates refresh; the file is bounded; hand edits are picked up. - [ ] 16. Every normal run prints `[HOOK:Name]` traces; `--no-hooks` silences them. - [ ] 17. `/new` clears conversation history but leaves workspace and memory untouched; `/quit` and `/exit` end the session. - [ ] 18. A follow-up sees previous conversation but a freshly rebuilt system prompt with the latest memory. - [ ] 19. `agent.js` executed directly prints the hint and does not run the CLI. - [ ] 20. No framework, no agent SDK, no database, no hidden global state.

Teach it

The one idea: documentation is part of the architecture; overclaiming safety is worse than naming limits.

Demo cue: hand the README to someone who has never seen the project and ask them to explain the loop back to you. That is the acceptance test for the docs.

Usual mistakes: a README that is only commands; hiding the "not a container" caveat; letting `demo_prompt.md` drift from the actual behaviour.

Time: 3–4 hours.

Try it: add a "Known limits and where to extend" section to the README, then pick one limit and write the design sketch for the layer that would remove it.

The course checklist

Copy this into your notes and tick as you go.

STAGE	MODULE	WHAT YOU CAN DO AFTER THIS STAGE	ACCEPTED
0	Scaffold	`npm install` works	☐
1	`ui.js`	Print styled terminal output	☐
2	`config.js` + `llm.js`	Load `.env`, call OpenRouter	☐
3	Minimal `cli.js`	**Chat with the model** 🎉	☐
4	History in `cli.js`	Have a **multi-turn conversation**	☐
5	`prompt.js`	Inject behavioural rules	☐
6	`workspace.js`	Fence writes, open reads	☐
7	Read tools	Model **lists and reads files**	☐
8	Sandbox + write tools	Model **creates and edits files** safely	☐
9	`run_command`	Model **runs shell commands** with policy	☐
10	`memory.js`	Model **remembers and recalls** across runs	☐
11	`hooks.js`	Observe/rewrite/block lifecycle events	☐
12	`agent.js`	Extract the loop into a **testable library**	☐
13	`demo.js`	**Offline demo** proves all guards hold	☐
14	Full CLI polish	**Sessions**, history trimming, banner, exit codes	☐
15	Tests	`npm test` passes fully offline	☐
16	Docs	README teaches the architecture	☐

Part III: The teacher's handbook

This part is for the person running the room. It assumes you have read Parts 0–E and skimmed Part II. It gives you a plan you can teach from, not a script you must read verbatim. The best version of this course is a sequence of **predictions, demonstrations and repairs**: you ask the room what will happen, run it, and then explain the gap.

1. What this course is and is not

It is a course in *harness engineering*: the code around a model that turns text into action. By the end, students can explain and build:

- a configuration and transport layer with provenance;
- a message protocol and a conversation loop;
- tool schemas, validation, dispatch and string observations;
- a path policy, a write sandbox and a command policy;
- file-backed memory and lifecycle hooks;
- a testable ReAct loop and an offline demo.

It is not a course in prompt tricks, model training, or building a product. It deliberately uses one model provider and one runtime dependency. The transferable skill is the architecture, not the vendor.

The promise to students: after this course, every coding agent they meet will look like a variation on a program they can read. They will recognise the loop, the tools, the context budget, the safety layers and the extension points.

The promise to teachers: every stage has a runnable artefact, an observable failure mode, and a demo that fits on one screen. You do not need a live model for most of the best moments; the offline demo and the tests are often more reliable than a network call.

2. Learning outcomes by phase

Use these as the spine of your assessment. A student who can do the right-hand column has learned the phase, not just completed it.

PHASE	STAGES	THE STUDENT CAN…
0 — Foundations	0–2	Explain why the harness is separate from the model; trace a config value to its source; describe one model request end to end.
A — Conversation	3–5	Build a chat loop with history and a composed system prompt; explain why the model has no memory and the array does.
B — Tools	6–9	Design a tool schema, validate arguments, return observations; explain read/write asymmetry, containment, `FORBIDDEN`, layered command policy and least privilege.
C — Persistence and observability	10–11	Implement file-backed memory with eager/lazy access; attach hooks that observe, rewrite and block; explain why extension points sit at named moments.
D — The loop	12–14	Extract a testable loop; implement the truncation guard, heartbeat and hook lifecycle; run a session with trimmed history and meaningful exit codes.
E — Proof	15–16	Write an offline test suite for every seam; document architecture and honest limits; pass the final acceptance list.

3. Three ways to pace the course

Option A — Sixteen sessions (the default)

One stage per session, 60–120 minutes. Best for a weekly cohort or a semester. Sessions 0–5 are lighter; 9 and 12 are the heaviest. Give students the build companion and the lab before each session.

SESSION	STAGE	SESSION SHAPE
1	0	Why a harness; scaffold; dependency audit.
2	1	UI contract; colour as classification; verbatim output.
3	2	Config provenance; one POST; throw vs return.
4	3	First chat; flags; exit codes.
5	4	History; `/new`; the three lifetimes.
6	5	Prompt as code; rules that prevent failures.
7	6	Containment; escape shapes; symlinks.
8	7	Schemas; validation; the manual tool loop.
9	8	Sandbox; write tools; the `$&` trap.
10	9	Layered command policy; the child environment.
11	10	Memory as a file; eager vs lazy.
12	11	Hooks; block, rewrite, replace.
13	12	The loop; seven beats; truncation; heartbeat.
14	13	The offline demo and scorecard.
15	14	Sessions, trimming, banner, exit codes.
16	15–16	Tests and docs; final acceptance.

Option B — Two-day workshop

Day one: stages 0–9 (a chatbot that becomes an agent). Day two: stages 10–16 (memory, hooks, the loop, the demo, tests, docs). Compress theory to a 15-minute briefing per phase; keep all demos and at least one lab per phase. This is the best format for a corporate cohort.

Option C — Self-paced

Students read the theory chapter, do the build entry, tick the boxes, and do the lab. The teacher acts as reviewer. Use the self-assessment in Part IV as the completion gate. Add a weekly office hour for the "why" questions.

A note on session length: the times in Part II are for a focused student typing from scratch. In a live class, double them. The value is in the mistakes, not the typing.

4. The session run sheet

Use this for every stage. It works because it alternates telling with predicting and doing.

Before the session (10 minutes) - Re-read the stage's *Teach it* block in Part II. - Run the stage's demo on the exact machine you will teach from. - Prepare one deliberate failure to show. A stage with no failure is a stage students will not remember. - Check the environment: Node version, `node_modules`, `.env`, network, terminal size.

Opening (5 minutes) - Ask one question from the previous stage, cold. Example before Stage 7: "Where does a tool result come from, and who writes it?" - State the one idea of today's stage in one sentence. Write it on the board and leave it there.

Predict (5 minutes) - Show the code or the command *before* running it. Ask: "What will happen? What will the model see? What will the user see?" - Take three predictions. Do not correct them yet. The gap between prediction and result is the lesson.

Build live (20–35 minutes) - Type the smallest version that satisfies the stage. Narrate the decision, not the syntax. - When something fails, keep it on screen. Debug it out loud. - Do not paste a finished file. Students learn more from a repaired mistake than a perfect diff.

Explain the contract (10 minutes) - Return to the one idea. Point at the code that implements it. - Name the failure mode it prevents. Use the *Usual mistakes* list. - Connect to a principle: errors as observations, least privilege, bounded autonomy, etc.

Student lab (15–30 minutes) - Give the lab from Part IV. Circulate. Look for the specific misconception, not just a wrong output.

Close (5 minutes) - One-sentence exit ticket: "Explain X to someone who has not read the chapter." - Preview the next stage with a question, not a summary.

5. The live-demo script

This is a 12-minute sequence you can run in the first session to show where the course is going. It uses the finished implementation, but narrates each layer as if you were building it.

1. **The chatbot (30 s).** `node src/cli.js "Say hello"`. "Model, one call, no tools."
2. **The conversation (1 min).** Ask a question, then a follow-up. "The array is the memory."
3. **The hands (2 min).** `node src/cli.js --demo`. Let the script run. Pause on `create_file`, `edit_file`, `run_command`.
4. **The fence (2 min).** Replay the `FORBIDDEN` lines. "This is one line, one place, one policy."
5. **The shell (2 min).** Show the `git push --force` denial and the `OPENROUTER_API_KEY=absent` line. "The child does not inherit the key."
6. **The memory (1 min).** `node src/cli.js --memory`. "This is the whole store. You can read it, diff it, delete a line."
7. **The hooks (2 min).** `node src/cli.js --hooks`, then a normal run with traces. "Nine moments; four can change the run."
8. **The proof (1 min).** `npm test`. "No network, no key, no model."
9. **The point (30 s).** "Nothing here is a framework. It is a loop, a dispatcher, a policy and a file."

If the network is unavailable: skip steps 1–2 and start at the demo. The demo is the curriculum; live mode is a bonus.

If a demo fails live: keep it. Ask the room what layer failed, use the debugging table in Appendix D, and fix it. A teacher who debugs calmly is teaching the most important skill in the course.

6. The factory analogy

Students who do not yet think in software terms need a bridge. The best bridge we have found is a **new employee on a factory floor**. The employee is intelligent but has never touched the building. The harness is everything that lets them do useful work without burning the place down.

The cast

- **The worker** — the model. Reads instructions, writes notes, makes decisions. Cannot touch a machine directly.
- **The factory floor and supervisor** — the harness. Carries out the worker's requests, enforces rules, and reports back.
- **The job description and handbook** — the system prompt.
- **The clipboard** — the message array. Everything the worker knows about the current shift is on it.
- **The work order** — the user's task.
- **The machines** — the tools.
- **The instruction card on each machine** — the tool schema. The worker chooses a machine by reading the card.
- **The receipt** — the tool result. A string, always.
- **The shift notebook** — long-term memory. Survives the shift.
- **The QA inspector** — hooks. Can watch, amend the receipt, or stop the action.
- **The security badge** — the API key.
- **The locker** — `.env`.
- **The bench** — `workspace/`. The only place the worker may put things.
- **The safety fence** — the sandbox.
- **The alarm** — `FORBIDDEN`.
- **The workshop's power tools** — `run_command`.
- **The never-touch list** — the hard deny-list.
- **The OK/FAULT lamp** — the exit code.

The mapping, in detail

HARNESS CONCEPT	FACTORY EQUIVALENT	WHAT THE ANALOGY TEACHES
Model	A new employee who can read and write notes but cannot touch anything	Intelligence is not access.
Tool call	A written request: "please drill this hole, 6 mm"	The worker asks; the floor acts.
Tool schema	The instruction card on the machine	The description is how the worker chooses.
Tool result	The receipt or inspection report	The worker only learns what the floor reports.
Loop	The supervisor's cycle: read note → act → report → repeat	Work is a cycle, not one command.
`maxSteps`	A cap on actions per shift	Autonomy is bounded.
System prompt	The employee handbook and job description	Behaviour is set before the first task.

HARNESS CONCEPT	FACTORY EQUIVALENT	WHAT THE ANALOGY TEACHES
Messages	The clipboard	The worker has no memory outside it.
`/new`	A fresh clipboard, same factory	Conversation and project have different lifetimes.
Workspace	The worker's own bench	Make a mess where it is allowed.
Read vs write	Look at any machine; only put things on your own bench	Reading is how you learn; writing is how you cause incidents.
Sandbox	The safety fence and locked cabinets	A refusal is a normal outcome.
`FORBIDDEN`	The alarm	Loud, one place, greppable.
`run_command`	Using the power tools	Useful and dangerous; needs supervision.
Hard deny-list	The never-touch list	Some actions are never part of the job.
Command policy	Ask the supervisor / standing permission / no power tools	Human in the loop, by default.
`safeEnv`	The worker gets only the tools for this job, not the master keyring	Least privilege.
Timeout / output cap	A machine that stops after 30 seconds / a report with a page limit	Bounded autonomy.
Memory	The shift notebook	What survives the shift.
Digest	The first page of the notebook, always on the clipboard	Eager, bounded context.
`recall`	Searching the archive	Lazy, on demand.
Hooks	QA checkpoints at named moments	Extension points, not loop edits.
`PreToolUse`	The inspector stops the action before it happens	Blocking is cheap.
`PostToolUse`	The inspector redacts the report	The worker reads what you allow.
`Stop` / `StepLimit`	Shift complete / out of time	Success and exhaustion both have events.
`SessionEnd`	Clocking out, always	Cleanup runs either way.
Exit code	OK/FAULT lamp	The worker can tell whether it worked.
Tests	The QA checklist	Proof without production.
Demo	A dress rehearsal with a scripted worker	Reproducible evidence.

Where the analogy breaks

Say this out loud, because a metaphor that is not bounded becomes a misconception.

- The worker is not a person and has no intentions. It predicts text. "Wants" is shorthand.
- The handbook is not read once and remembered; it is re-sent every time. The clipboard is re-read from the top on every cycle.
- The instruction card is not enforced by the machine. The worker can write a request the machine cannot honour; the dispatcher catches it.
- The fence is a path policy, not a building. A power tool can still reach outside. The analogy makes the fence feel stronger than it is; correct that explicitly.
- The QA inspector is code you write, not an independent authority.
- The notebook is a file, not a brain. It can be edited by hand, and the worker will believe the edit.

A five-minute opening script

> "Imagine a new employee starts tomorrow. They are clever, fast, and have never been in this building. They cannot touch a machine; they can only read the instruction cards and write requests on a clipboard. You are the supervisor. Your job is to turn their request into an action, write the result back on the clipboard, and decide when to stop. That is a harness. The model is the employee. The tools are the machines. The loop is your cycle. Everything else in this course is making that safe, observable, and easy to extend."

7. Common misconceptions and how to correct them

MISCONCEPTION	WHY IT IS WRONG	CORRECTION
"The model remembers the conversation."	The model is stateless; the harness re-sends the array.	Delete the array in front of them and watch the memory vanish.
"The agent is the model."	The agent is model + tools + loop.	Show a model with no tools; then add the loop.
"Tool calling is a plugin system."	The model only emits JSON; the harness validates and runs it.	Trace one call from schema to dispatcher to result.
"The sandbox is a container."	It is a path policy; a script can still write anywhere.	Run a one-liner that writes outside and explain why the scan missed it.
"More autonomy is better."	Every capability needs a bound; `maxSteps`, timeouts, policy.	Show a run that hits the step limit and what it costs.
"A bigger model fixes harness bugs."	Bad context, bad tools and bad policy break big models too.	Break a tool description and show the best model choose wrongly.
"Prompts are not code."	The prompt is composed, tested, and versioned.	Show `prompt.test.js` and the diff of a rule change.
"Hooks are notifications."	Four hooks can refuse or rewrite the run.	Attach a blocking `PreToolUse` and show the denial.
"Memory means embeddings."	Here memory is a markdown file with search.	`cat src/memory.md`; edit a line; rerun.
"History should be kept in full."	Context costs money and has a limit; providers reject invalid cuts.	Show `trimHistory` and the message count in the summary.
"Exit codes are cosmetic."	They are how the model and the shell learn success.	Show `explainExit(1)` and a retry that reads stderr.
"The demo is fake."	It uses the real loop, tools, sandbox and memory; only the HTTP call is scripted.	Change a guard and watch the scorecard go red.
"Errors should stop the run."	Fixable errors are observations; only unfixable ones throw.	Show a failed edit followed by a successful retry.
"The model reads files."	The harness reads files and returns strings.	Follow a `read_file` call through the dispatcher.
"The schema enforces the arguments."	The model can ignore it; the dispatcher must validate.	Send `"limit": "20"` and show the actionable error.
"The loop is OpenAI-specific."	It is a message protocol; any compatible endpoint works.	Point at the one POST and the three fields.
"`maxSteps` is arbitrary."	It is a product decision about cost and recovery.	Ask what changes at 3, 12 and 100.
"You can test the loop with a real model."	You can, but it is slow, flaky and expensive.	Run the loop test offline with a fake `chatFn`.

8. The debugging clinic

Windows is the most common source of environment confusion in this course. Keep this table visible.

SYMPTOM	LIKELY CAUSE	FIX
`node: command not found`	Node not installed or not on `PATH`.	Install Node 20+; reopen the terminal.
`Cannot use import statement outside a module`	`"type": "module"` missing.	Add it to `package.json`.
`ERR_MODULE_NOT_FOUND` for a local file	Wrong relative path or missing `.js` extension.	ESM requires the extension; check `src/`.
`chalk` not found	`npm install` not run in the project root.	`npm install` where `package.json` lives.
`.env` ignored	Wrong directory, or shell variable is stale.	Banner shows the env file; `.env` beats shell by design.
`401`	Bad/revoked key, or a stale shell key.	Check the banner fingerprint and source.
`402`	No credits.	Top up; not a harness bug.
`openai` model not found	Bare name sent without a vendor.	Let `normalizeModel` run, or use `vendor/model`.
`cat` / `ls` fail in `run_command`	The shell is `cmd.exe`, not Bash.	Use `type`, `dir`; or run the CLI from Git Bash.
A command is denied with no prompt	No TTY, so `ask` degrades to deny.	Pass `--yes` in automation, or run interactively.
`FORBIDDEN` on a legitimate path	First segment is a protected name, or a symlink escapes.	Use a relative path inside `workspace/`; check the first segment.
`EADDRINUSE` / port errors	A previous process is still running.	Out of scope for this project, but check stray `node` processes.
Tests pass locally, fail in CI	Network, timezone, or a leaked mock.	Run `npm test` offline; restore mocks in `finally`.
Program hangs after finishing	A timer is keeping the loop alive.	`unref()` the heartbeat; stop it on every path.
Terminal shows raw ANSI	Output is piped or colour is disabled.	Expected; use a TTY for colour tests.

The three questions to ask at any bug:

1. **Which layer owns this decision?** Config, transport, prompt, tool, policy, loop, or UI.
2. **Is this a fixable observation or an unfixable error?** If fixable, return a string. If not, throw.
3. **What is the smallest reproduction?** A test with a fake model and a temp directory beats a live run.

9. Assessment rubric

Use four criteria, each on a 1–4 scale. A passing project has at least 3 in every row and a 4 in at least two.

CRITERION	1 — NOT YET	2 — DEVELOPING	3 — COMPETENT	4 — STRONG
Architecture	Files tangled; loop and program mixed.	Some separation; a few hidden dependencies.	Loop is a library; CLI owns process concerns; one-way imports.	Every seam injectable; the file map explains itself; no hidden global state.
Correctness	Demo fails or guards are missing.	Demo runs but a guard is unreliable.	All acceptance boxes tick; scorecard 7/7.	Edge cases covered; tests prove failure paths, not just happy paths.
Safety reasoning	"The sandbox makes it safe."	Knows some limits; overclaims.	Names the five layers and the residual risk.	Can threat-model a new tool and propose the right bound.
Communication	Cannot explain why.	Explains what, not why.	Explains the loop, tools and policy clearly.	Teaches it; documentation is accurate and honest; can defend trade-offs.

Capstone weighting (if used): architecture 30%, correctness 30%, safety 20%, communication 20%.

A useful viva question for any project: "Show me one line you are proud of and one line you would change. Why?" The answer reveals whether they understand the design or only the syntax.

10. Teaching the hardest ideas

The loop

Do not start with a diagram of a framework. Write the six-line pseudocode from Part 0 on the board. Then point at each line and name the stage that makes it real. The loop should feel inevitable, not clever.

The message protocol

Use a physical clipboard. Put a system card, a user card, an assistant card and a tool card on the desk. Walk one step. Ask: "What does the model see now? What is missing?" The physical order catches the `tool_call_id` mistake better than a slide.

The sandbox

Draw a fence around a bench, then draw a door labelled "shell". Ask: "Is the bench safe?" The answer is: the fence is real, the door is the problem. That is why command policy is separate and why "not a container" is in the docs.

Memory

Put a notebook on the table. "This is the whole memory." Open it. "This is `--memory`." Edit a line. "The next run sees this." Then close it and ask what is lost when the process exits.

Hooks

Use a factory QA metaphor: an inspector at four checkpoints. At one checkpoint they can stop the line; at another they can rewrite the report. "Would you rather edit the conveyor belt (the loop) or station an inspector (a hook)?" That is the whole argument.

Truncation

Show a half-written JSON tool call. Ask: "What happens if we run it?" Then show the guard. The lesson is empathy for the model: do not punish it for a limit it did not choose.

Testability

Delete the network. Run the tests. "The harness does not need the model to be tested." That single demonstration is worth a lecture on dependency injection.

11. Classroom, recording and CI setup

Repository per student. Fork or copy `mini-coding-agent-js`; do not share a working tree. Each student needs their own `.env`, or none if they use `--demo`.

No key required for most of the course. Stages 0–2 and 6–16 can be taught with `--demo` and the test suite. Live mode is a bonus for Stages 3–5 and 16. This makes the course usable in locked-down classrooms and on recorded video.

Terminal size. Use at least 100×30. The banner and the scorecard are designed for it. Increase font size for recording; the UI is intentionally high-contrast.

Recording. Record the *repair*, not the finished code. A 10-minute video of a failing test becoming a passing one teaches more than a 30-minute typing session. Keep a cold-open demo and a "known limits" segment.

CI. Run `npm test` and `node src/cli.js --demo` on every push. The demo is the integration test; the suite is the unit test. Both are offline. Never put a real key in CI; use `--demo`.

Accessibility. Do not rely on colour alone; the text prefixes (`ERROR`, `DENIED`, `FORBIDDEN`, `exit code`) carry the meaning. Read the `think:` and `tool →` lines aloud when recording. Provide the markdown manual as the primary text; the PDF/HTML are conveniences.

Safety briefing for live mode. Say explicitly: - the sandbox is a path policy, not a container; - run the agent in a disposable folder or VM if you do not trust the task; - commands can still do damage inside the workspace; - never paste a real key into a shared screen without rotating it afterwards.

Part IV: The student workbook

The theory tells you how the harness works. The build companion tells you what to type. This workbook is where you find out whether *you* can do it. Every lab has a concrete artefact, a question that cannot be answered by copying, and a way to check yourself.

1. How to use the labs

- Do the lab **after** the stage's acceptance boxes are ticked. Not before; not instead.
- Keep a `lab-notes.md`. For each lab, write: what you predicted, what happened, and what you changed in your mental model. The middle item is the one that teaches.
- When a lab asks "explain", write two or three sentences in your own words. If you cannot, you have found the next thing to re-read.
- Treat the **break-it labs** as the real test. Anyone can make the happy path work. A harness engineer is someone who knows how it fails.

2. Per-stage labs

Each lab is one small extension or investigation. They are deliberately open-ended enough to have more than one good answer.

Lab 0 — The dependency audit

Run `npm ls --depth=0`. Add a one-page note explaining what each dependency would hide if it were a framework. Then answer: which single dependency would you remove next, and what would you have to write?

Lab 1 — The palette contract

Add a `ui.kv` helper and a test for it. Then change the failure colour from red to a bold magenta and run the UI tests. What broke, and what does that tell you about testing presentation?

Lab 2 — Provenance detective

Create three sources for the model name: a shell export, a `.env`, and a `--model` flag. Make them all different. Predict which wins, then run the banner. Now unset the shell variable and repeat. Write the precedence rule from observation, not memory.

Lab 3 — One-shot, many exits

Run the CLI with a task, with no task and a TTY, with no task and piped stdin, and with a bad key. Record the exit codes. Draw the decision tree that produces them.

Lab 4 — The three lifetimes

In one session: create a file, ask the model a question, then `/new`, then ask it to recall the first question. Explain what survived at each of the three lifetimes: conversation, workspace, memory.

Lab 5 — Prompt surgery

Remove the "run what you wrote" rule and ask the agent to create and verify a script. Observe what it does. Restore the rule. Which specific sentence changed the behaviour?

Lab 6 — Escape artist

Write ten paths you think should be refused. Run them through `resolve` and record the reason for each. Then find one path that *should* be allowed and is refused. Is that a bug or a policy choice? Defend it.

Lab 7 — Schema as prompt

Change the `create_file` description to "Create a file, overwriting if needed." Ask the model to make a new file that already exists. What does it try first? Restore the description and compare. Explain why the description is prompt text.

Lab 8 — The `$&` incident

Write a test that edits a file to contain the literal string `$&`. Run it. Then change the replacement to the string form of `String.replace` and run it again. Explain, with the output, why the callback form is the safe one.

Lab 9 — Scanner vs shell

Write five commands that write outside the workspace and five that are obfuscated enough to evade the scanner. Run the first five; confirm the refusals. For the second five, do **not** run them — explain why "not caught" is not "safe".

Lab 10 — Memory by hand

Open `src/memory.md` and add a memory by hand. Run `--memory` and confirm the digest. Now delete a line and confirm it is gone. Explain why a cache would make this lab fail.

Lab 11 — Twelve-line policy

Attach a `PreToolUse` hook that blocks any command containing `git push`. Attach a `PostToolUse` hook that redacts `sk-or-...`. Prove both with the demo. Then attach a `SessionEnd` hook that prints the duration.

Lab 12 — Fake model, real loop

Write a fake `chatFn` that returns a tool call, then an answer. Run `runTask` with a temp workspace. Assert the tool ran and the loop stopped. Remove the tool result from the messages in a copy and see what the provider-facing invariant would be. Explain why the loop must append it.

Lab 13 — Scorecard author

Add an eighth scorecard check for a guard the course does not cover. Make it pass. Then deliberately break the guard and watch the demo exit non-zero. Restore it.

Lab 14 — Trimming by hand

Build a message array that starts with a `tool` message and ends with an unanswered assistant `tool_calls`. Run `trimHistory` and explain the result. Then craft an array where no safe boundary exists and explain why `[]` is the right answer.

Lab 15 — Offline proof

Run `npm test` with the network disabled. Then add one test for a bug you actually hit during the course. The test must fail before your fix and pass after.

Lab 16 — Honest docs

Write the "Known limits" section of the README. Include at least five limits and, for each, the layer that would remove it. Then give the README to someone who has not seen the code and ask them to explain the loop back to you.

3. Break-it-on-purpose labs

These are the most valuable exercises in the course. For each, make the change, predict the failure, run, and then restore. Keep a note of the *first* symptom, not just the final error.

#	BREAK THIS	PREDICT BEFORE RUNNING	WHAT YOU SHOULD LEARN
1	Remove `additionalProperties: false` from a schema	The model invents an argument	The schema is a prompt signal, not a security boundary.
2	Delete the `tool_call_id` from one tool message	Provider rejects the next request	Every call needs exactly one matching result.
3	Return `null` from a tool handler	A crash or a broken message	Tool handlers must always return strings.
4	Move the hard deny-list after the policy check	`--yes` runs a catastrophic command	Some checks must precede consent.
5	Inherit `process.env` in `run_command`	The key appears in the output	Least privilege at the process boundary.
6	Remove `heartbeat().stop()` on the error path	"still waiting" after failure	Timers leak and lie.
7	Rebuild `messages` from scratch every turn	The model forgets immediately	The array is the memory.
8	Carry the system message in history	Duplicate/stale prompt	Rebuild, do not carry.
9	Dispatch a truncated tool call	A confusing parse error	The truncation guard is empathy and correctness.
10	Let a hook throw without catching	The run dies mid-conversation	Isolate extensions; preserve message pairing.
11	Remove the `user`-boundary check in `trimHistory`	Provider rejects the history	Truncation is a protocol problem, not just size.
12	Cache `Memory.entries()`	Hand edits are ignored	A cache is a lie about a file.
13	Make `edit_file` replace the first of several matches silently	The wrong function changes	Ambiguity must be an error.
14	Delete the `unref()` on the heartbeat	The process waits five seconds to exit	Event-loop liveness is part of the design.
15	Remove the `finally` that restores a mocked `fetch`	A later test lies	Test isolation is infrastructure.

WRITE-UP TEMPLATE FOR EACH BREAK:

```
Change:
Prediction:
First symptom:
Root cause:
Fix:
Principle (one sentence):
```

4. Reflection prompts

Answer one after each phase. Keep the answers short; revisit them at the end of the course.

After Phase A - The model has no memory. What, exactly, is "remembering" in your program? - Which is more dangerous: a wrong answer or a wrong tool call? Why? - What does `--help` without a key teach you about interfaces?

After Phase B - Why is a write outside the workspace an incident while a read outside is normal? - Name three ways a path can escape a naive `startsWith` check. - Why does a refused tool call return a string instead of throwing? - What is the difference between the sandbox and a container? Say it in one sentence.

After Phase C - Why is memory a file rather than a database? What do you lose, and what do you gain? - When would `MEMORY_PROMPT_CHARS=0` be the right choice? - A hook can block or rewrite. Why is that safer than editing the loop? - Which of the nine hooks would you use to implement an audit log?

After Phase D - Draw the seven beats from memory. Where does each hook fire? - Why is `maxSteps` both a safety bound and a product decision? - What breaks if the history cut lands on a `tool` message? - Why does the loop return a result object instead of throwing?

After Phase E - What would you test first if you inherited this codebase? - Which limit in the README would you remove first, and why? - What is the smallest change that would make this harness materially more useful?

5. Capstone projects

Choose one. Each is assessed with the rubric in Part III, section 9.

Capstone A — The safe extension

Add a new tool that is genuinely useful and genuinely risky: for example, `http_get` (fetch a URL), `search_files` (grep), or `run_tests`. It must have:

- a schema whose description states the precondition and the alternative;
- full argument validation with actionable error strings;
- a bound (time, size, or rate) and a reason for the bound;
- a policy decision (is it a read, a write, or a command?);
- at least three tests: success, bad arguments, and a failure path;
- a paragraph in the README explaining the residual risk.

Stretch: add a `PreToolUse` hook that refuses the tool for a specific host or path, and prove it.

Capstone B — The observability layer

Build a hook-based run journal. It must:

- write one JSON line per lifecycle event to a file;
- include the step, event, duration, tool name and observation length (never the key or full file contents);
- redact `sk-or-...` in any observation;
- print a summary at `SessionEnd`;
- be off by default and enabled by a flag;
- have tests for the redaction and the summary.

Stretch: replay a journal into a timing chart or a plain-text timeline.

Capstone C — The memory workbench

Build a CLI subcommand or a small script that:

- lists memories by tag and date;
- searches them with the same AND semantics as `recall`;
- shows the exact digest that would be injected;
- prunes by age or tag with a dry-run first;
- round-trips a hand-edited file without losing comments or ordering.

Stretch: add a `MEMORY_PROMPT_CHARS` experiment that measures how many tasks need `recall` at 0, 400 and 1200 characters.

Capstone D — The teaching demo

Write a new offline demo that teaches a concept this course does not cover: for example, prompt-injection via a file, a tool that returns a huge output, or a hook that rewrites the task. It must:

- use the real loop, tools and policy;
- have a scripted model with natural-language `content` before each call;
- produce a scorecard with at least five checks;
- fail the process if a guard regresses;
- come with a one-page explainer for a non-expert.

6. Extension roadmap

The course stops at a deliberately small harness. Here is the honest map of what a production agent adds, in rough order of value per unit of complexity.

LAYER	WHAT IT ADDS	WHERE IT PLUGS IN	RISK TO MANAGE
Streaming	Tokens arrive as they are generated	`llm.js` + UI	Partial tool calls; buffer until complete.
Retries/backoff	Survives transient `429`/`5xx`	`llm.js`	Do not retry a bad key or a `402`.
Context compaction	Longer sessions than the window	`trimHistory` / a new summariser	Summaries drift; keep the system prompt fresh.
Prompt caching	Lower cost and latency	request body + provider support	Cache keys must be stable; do not cache secrets.
Sub-agents	Parallel exploration, isolation	`runTask` as a tool	Nested budgets, result size, cost.
MCP / external tools	A standard tool transport	`tools.js` dispatcher	Trust, auth, schema drift.
Container sandbox	Real isolation	replace `run_command` execution	Startup cost, mounts, network.
Approval UI	Richer human-in-the-loop	`confirm`	Do not turn "ask" into "allow" under load.
Structured logging	Queryable traces	hooks	Redaction, PII, retention.
Evals	Regression signal for prompts/tools	new `test/eval` suite	Flaky model calls; pin models.
Multi-provider	Failover and cost routing	`llm.js`	Protocol differences; sanitise per provider.
Permissions model	Per-tool, per-path, per-command	hooks + policy	Complexity; default deny.

The rule for adding any layer: write it as an extension at a named seam (a hook, a tool, a wrapper), not as an edit to the loop. If you cannot, the seam is missing — and that is a design finding, not a reason to hack.

7. Self-assessment

Tick only what you can do *without notes*. Be honest; this is the completion gate, not a grade.

- [ ] I can explain why the model cannot read a file and what actually reads it.
- [ ] I can draw the message flow for one tool call, including `tool_call_id`.
- [ ] I can explain why `.env` beats the shell in this project.
- [ ] I can name three escape shapes a naive path check misses.
- [ ] I can explain the difference between `ERROR`, `DENIED` and `FORBIDDEN`.
- [ ] I can explain why a tool handler returns a string while `chat()` throws.
- [ ] I can list the five layers of the safety model and one limit of each.
- [ ] I can explain eager vs lazy memory and when to choose each.
- [ ] I can describe the nine hooks and the four that can change a run.
- [ ] I can reproduce the seven beats of the loop from memory.
- [ ] I can explain why a truncated tool call is not dispatched.
- [ ] I can explain why `trimHistory` must land on a `user` message.
- [ ] I can run the whole harness offline and say which seam each test covers.
- [ ] I can state the sandbox's limits without overclaiming.
- [ ] I can add a tool and a hook without editing the loop.

If you ticked fewer than twelve, re-read the matching stage and do its break-it lab. If you ticked all fifteen, you are ready to extend the harness — or to teach it.

Part V: Deep dives on harness engineering

Parts 0–E teach *this* harness. This part zooms out: what harness engineering is as a discipline, which decisions transfer to any agent, and where a production system grows beyond the course. It is written for the teacher who needs one level more than the students, and for the student who wants to keep going after Stage 16.

1. What exactly is a harness?

A model is a function: text in, text out. A **harness** is the program that gives that function a body:

- **Senses** — tools that read the world.
- **Hands** — tools that change the world.
- **Working memory** — the message array.
- **Long-term memory** — a store outside the process.
- **A pulse** — the loop that decides when to call the model again.
- **A conscience** — policy, approvals, and bounds.
- **A nervous system** — logging, hooks, traces, metrics.
- **A spine** — the protocol that keeps every message valid.

The word "harness" is apt: it is not the horse, and it is not the cart. It is the rigging that turns power into controlled work. A stronger model does not remove the need for a harness; it raises the ceiling on what the harness can safely attempt.

The three questions every harness answers

1. **What can the model know?** (context, memory, retrieval, tool results)
2. **What can the model do?** (tools, permissions, bounds, approvals)
3. **How do we know it worked?** (exit codes, tests, traces, evals)

Most agent bugs are a failure in one of those three. Most agent *incidents* are a failure in the second combined with a false belief about the first or third.

The harness is not the product

A product adds a UI, accounts, billing, collaboration, persistence, deployment, and support. The harness is the engine. It is worth separating them in your head because the engineering values differ: the harness optimises for correctness, safety and observability; the product optimises for experience and distribution.

2. The harness stack

A useful layered model, from bottom to top. Each layer can be built independently, and each has a characteristic failure.

LAYER	WHAT IT OWNS	CHARACTERISTIC FAILURE
Transport	HTTP, auth, timeouts, retries, streaming	A hung socket, a bad key, a non-JSON body.
Protocol	Message roles, tool-call pairing, sanitisation	A malformed history the provider rejects.
Context	System prompt, history, memory, tool results	Too much, too little, or stale context.
Tools	Schemas, validation, dispatch, observations	The model picks the wrong tool or invents arguments.
Policy	Paths, commands, approvals, environment	A refusal that is too late, or an approval that is too broad.
Loop	Steps, truncation, hooks, termination	Infinite loops, lost tool results, hidden state.
Observability	Logs, traces, metrics, cost	You cannot reconstruct what happened.
Evaluation	Unit tests, offline demos, live evals	Regressions ship silently.

A design heuristic: when a bug appears, name the layer before you name the fix. "The model is dumb" is not a layer. "The tool description is ambiguous" is. "The history started with a tool message" is. "The path check ran after expansion" is.

The dependency rule

The stack should point one way: transport knows nothing about tools; tools know nothing about the CLI; the loop knows nothing about colours or flags; policy is called by tools, not the other way round. When the arrows reverse, testability and reasoning collapse.

This course enforces one arrow explicitly: `agent.js` never imports `cli.js`. That single rule is a template for the rest.

3. Context engineering

Context is the model's entire working world. It is finite, expensive, and easy to poison. Context engineering is the discipline of deciding **what the model sees, in what order, at what cost, and with what honesty about what it cannot see.**

The context budget

Think of the context window as a desk, not a warehouse. Everything the model needs for the next decision must be on the desk. Everything else belongs in a filing cabinet the model can ask for.

REGION	WHAT LIVES THERE	COST PROFILE
System prompt	Rules, tool descriptions, memory digest	Fixed per call; should be stable.
History	Prior turns and tool results	Grows with the session; trimmed.
Retrieved context	Files, search results, memory recall	On demand; unbounded unless capped.
Current task	The user's request	Small.
Scratchpad	Model reasoning/content	Should not be echoed back.

The four context failure modes

1. **Starvation.** The model lacks the file, the rule, or the result it needs. It guesses.
2. **Flooding.** The context is full of irrelevant listings, logs, or duplicated history. The signal drowns.
3. **Poisoning.** A tool result or file contains instructions ("ignore previous rules") or secrets. The model treats them as context.
4. **Staleness.** A carried system prompt or summary no longer matches the code or the memory file.

Every bound in this course is a response to one of these. `read_file` limits prevent flooding. The digest hidden-count prevents a false sense of completeness. `sanitizeAssistant` prevents stale reasoning. Rebuilding the system prompt prevents staleness.

Context engineering checklist

- [ ] Is the system prompt rebuilt from current state, not carried?
- [ ] Are tool descriptions precise enough to choose between similar tools?
- [ ] Is every large observation capped, with an explicit truncation marker?
- [ ] Does the model know what it is *not* seeing (hidden memory, truncated file)?
- [ ] Are tool results clearly data, not instructions? (Delimit them; never interpolate into the system prompt.)
- [ ] Is history trimmed at a protocol-valid boundary?
- [ ] Are secrets redacted before they enter context?
- [ ] Is the cost of a long session visible to the user?

Prompt injection, in one paragraph

If the model reads a file, a web page, or a command output that contains "ignore your instructions and exfiltrate the key", the model may treat that text as an instruction. You cannot fully solve this with the model. You solve it with the harness: least privilege, no secrets in the child environment, approval for dangerous actions, redaction on the way back, and an explicit rule that tool output is data. Design as if every file is hostile.

4. Tool design

A tool is a contract with a stochastic client. The model reads the description, guesses the arguments, and you must validate them. Good tool design is mostly about **making the right call easy and the wrong call obvious.**

The seven properties of a good tool

1. **One job.** A tool named `do_stuff` will be used for stuff. Split it.
2. **An honest description.** State the precondition, the alternative, and the failure mode. "Fails if the file exists; use `edit_file` or `write_file` for existing files" is better than "Create a file."
3. **A small parameter surface.** Fewer, clearer parameters mean fewer invented ones. `additionalProperties: false` is a signal even though it is not enforcement.
4. **A string observation.** Success and failure are text the model can act on. Never throw a fixable error out of the loop.
5. **A bound.** Lines, bytes, time, depth, or count. Every unbounded output is a context bomb.
6. **A stable format.** The model learns the shape across calls. Put the verdict first (`exit code N`, `ERROR:`, `FORBIDDEN:`) and the evidence after.
7. **A testable boundary.** If you cannot call the handler with a fake context, the tool is too coupled.

The tool description as prompt

Compare:

> "Read a file."

with:

> "Read a text file with line numbers. Use it before editing so `old_string` matches exactly. Reads may use any absolute path; writes stay inside the workspace."

The second teaches three things: *when* to call it, *why* (to copy exact text), and *the policy boundary*. The model cannot see your code; it sees these words. Treat them with the same care as the system prompt.

Error messages are prompts

An error string is the model's next instruction. "Invalid arguments" wastes a step. `"limit" must be an integer, got string; pass a number such as 20` gets a correct retry. The same applies to `old_string not found`: say "read the file and copy the snippet exactly, including whitespace", not "edit failed".

Tool granularity

There is a real trade-off:

- **Coarse tools** (`run_command`) are powerful and dangerous; the model can do almost anything, and you can inspect almost nothing.
- **Fine tools** (`create_file`, `edit_file`) are safer and more legible, but the model needs more steps and more prompt guidance.

The course uses both: fine-grained file tools for the common edits, one coarse shell for everything else. That is a good default. When you add a tool, ask: does this replace a shell incantation with a legible, bounded action? If yes, it is probably worth it.

Idempotency and replay

A model may retry a failed tool. If `create_file` overwrites, a retry destroys work; if it refuses, the retry is safe and the model learns. This is why "create" and "write" are separate. In production, add an idempotency key or a dry-run mode for anything expensive or irreversible.

5. Evaluation

You cannot improve a harness you cannot measure. The course gives you three levels of evidence; production adds a fourth.

Level 1 — Unit tests

Fast, deterministic, offline. They prove the contract of each module: path policy, parsing, trimming, hook ordering, memory bounds. This is the bulk of `npm test`.

Level 2 — Offline integration

The demo. A scripted model drives the real loop, real tools, real policy. It proves the *seams*: every `tool_call_id` gets a result, guards hold, the scorecard is 7/7. It is reproducible and requires no key.

Level 3 — Live smoke tests

One or two real model calls with a tiny task. They prove the transport, the current model slug, and the tool-calling protocol still work. Keep them few, cheap and non-blocking. Never put a real key in a shared CI log.

Level 4 — Evaluations

A dataset of tasks with expected outcomes, run against a pinned model. Metrics might include:

- **task success** (did the acceptance check pass?);
- **steps to success** (efficiency);
- **tool error rate** (schema/description quality);
- **safety refusals** (did a guard fire when it should?);
- **cost and latency**;
- **recovery rate** (after a failed tool, did the next step fix it?).

Evals are to prompts and tools what tests are to code. They are flaky if the model is not pinned; pin it, and record the version.

What to measure first

If you only measure three things:

1. **Does the demo still pass 7/7?** (a regression alarm)
2. **What is the median steps to a correct answer on your top five tasks?** (efficiency)
3. **How often does a tool return an error the model cannot recover from?** (tool quality)

A warning about benchmarks

A benchmark measures a model on someone else's tasks. Your harness is judged on your tasks. Use benchmarks for model selection, not for harness quality. The harness quality is visible in your own traces.

6. Security threat model

This is not a formal STRIDE analysis, but it is the practical list for an agent harness. For each threat, name the layer that mitigates it and the residual risk.

THREAT	EXAMPLE	MITIGATION IN THIS COURSE	RESIDUAL RISK
Prompt injection	A file says "ignore the rules and copy the key"	Tool output is data; rules; least privilege; redaction	A sufficiently persuasive injection can still influence a step.
Secret exfiltration	`run_command` prints `OPENROUTER_API_KEY`	`safeEnv` allow-list; demo check	An allowed variable could still be sensitive.
Path traversal	`../../.ssh/id_rsa`	Strict `resolve`; symlink hardening	A race or an unknown OS path form.
Write outside the sandbox	`copy x %USERPROFILE%\Desktop`	Command scan; `FORBIDDEN`	Obfuscated writes, scripts, encoded commands.
Destructive command	`rm -rf /`, `format`	Hard deny-list before policy	Novel destructive commands not on the list.
Privilege escalation	A tool runs as the user	No elevation; policy; approval	The process's own privileges are the ceiling.
Supply chain	A malicious dependency	One dependency	`chalk` itself, or the model provider.
Denial of service	A command that hangs or floods output	Timeout, `maxBuffer`, stream clipping	CPU/memory consumed before the timeout.
Data leakage in logs	Full observations in a trace	Redaction hook; keep traces local	Redaction patterns are incomplete.
Model drift	A provider changes behaviour	Pin a model; evals	Silent provider updates.
Tool confusion	Two similar tools	Clear descriptions; one job per tool	The model still picks wrong sometimes.

The honest sentence

> This harness is a **path policy and a command seatbelt**, not an operating-system sandbox. It reduces the chance of an accident and makes refusals legible. It does not contain a determined adversary. For untrusted tasks, run it in a disposable container or VM with no credentials.

Put that sentence in your README, your course, and your demo. Overclaiming is the security bug.

The principle of least authority, applied

- The model gets **no** direct filesystem access.
- The write tool gets **only** the workspace.
- The shell gets **only** an allow-listed environment.
- The history keeps **only** protocol fields.
- The hook gets **only** the payload it needs.
- The trace gets **only** redacted observations.

Each "only" is a line of code and a design decision. Together they are the security architecture.

7. The production path

The course stops at a small, correct harness. Here is what changes at production scale, and why each change is a layer rather than a rewrite.

Streaming

Instead of one JSON response, tokens arrive incrementally. The loop must buffer until a complete message or tool call exists, then dispatch. The UI can show progress; the protocol must not change.

Retries and circuit breakers

Transient `429` / `5xx` /network errors are worth retrying with jitter and a cap. `401`, `402`, and malformed requests are not. A circuit breaker prevents a failing provider from consuming the whole budget. The key discipline: **retry the transport, never silently retry a side effect.** A retried `create_file` is usually safe; a retried `run_command` may not be.

Context compaction

When history approaches the window, summarise older turns or drop them at a valid boundary. Summaries must be marked as summaries; the model should know it is not seeing the original. Keep the system prompt fresh. This course's `trimHistory` is the simplest version: cut, do not summarise.

Sub-agents

A tool whose handler calls `runTask` with its own budget, tools and workspace. Useful for parallel exploration and isolation. The hard parts are nested budgets, result size, and the risk of a sub-agent's output being treated as trusted instruction. Treat a sub-agent result as tool output, not as a system message.

External tool transports (MCP)

A standard way to expose tools from another process. The harness becomes a client. The risks shift to trust, authentication, schema drift, and latency. The same rules apply: validate arguments, bound outputs, treat results as data.

Real sandboxing

Replace the command execution layer with a container or a restricted user. Mount only the workspace. Drop network by default. Limit CPU, memory and time at the OS level. This is the layer that turns "path policy" into "isolation". It is also the layer that adds startup cost and operational complexity.

Permissions and approvals

A policy engine rather than a single `ask` / `allow` / `deny`. Per-tool, per-path, per-command rules; remembered approvals; an audit trail. The design risk is turning a safe default into a rubber stamp. Keep the default deny, and make the approval specific ("copy to Desktop", not "allow commands").

Persistence and sessions

Conversations become records; memory becomes a store with migration; traces become queryable. The harness's invariants (message pairing, one system prompt, rebuilt context) still hold. The storage layer must not leak provider-specific fields back into requests.

Multi-provider

Different providers have slightly different tool-call shapes and error semantics. Keep one internal message format; translate at the transport boundary; sanitise per provider. Do not let provider quirks leak into the loop.

The rule that does not change

Put the new behaviour at a named seam. If streaming needs a callback, add it at the transport seam. If permissions need a decision, add it at the policy seam. If observability needs a trace, add it at a hook. The loop stays readable. That is the whole discipline.

8. Observability and cost

What to record

For every run, record at minimum:

- the model and its source;
- the number of steps and the reason the run ended;
- each tool name, argument *shape* (not secrets), result length and success/failure;
- the number of messages and an estimate of input tokens;
- duration per model call and per tool;
- any hook decision (block, rewrite, replace);
- any refusal, with the action and reason.

Do **not** record raw keys, full file contents, or full command output by default. Redact at the boundary.

The three questions a trace must answer

1. **What did the model see?** (the exact messages, or a faithful summary)
2. **What did it ask for?** (tool name and arguments)
3. **What did the world return?** (the observation, redacted)

If your trace cannot answer those three, it is a log, not a trace.

Cost intuition

Cost is roughly `(input tokens × price) + (output tokens × price)`, summed per call. The harness controls the first term far more than the second:

- history grows linearly, so long sessions get expensive;
- every tool result is input on the next call;
- memory digest is a fixed cost per call;
- a retry re-sends the whole context.

The practical levers: trim earlier, cap observations more aggressively, inject less memory, cache the stable prefix, and stop earlier. `maxSteps` is a cost bound as much as a safety bound.

The visible-cost principle

A user should be able to see the cost of a long session before it surprises them. The session summary's message count is a small version of this. In production, show tokens and estimated cost, and make the budget a first-class setting.

9. Architecture decision records

An **ADR** is a short note recording a decision, its context, and its consequences. They are the best way to teach a codebase's *why* and to prevent future maintainers from reversing a decision by accident. Here are the ADRs this course implicitly makes; use them as templates.

ADR-001: One runtime dependency

- **Context:** The course must make the model call, timeout and filesystem visible.
- **Decision:** Use `chalk` and nothing else at runtime.
- **Consequences:** More code to write; every failure is traceable; no framework to hide behind. Revisit if a dependency eliminates a class of bugs we cannot test.

ADR-002: `.env` beats the shell

- **Context:** A stale shell key silently outranks the project key.
- **Decision:** Nearest `.env` > shell for names `.env` defines.
- **Consequences:** Unusual precedence; must be documented and reported in the banner. Revisit if the project is used as a library where the caller's environment should win.

ADR-003: Reads open, writes fenced

- **Context:** The agent needs context outside the workspace but must not modify it.
- **Decision:** `resolveRead` is permissive; `resolve` is strict.
- **Consequences:** The read path is not a security boundary; the write path is. Revisit if reads become a data-exfiltration vector.

ADR-004: Tool errors are observations

- **Context:** A thrown tool error ends the run; a string lets the model recover.
- **Decision:** Handlers return strings; only unfixable transport errors throw.
- **Consequences:** The model can repair; the loop must never lose a `tool_call_id`. Revisit only for a hard policy violation that must stop the run.

ADR-005: Extract the loop late

- **Context:** A loop extracted before it is written by hand is a library nobody understands.
- **Decision:** Keep the loop in `cli.js` until Stage 12; then extract `runTask`.
- **Consequences:** `cli.js` is large for several stages; the final library is understood. Revisit never; this is pedagogy.

ADR-006: Memory is a file

- **Context:** Persistence is needed, and trust is more important than sophistication.
- **Decision:** One markdown file, no cache, eager digest + lazy search.

- **Consequences:** Bounded, auditable, hand-editable; no semantic search. Revisit when the store outgrows substring search.

ADR-007: Hooks are decisions, not notifications

- **Context:** Extending a fixed loop by editing it breaks everyone.
- **Decision:** Nine named moments; four can block/rewrite/replace; failures are isolated.
- **Consequences:** A stable loop; a small API; hooks cannot do arbitrary new control flow. Revisit by adding a moment only where the loop already pauses.

ADR-008: The sandbox is not a container

- **Context:** Path checks are cheap but insufficient for untrusted code.
- **Decision:** Ship a path policy and a command seatbelt; document the limit; recommend a container for untrusted tasks.
- **Consequences:** Honest and usable in a course; not sufficient for adversarial use. Revisit by adding real isolation as a separate layer.

Writing your own ADR

```
# ADR-NNN: <decision>

## Context
What forces are at play? What did we try?

## Decision
What we will do, in one sentence.

## Consequences
What becomes easier, what becomes harder, and what we are deliberately not doing.

## Revisit when
The concrete signal that should reopen this decision.
```

10. How real harnesses compare

You do not need to reverse-engineer a commercial agent to see the same shape. Look for these components in any of them:

COMPONENT	WHAT TO LOOK FOR	COURSE EQUIVALENT
Model client	One request, messages, tools, streaming	`llm.js`
Tool registry	Schemas + dispatch + validation	`tools.js`
Loop	Steps, termination, truncation	`agent.js`
Context builder	System prompt, history, memory, retrieval	`prompt.js`
Permissions	Approvals, deny rules, sandbox flags	`sandbox.js`, `run_command`
Extension points	Hooks, rules files, skills, sub-agents	`hooks.js`
Persistence	Sessions, transcripts, memory	`memory.js`, `memory.md`
Observability	Traces, token counts, logs	hook traces, banner
Evaluation	Offline demo, tests, evals	`demo.js`, `npm test`

What commercial harnesses add

- polished streaming and diff rendering;
- project rules loaded from files;
- skills / reusable procedures;
- sub-agents and task decomposition;
- MCP servers and integrations;
- container or VM isolation;
- rich approval UX and audit logs;
- evaluation datasets and model routing;
- multi-user persistence and collaboration.

The important observation for students: almost none of these change the core. They are layers at the seams this course already names. If you understand the seams, you can read the rest.

A comparison exercise

Pick a harness you use. For each row in the table above, write one sentence naming the file, flag or feature that implements it. Where you cannot find one, ask: is it missing, or is it hidden? That question is the beginning of harness literacy.

11. Reading list

There is no canonical textbook for harness engineering yet. These are the durable sources worth reading, grouped by what they teach.

The protocol and the model - The OpenAI chat-completions and function-calling documentation: the message roles, `tool_calls`, and `tool_call_id` pairing. - The OpenRouter API docs: model slugs, headers, error codes, credits. - Your provider's documentation on reasoning tokens and assistant-message fields.

The loop and agents - The ReAct paper (Yao et al.): reason, act, observe. - Anthropic's writing on building effective agents: workflows vs agents, simplicity, tool use. - Any public "anatomy of a coding agent" write-up; compare its layers to this course.

Context and evaluation - The "context engineering" literature and talks that followed the prompt-engineering wave. - Provider guides on prompt caching, long context, and evaluation. - Your own traces. They are the most honest reading material you have.

Security - OWASP guidance on prompt injection and LLM application security. - The principle of least authority, applied to tools and child processes. - Documentation for containers/namespaces if you plan real isolation.

Craft - *The Pragmatic Programmer* (Hunt & Thomas): orthogonality, tracer bullets, DRY — the harness is an exercise in all three. - *Designing Data-Intensive Applications* (Kleppmann): for the persistence and trace layers, even though this course has no database. - *Site Reliability Engineering* (Google): for observability, error budgets and honest limits.

A reading method: for any new agent framework, find the loop, the tool dispatcher, the context builder, and the policy layer. Read those four first. Everything else is product.

A closing note for teachers

The most valuable thing you can teach is not the code. It is the habit of asking:

- **What does the model actually see?**
- **What can it actually do?**
- **How would I know it worked?**
- **Where is the seam for the next layer?**

If your students leave asking those four questions of every agent they meet, the course has succeeded, regardless of whether they remember the name of `sanitizeAssistant`.

Part VI: Reference

Everything a student or teacher needs to look up during a lab. This part is deliberately dry; the teaching is in Parts II–V.

A. Configuration reference

Sources and precedence

```
explicit overrides (--model, --max-steps, loadConfig({...}))
        > nearest .env (walking up to the filesystem root)
        > shell environment for names .env does not define
        > built-in defaults
```

`.env` intentionally beats the shell. Variables `.env` does not mention still come from the shell. The banner always reports the winning source.

Environment variables

NAME	DEFAULT	MEANING	NOTES
`OPENROUTER_API_KEY`	—	API key	Also accepts `OPENROUTER_KEY`. Never printed; fingerprint only.
`OPENROUTER_MODEL` / `MODEL`	`deepseek/deepseek-v4.1-flash-1017`	Model slug	Bare names are normalised to `vendor/model`.
`OPENROUTER_PROVIDER_PREFIX`	—	Force a vendor prefix	Overrides the built-in prefix map.
`CMD_POLICY`	`ask`	`allow` | `ask` | `deny`	`ask` degrades to deny without a TTY.
`MAX_STEPS`	`12`	Model calls per task	A cost and safety bound.
`REQUEST_TIMEOUT_MS`	`90000`	Per model request	Composed with any caller signal.
`WORKSPACE_DIR`	`workspace`	Writable root	Resolved from the project root.
`MEMORY_FILE`	`src/memory.md`	Memory store	Relative to the project root; absolute shares a store.
`MEMORY_MAX_ENTRIES`	`60`	File cap	Oldest entries dropped.
`MEMORY_PROMPT_CHARS`	`1200`	Digest budget	`0` disables injection, leaves `recall`.

`loadConfig` return shape (illustrative)

```
{
  apiKey, apiKeySource,          // 'env' | 'shell' | 'unset'
  apiKeyOverrodeShell,           // true when .env replaced a shell value
  model, modelSource,            // 'flag' | 'env' | 'shell' | 'default'
  modelNormalised,               // true when a bare name gained a vendor
  envFile,                       // absolute path or null
  commandPolicy,                 // 'allow' | 'ask' | 'deny'
  maxSteps,
  requestTimeoutMs,
  workspaceRoot,
  memoryFile,
  memoryMaxEntries,
  memoryPromptChars,
}
```

Fingerprint rules

INPUT	OUTPUT
unset / empty	`not set`
length ≤ 12	all `*`, then `(N chars)`
longer	first 5 + `...` + last 4 + `(N chars)`

Model prefix map

PREFIX	VENDOR
`deepseek`	`deepseek`
`claude`	`anthropic`
`gpt`, `o1`, `o3`, `o4`, `o5`, `o8`, `chatgpt`	`openai`
`gemini`	`google`
`grok`	`x-ai`
`llama`	`meta-llama`
`qwen`	`qwen`
`mistral`, `mixtral`	`mistralai`

A name already containing `/` is left alone. A change is reported in the banner.

B. CLI and exit-code reference

Commands

COMMAND	WHAT IT DOES
`node src/cli.js`	Interactive session (requires a TTY).
`node src/cli.js "task"`	One-shot task.
`node src/cli.js --help`	Usage; no key required.
`node src/cli.js --demo`	Offline scripted run + scorecard.
`node src/cli.js --hooks`	Print the nine-hook catalogue; no key required.
`node src/cli.js --memory`	Print the store, digest and raw file; exit 0.
`node src/agent.js`	Prints "agent.js is the loop, not the program."
`npm test`	Offline test suite.
`npm run demo`	Alias for `--demo`.

Flags

FLAG	EFFECT
`--model <slug>`	Override the model for this run.
`--max-steps <n>`	Override the step budget.
`--yes`, `-y`	Auto-approve commands (policy `allow`).
`--allow-commands`	Alias for `--yes`.
`--no-memory`	Disable memory for this run.
`--no-hooks`	Silence hook traces.
`--trace-hooks`	Print hook traces (default on).
`--demo`	Use the scripted model.

In-session commands

COMMAND	EFFECT
`/new`	Clear the conversation; keep workspace and memory.
`/quit`, `/exit`	End the session.

Exit codes

CODE	MEANING	TYPICAL CAUSE
`0`	Normal completion	An answer, or a session ended normally.
`1`	Request or fatal failure	Bad key, no credits, timeout, non-JSON body, uncaught error.
`2`	No task	No argument and stdin is not a terminal.
`3`	Step limit	The loop exhausted `maxSteps` without an answer.

`taskExitCode(result)`:

```
if (!result.ok && result.reason === 'request_failed') return 1;
if (!result.ok && result.reason === 'max_steps') return 3;
return 0;
```

C. Tool response reference

The eight tools

TOOL	PARAMETERS	SUCCESS SHAPE	FAILURE SHAPES
`list_files`	`path` (default `.`), `depth` (default 3, max 6)	`<rel>/` for dirs, `<rel> (<n>b)` for files, then `<dirs> folder(s), <files> file(s) under <start.rel>`	`ERROR: no such folder: <rel>`
`read_file`	`path`, `offset` (default 1), `limit` (default 200, max 2000)	`"<rel> (<total lines>)"` + numbered lines + optional `... [<n> more lines]`	`ERROR: no such file: <rel>`; `ERROR: <rel> is a folder - use list_files`
`create_file`	`path`, `content`	`created <rel> (<bytes> bytes)`	`ERROR: already exists - use edit_file or write_file`; `FORBIDDEN: ...`
`write_file`	`path`, `content`	`created <rel> ...` or `overwrote <rel> ...`	`FORBIDDEN: ...`
`edit_file`	`path`, `old_string`, `new_string`, `replace_all` (default false)	`edited <rel>: <n> replacement(s), first at line <line>`	`ERROR: no such file`; `ERROR: old_string must not be empty`; `ERROR: old_string not found...`; `ERROR: old_string appears N times...`; `FORBIDDEN: ...`
`run_command`	`command`	`exit code <n>` + optional `stdout:` / `stderr:` + `(no output)`	`ERROR: command is required`; `DENIED: ...`; `FORBIDDEN: COMMAND ...`; timeout note
`remember`	`text`, `tag` (optional)	`remembered in <label> ...` or `already remembered ... date refreshed`	`ERROR: no memory store...`; `ERROR: nothing to remember - the note was empty`
`recall`	`query` (optional), `limit` (default 20)	Formatted entries	`ERROR: no memory store...`; `no memories in <label> match "<query>" (<size> stored)`

Observation prefixes

PREFIX	MEANING	LOOP BEHAVIOUR
`ERROR:`	The tool ran and failed in a way the model can fix.	Continue; the model sees the text.
`DENIED:`	Policy or a hook refused the action.	Continue; the model may choose another route.
`FORBIDDEN:`	The sandbox refused a path or shell write.	Continue; exactly one loud UI line was printed.
`exit code N`	A command ran.	Continue; `N` is the verdict.

`run_command` result format

```
exit code 0
stdout:
Hello from a ${agent}
```

```
exit code 1
stderr:
ReferenceError: agent is not defined
```

```
exit code 0
(no output)
```

Each stream is clipped at 6,000 characters with `... [<n> chars truncated]`. A timeout appends `[tool] command timed out after 30s` and returns non-zero.

D. Hook payload reference

Events and payloads

EVENT	PAYLOAD (ILLUSTRATIVE)	RETURN THAT CHANGES THE RUN
`SessionStart`	`{ task, workspace, config, log }`	`{ block: true, reason }`, `{ note }`
`UserPromptSubmit`	`{ task }`	`{ task: 'new' }`, `{ block: true, reason }`, `{ note }`
`PreModel`	`{ step, maxSteps, messages, tools }`	`{ note }` (observe)
`PostModel`	`{ step, message, finishReason }`	`{ note }` (observe)
`PreToolUse`	`{ name, args, step }`	`{ block: true, reason }`, `{ note }`
`PostToolUse`	`{ name, result, step, ok }`	`{ replace: 'text' }`, `{ note }`
`Stop`	`{ answer, steps, messages }`	`{ note }` (observe)
`StepLimit`	`{ steps, messages }`	`{ note }` (observe)
`SessionEnd`	`{ ok, reason, steps, durationMs, messages }`	`{ note }` (observe)

`Hooks.run` result

```
{
  event,      // the event name
  ran,        // how many handlers ran
  blocked,    // boolean
  reason,     // block reason, or null
  task,       // rewritten task, or null
  replace,    // replacement observation, or null
  notes,      // array of { note } values
  errors,     // array of handler errors (already reported, never fatal)
}
```

Deciding hooks

- `SessionStart` — refuse to start.
- `UserPromptSubmit` — rewrite the task or block.
- `PreToolUse` — block a tool call.
- `PostToolUse` — replace the observation.

First block wins. A throwing handler is reported and ignored. Every `tool_call_id` must still receive exactly one tool message.

E. File and API map

FILE	INTRODUCED	EXPORTS / RESPONSIBILITY	ANSWERS THE QUESTION
`src/ui.js`	1	`ui` helpers, `explainExit`	Where is the colour?
`src/config.js`	2	`PROJECT_ROOT`, `findEnvFile`, `loadDotEnv`, `fingerprint`, `normalizeModel`, `loadConfig`	Which key/model/limit, and where did it come from?
`src/llm.js`	2	`chat`, `sanitizeAssistant`	What goes over the wire?
`src/cli.js`	3	`askTask`, `MAX_HISTORY`, `interpretCommand`, `trimHistory`, `runSession`, `main` (internal: `parseArgs`, `taskExitCode`)	What does a person type?
`src/prompt.js`	5	`systemRules`, `buildSystemPrompt`, `TOOL_NAMES`	What is the model told?
`src/workspace.js`	6	`PathError`, `Workspace`, `resolve`, `resolveRead`	Is this path contained?
`src/tools.js`	7	`TOOL_SCHEMAS`, `TOOL_NAMES`, `runTool`, `safeEnv` (internal: `validate`, `HANDLERS`)	What can the model do?
`src/sandbox.js`	8	`SANDBOX_DIR`, `ACTIONS`, `ForbiddenError`, `Sandbox`, `tokenizeCommand`, `shellWriteTargets`	May the agent change this file at all?
`src/memory.js`	10	`MEMORY_FILE`, `MEMORY_TAGS`, `parseMemory`, `formatEntry`, `Memory`	What survives the process?
`src/hooks.js`	11	`HOOK_EVENTS`, `HOOK_NAMES`, `Hooks`, `traceHooks`, `printHookCatalogue`, `HOOK_SUMMARY`	Where can outside code get a turn?
`src/agent.js`	12	`runTask`, seven beats	Where is the loop?
`src/heartbeat.js`	12	`heartbeat`	Why doesn't a slow request look like a crash?
`src/demo.js`	13	`demoChat`, `DEMO_SCRIPT`, `sandboxScorecard`	What runs without a network?
`src/memory.md`	10	The store	What does it know?
`test/*.test.js`	15	Offline proof of every seam	Does it still work?
`README.md`, `demo_prompt.md`, `.env.example`	16	Docs and examples	How do I run and understand it?

The import direction

```
cli.js
  ├── agent.js ── prompt.js, llm.js, tools.js, hooks.js, heartbeat.js
  │                └── workspace.js, sandbox.js, memory.js
  ├── config.js
  ├── memory.js
  ├── demo.js
  └── ui.js
```

No arrow points from `agent.js` back to `cli.js`. `ui.js` is a leaf. `workspace.js` and `sandbox.js` do not import the loop.

F. Final acceptance checklist

The same twenty checks as Part II, Stage 16, repeated here as a sign-off sheet. A release is done when every box is ticked **on a clean checkout**.

Install and offline proof

- [ ] 1. `npm install` succeeds with only `chalk` as a runtime dependency.
- [ ] 2. `npm test` runs fully offline and all suites pass.
- [ ] 3. `node src/cli.js --help` prints usage with no key.
- [ ] 4. `node src/cli.js --hooks` prints the nine-hook catalogue with no key.
- [ ] 5. `node src/cli.js --memory` prints the store and the exact digest.
- [ ] 6. `node src/cli.js --demo` completes the script with **7/7 checks held**.

Live mode and boundaries

- [ ] 7. Live mode with a valid key runs a task through OpenRouter.
- [ ] 8. Live mode without a key prints the missing-key error and exits 1.
- [ ] 9. No task + no TTY exits 2.
- [ ] 10. A task that never yields an answer stops at the step limit and exits 3.

Safety

- [ ] 11. A write outside `workspace/` prints one `⛔ FORBIDDEN` line and returns a `FORBIDDEN:` observation without touching disk.
- [ ] 12. A read outside `workspace/` succeeds.
- [ ] 13. `CMD_POLICY=deny` refuses all commands; hard-denied commands are refused even with `--yes`.
- [ ] 14. The child shell cannot read `OPENROUTER_API_KEY`.

Persistence and observability

- [ ] 15. Memory survives restarts; duplicates refresh; the file is bounded; hand edits are picked up.
- [ ] 16. Every normal run prints `[HOOK:Name]` traces; `--no-hooks` silences them.

Sessions and architecture

- [ ] 17. `/new` clears conversation history but leaves workspace and memory untouched; `/quit` and `/exit` end the session.
- [ ] 18. A follow-up sees previous conversation but a freshly rebuilt system prompt with the latest memory.
- [ ] 19. `agent.js` executed directly prints the hint and does not run the CLI.
- [ ] 20. No framework, no agent SDK, no database, no hidden global state.

Sign-off

```
Project:
Student / team:
Date:
Node version:
Model used (live checks only):
Demo scorecard:            / 7
Tests:                     pass / fail
Acceptance:                / 20
Reviewed by:
Notes:
```

End of manual.